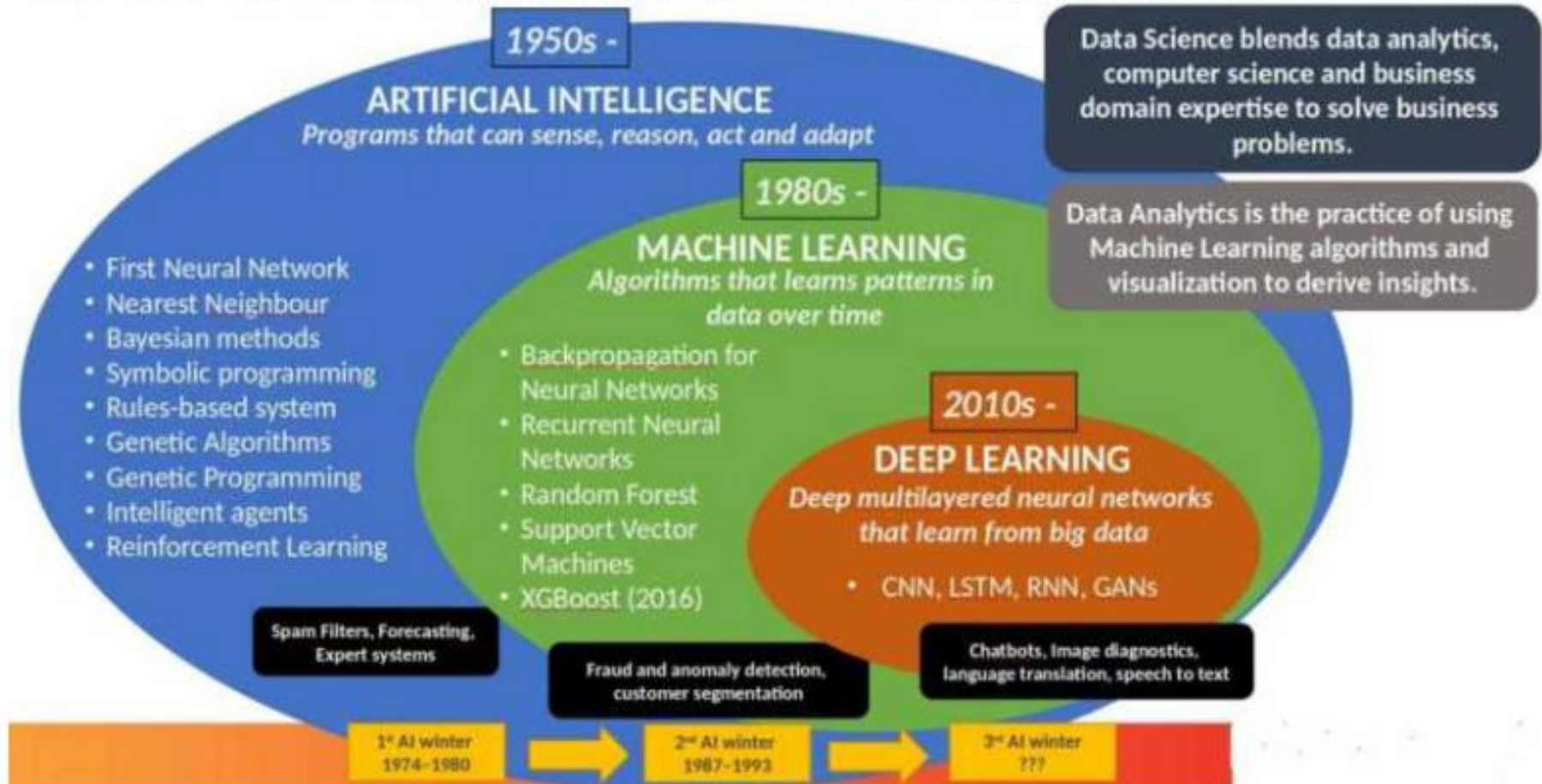


# AI & CHATBOT

Aula 17 – Aprendizado de Máquina  
Supervisionado:  
Algoritmos de Classificação

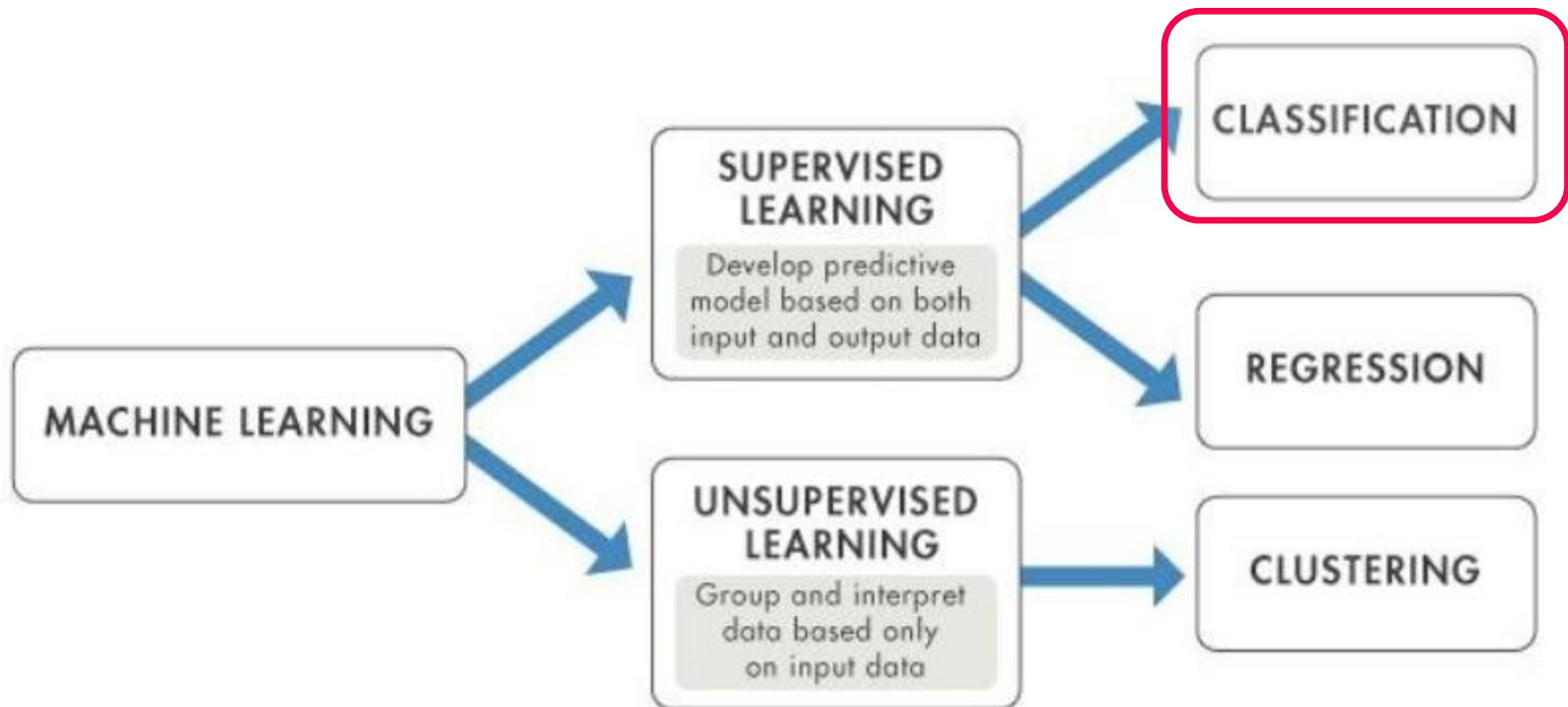
Prof. Henrique Ferreira

## AI VS ML VS DATA ANALYTICS VS DATA SCIENCE



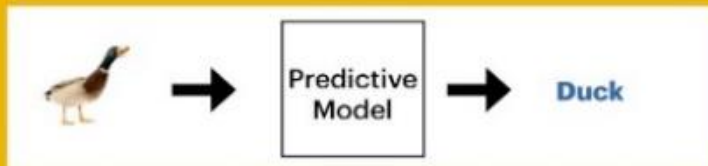
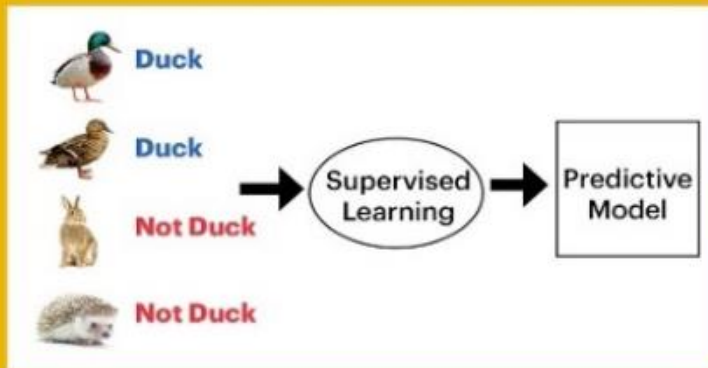
# Classificação

A tarefa de classificação é agrupada dentro dos algoritmos de aprendizado de máquina supervisionada.



# Classificação

## Supervised Learning (Classification Algorithm)



$$f(x_1, x_2, \dots, x_N)$$

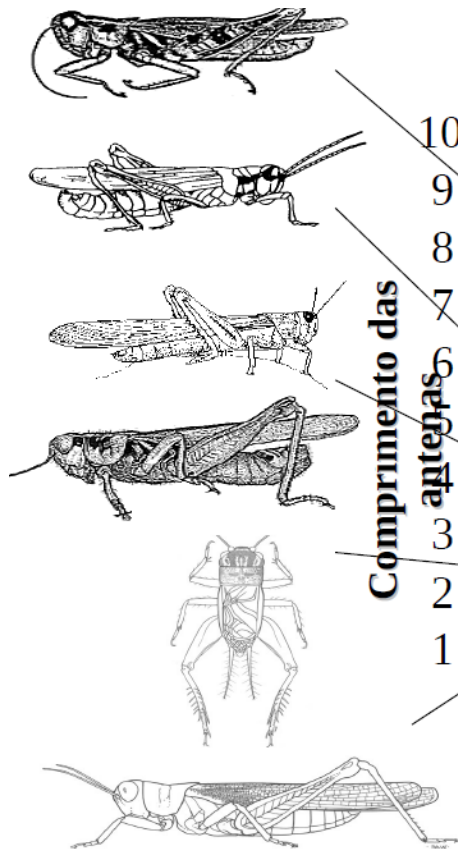
Atributos descritivos      Atributo alvo

| Índice da linha | x1     | x2       | ... | xn      | y         |
|-----------------|--------|----------|-----|---------|-----------|
| 1               | 548.4  | -9789    | ... | 0.4875  | Pato      |
| 2               | 689.4  | -10235   |     | -0.358  | Pato      |
| 3               | 3154.8 | -1031858 |     | -0.1458 | Tartaruga |
| ...             | ...    |          |     | ...     |           |
| k               | 803.54 | -20000   |     | 1.054   | Gato      |

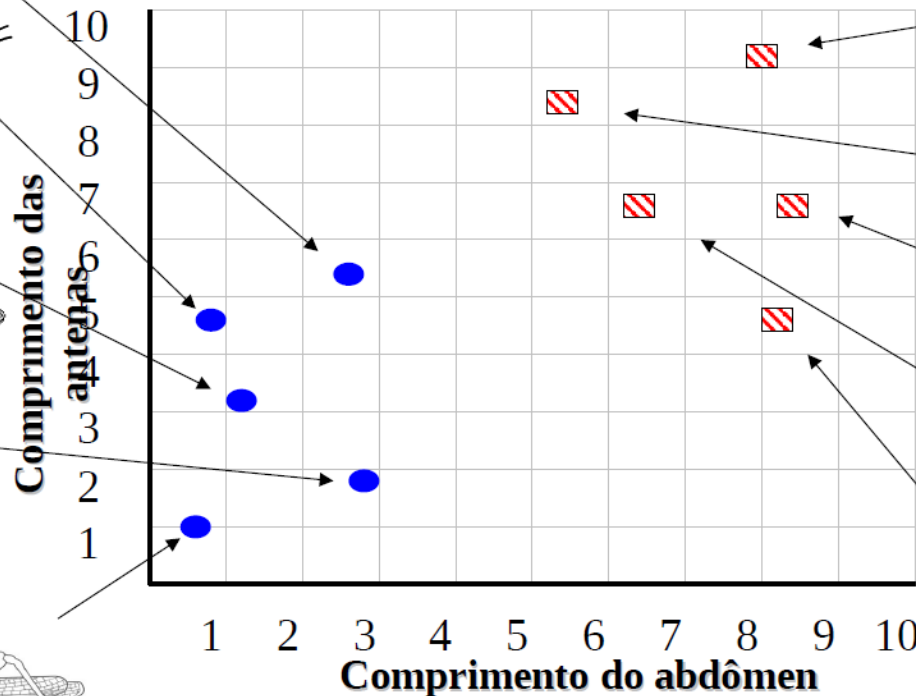
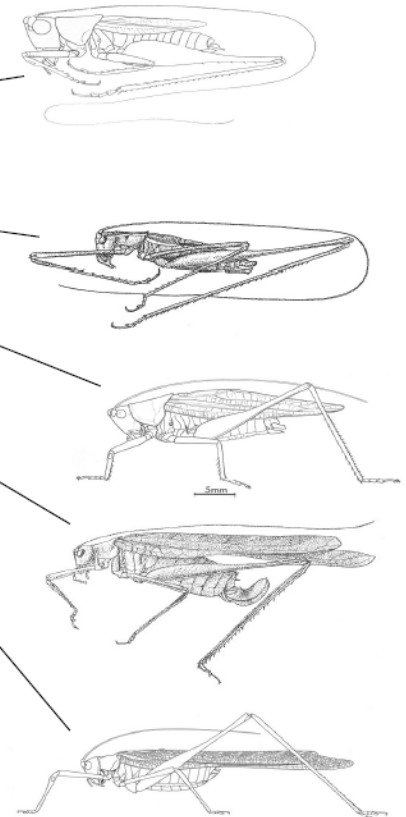
# Classificação

Classificação é a tarefa de **rotular** uma determinada observação usando como base os **atributos** observados.

## Gafanhotos



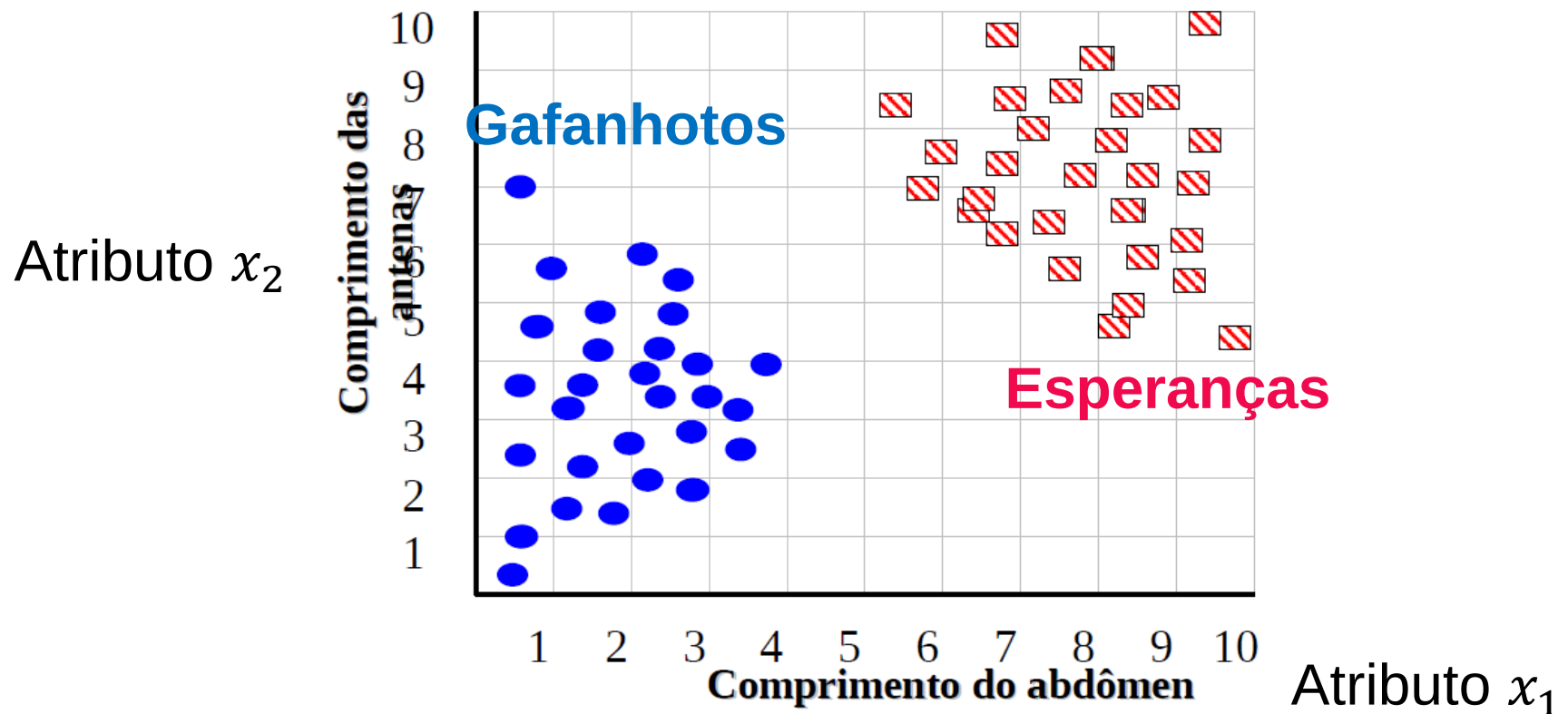
## Esperanças



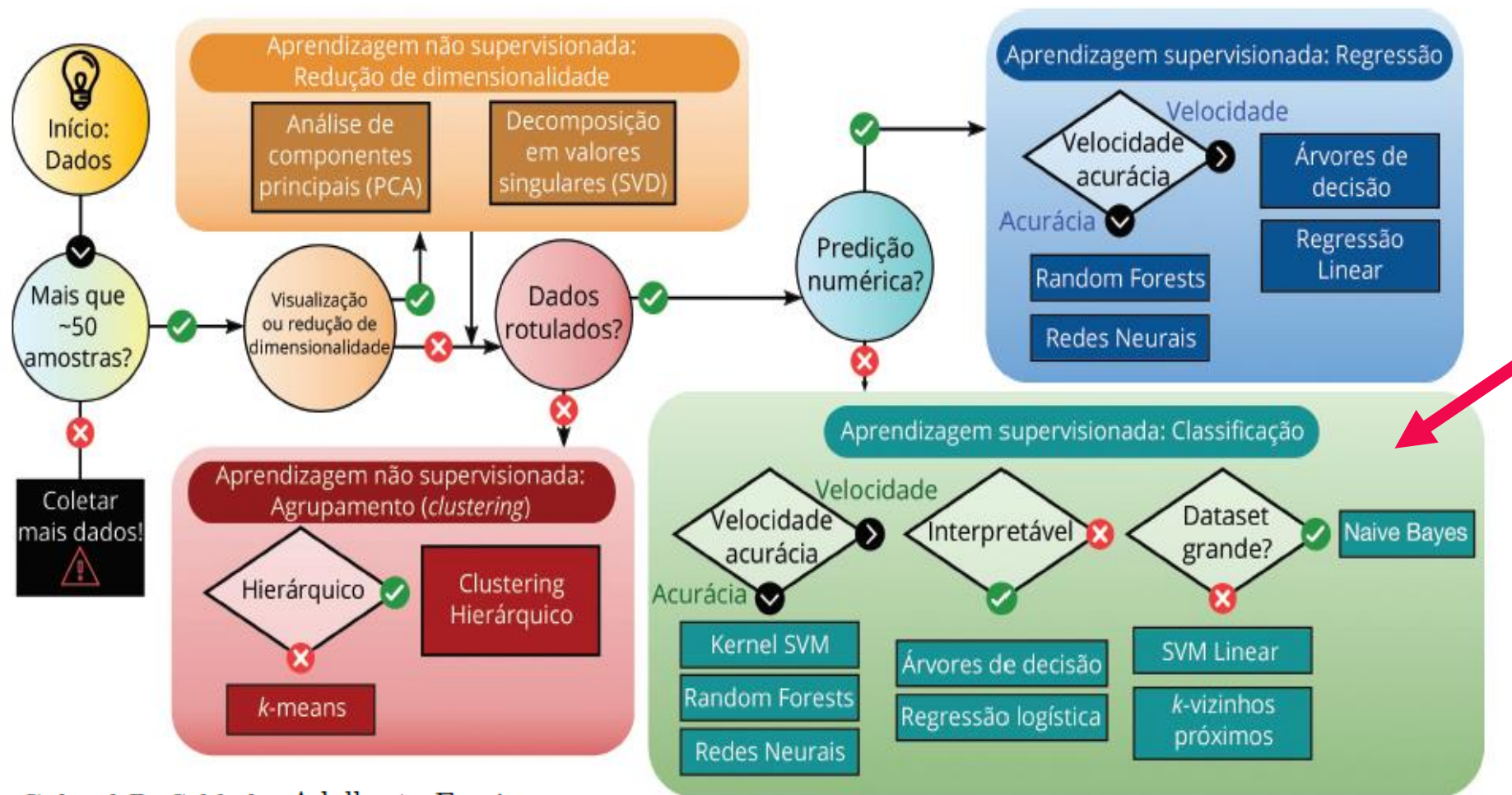


# Classificação

A classificação é uma tarefa supervisionada. Fornecemos pares de exemplos (observação, rótulo) para o algoritmo aprender como classificar. Lembre-se, a observação pode ser composta por vários atributos (features). Esses atributos são as diferentes colunas do nosso dataframe, i.e.  $(x_1, x_2, x_3, \dots, x_n)$ .

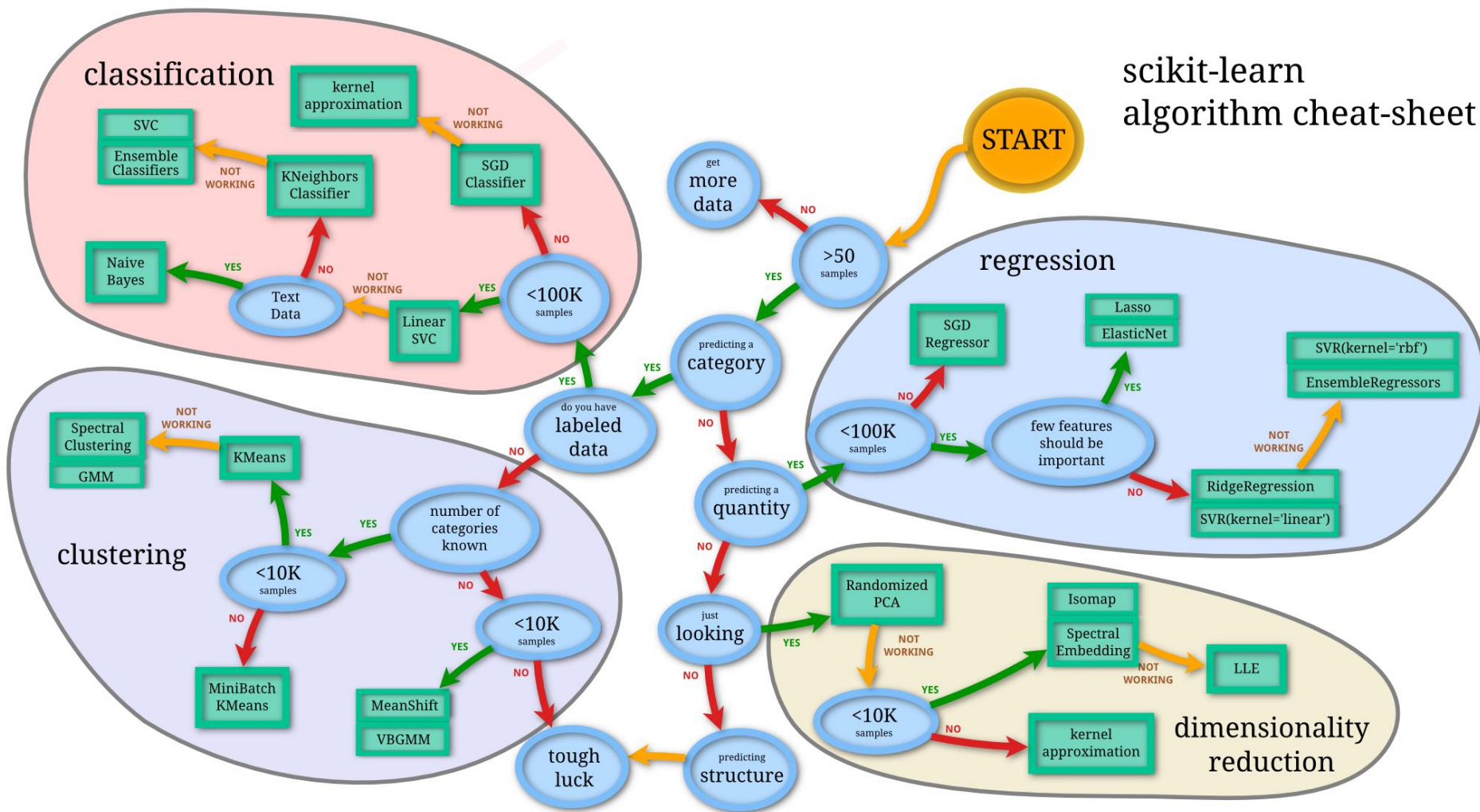


# Classificação



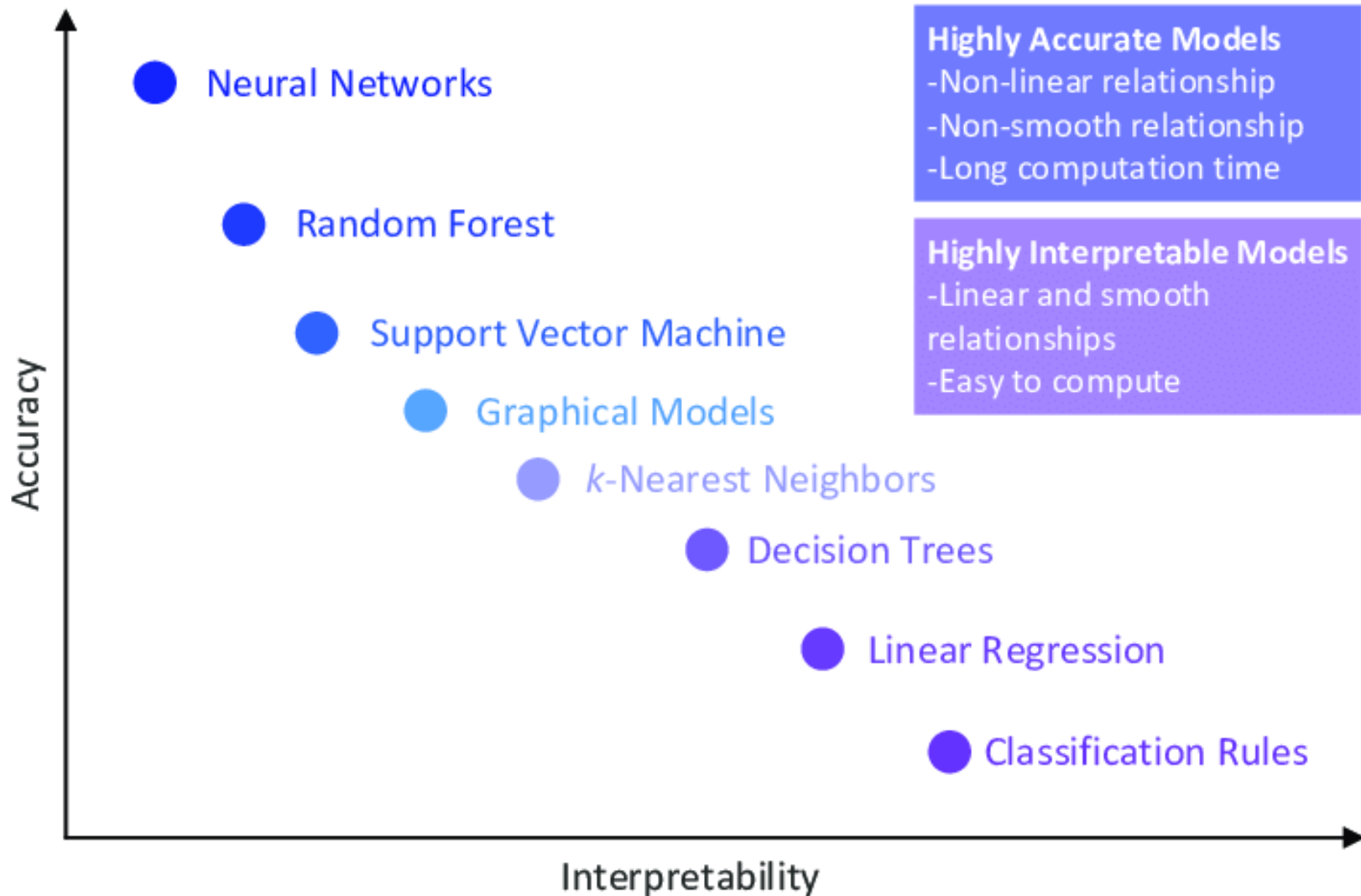
# Classificação

scikit-learn  
algorithm cheat-sheet

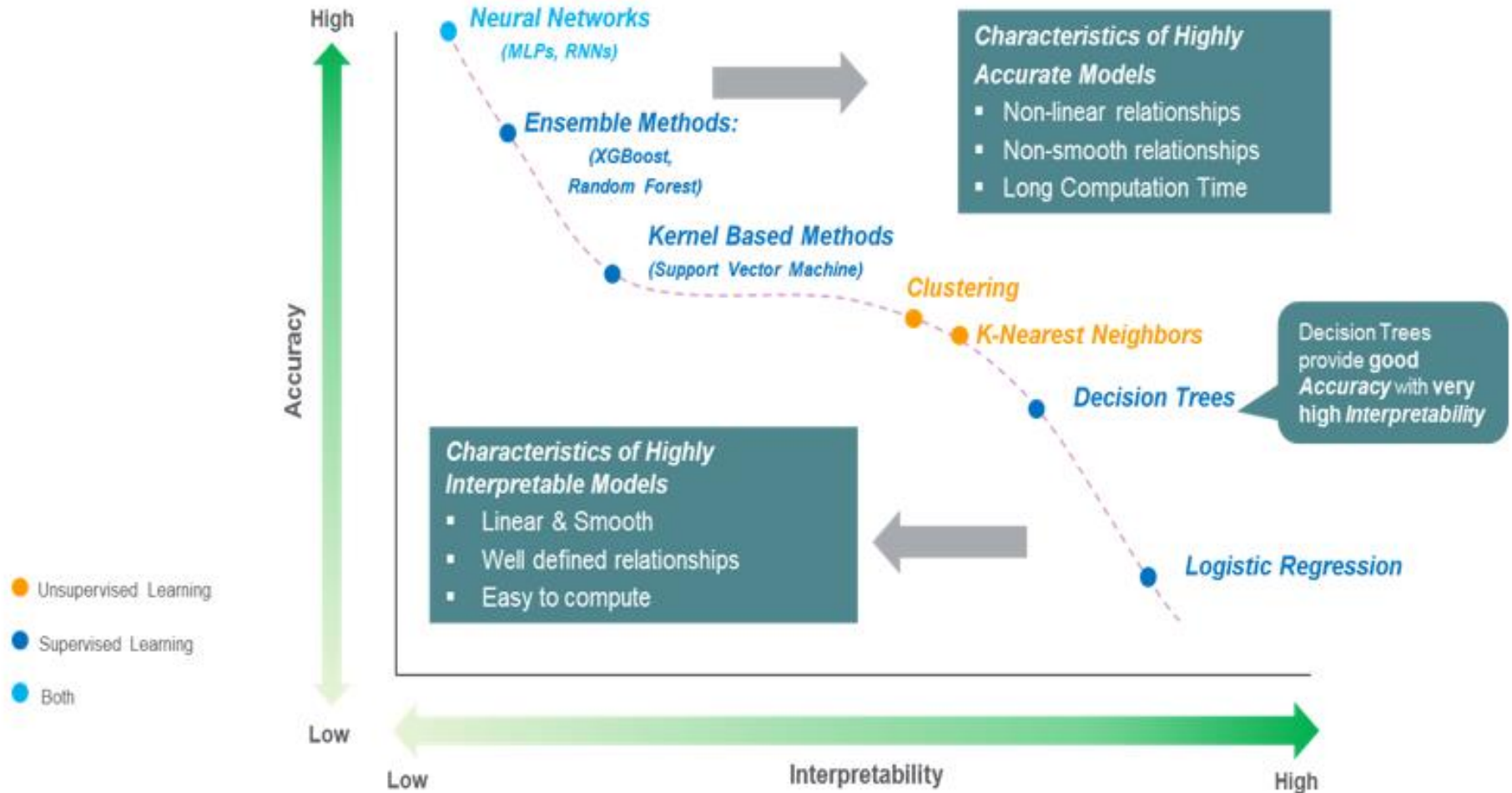




# Interpretabilidade vs Acurácia



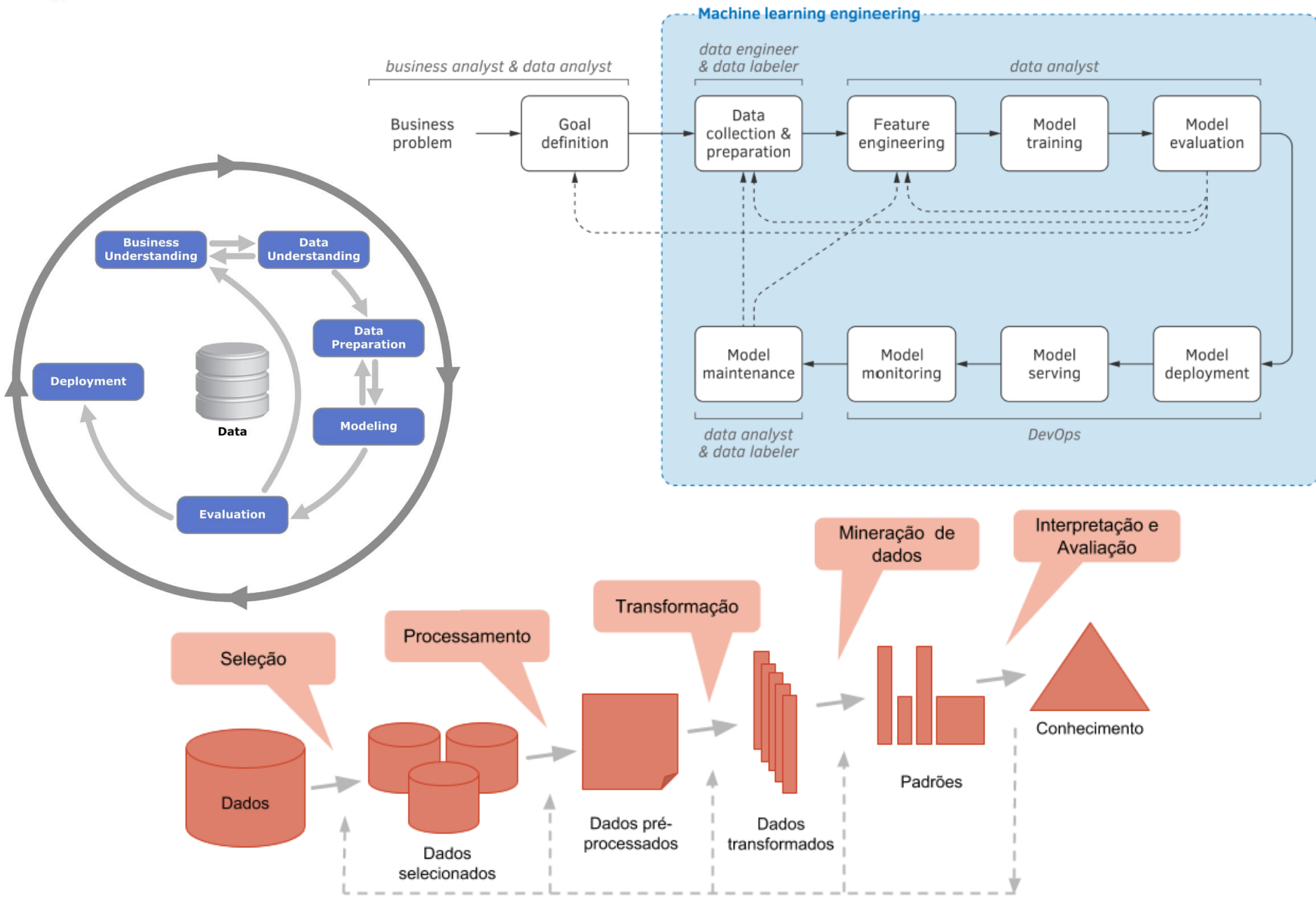
# Interpretabilidade vs Acurácia





# **METODOLOGIA DE TRABALHO COM ML SUPERVISIONADO**

# Metodologia de trabalho em ML





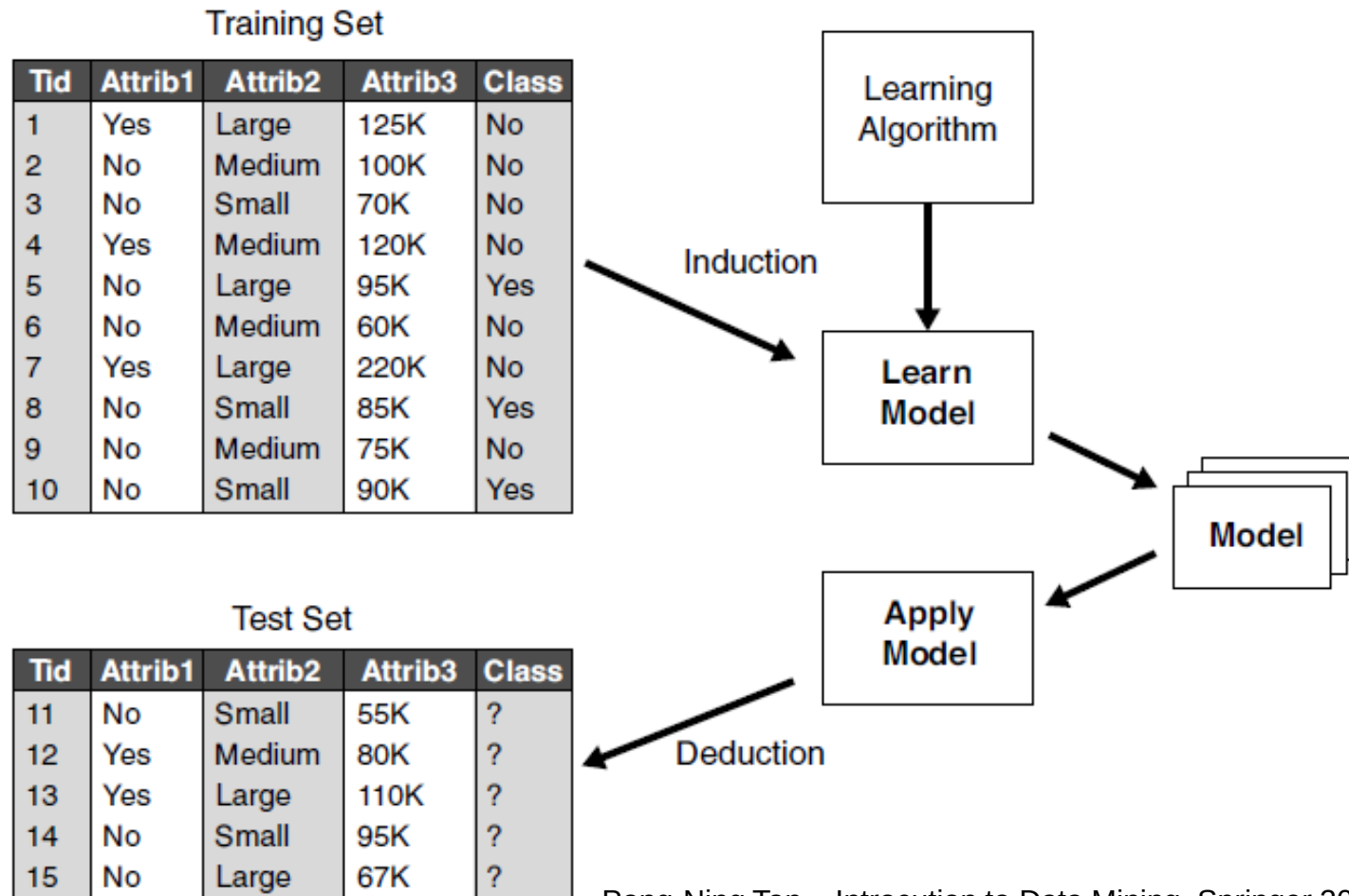
- **Primeiro passo:** carregar os dados e entendê-los (Análise Exploratória);
  - Durante a análise exploratória podemos realizar algumas limpezas e transformações de dados: retirar células vazias, remover outliers, ajustar dados categóricos (transformar em numéricos);
- **Segundo passo:** separar dados;
  - Nessa etapa devemos selecionar as colunas que serão features e a coluna que será alvo;
  - Separamos conjunto de treinamento, teste e validação quando estamos fazendo ML supervisionado;
- **Terceiro passo:** transformar os dados;
  - Escalonamento em dados numéricos ou Encodings em dados categóricos;

- **Quarto passo:** treinar o algoritmo
  - Escolher o algoritmo ou conjunto de algoritmos que serão usados;
  - Ajustar os hiperparâmetros dos algoritmos com dados de validação (isso pode ser feito em uma segunda etapa);
- **Quinto passo:** avaliar o desempenho
  - Escolher as métricas de desempenho que serão usadas;
  - Testar em cima dos dados de teste em ML supervisionado;
- **Sexto passo:** produção
  - Fazer o deploy do modelo na aplicação (nuvem, embarcado, etc);
  - Essa etapa envolve outras competências e áreas;

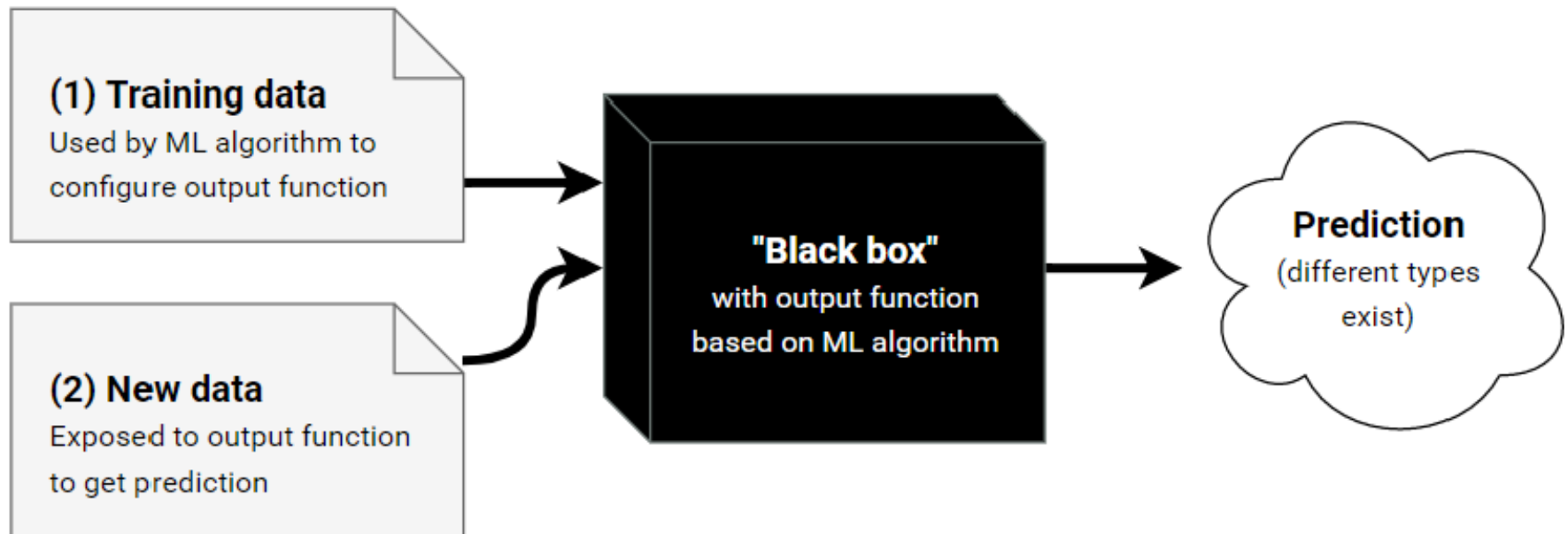


# SEPARAÇÃO DE DADOS

# Metodologia de trabalho em ML



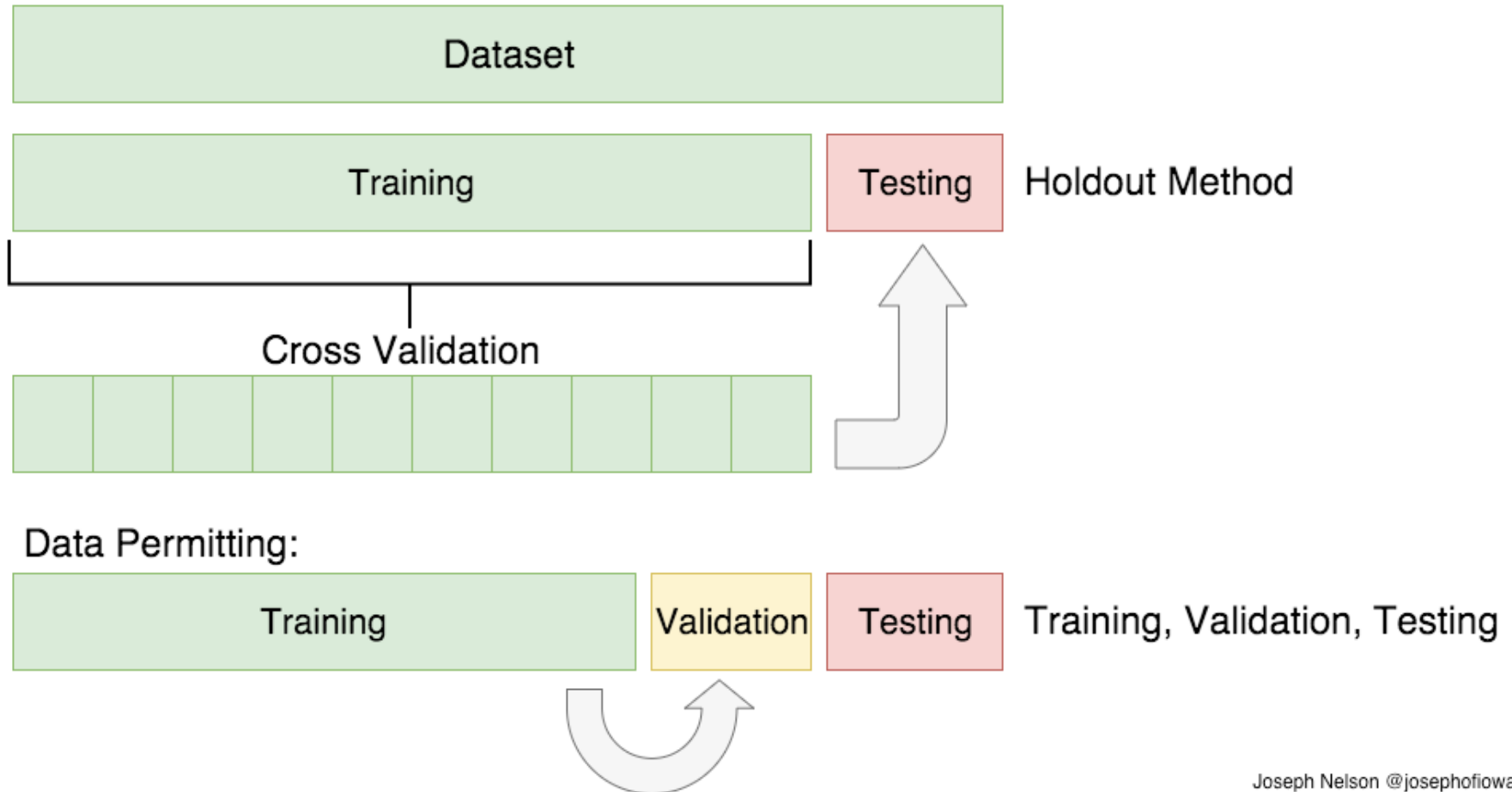
# Metodologia de trabalho em ML





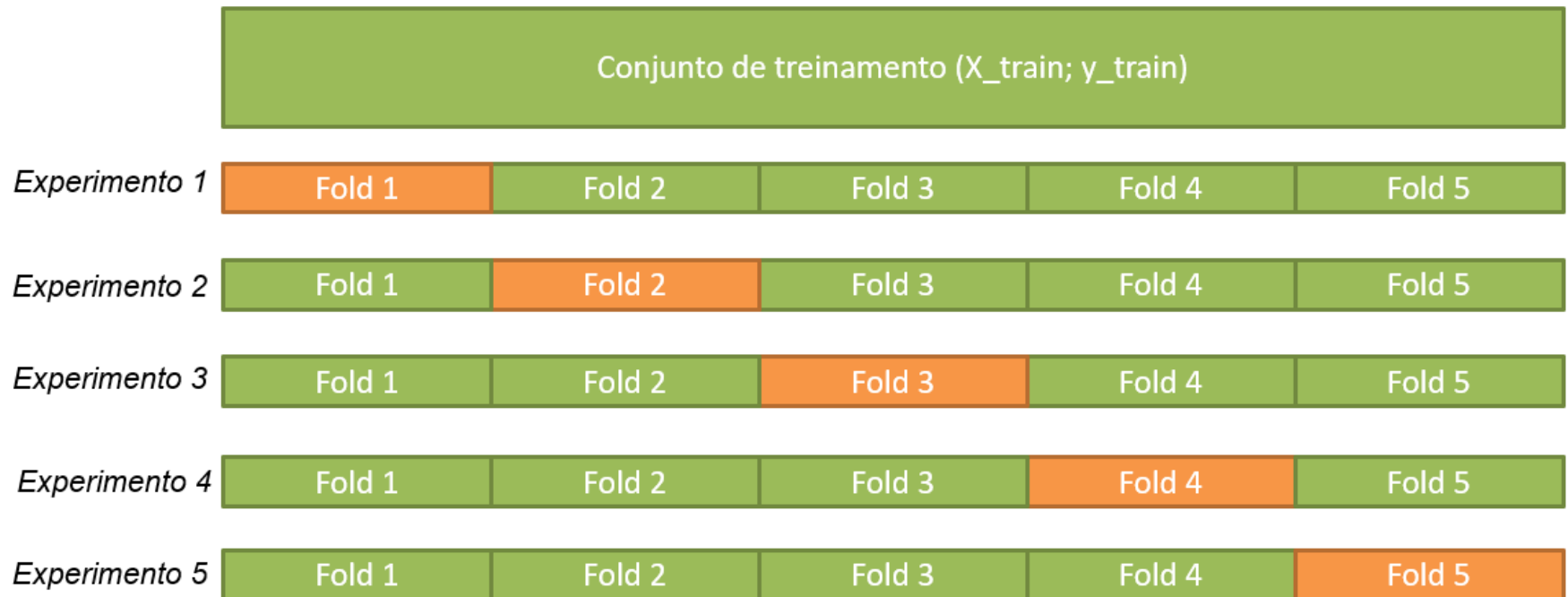
# Separação Treino/Teste

- A separação dos dados em dois conjuntos de treino e teste é uma metodologia de trabalho que visa permitir testar a performance do seu modelo minimizando as chances de overfitting;
- Existem várias técnicas de separação, como a 80/20 e a Cross Validation.



# Separação Treino/Teste

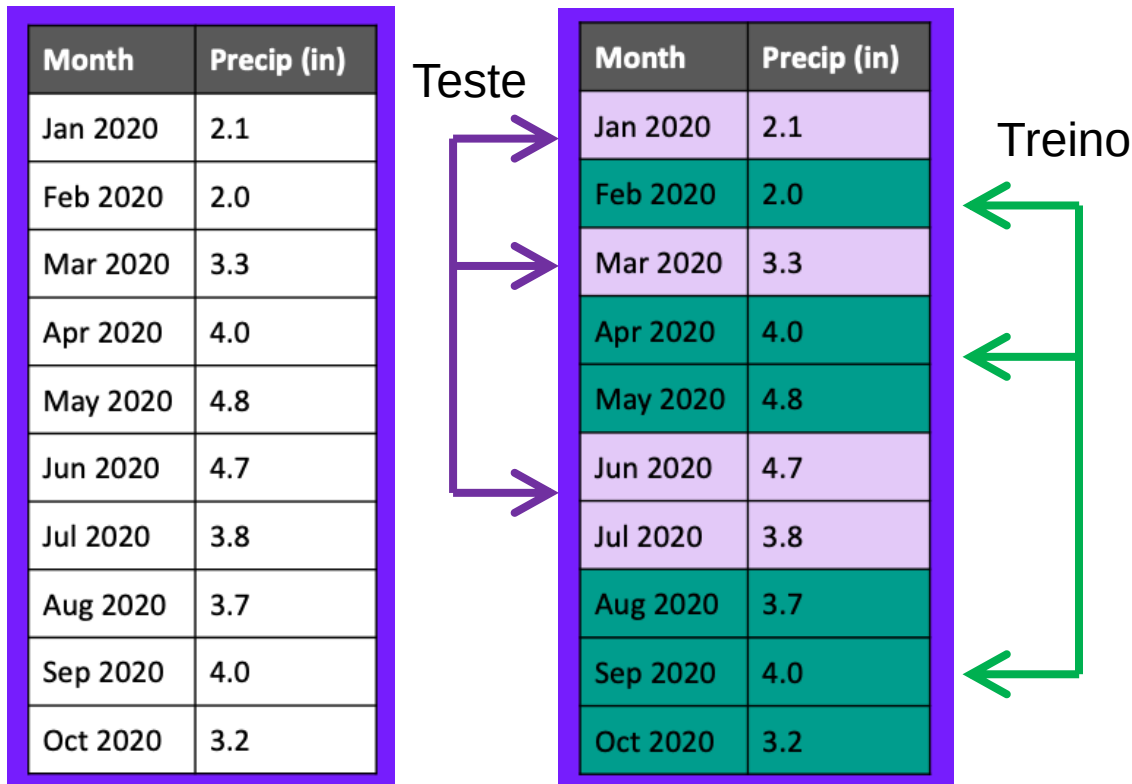
- Validação Cruzada (kfold)



# Separação Treino/Teste

- Contudo não podemos separar os dados sem antes entender do que se tratam os dados (na etapa de Análise Exploratória);
- Exemplo, dados de série temporal podem gerar problemas ao serem separados treino e teste:

Esse problema é chamado de Data Leakage (vazamento de dados) [8].

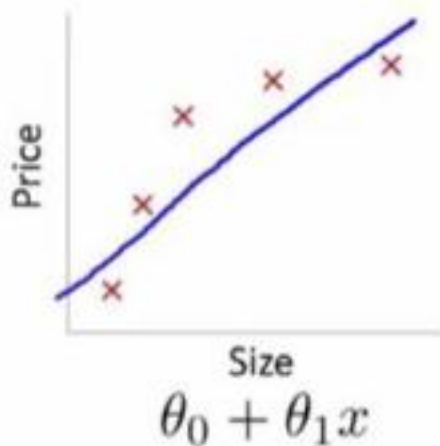


Uma forma de resolver é fazer o janelamento (amostragem progressiva) sobre os dados:

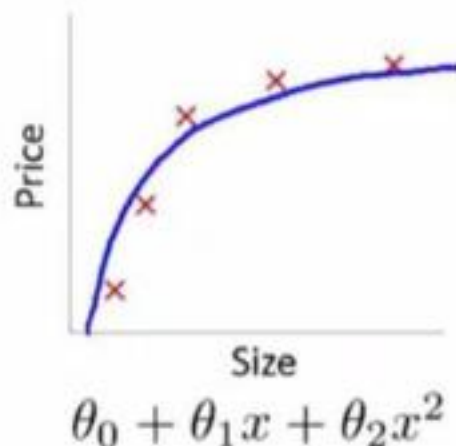


# Overfitting

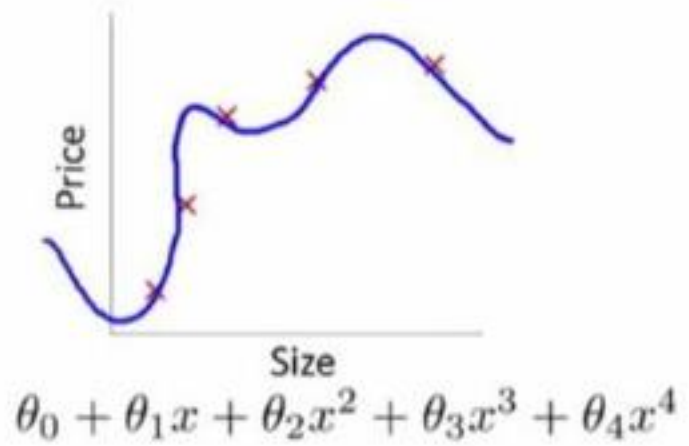
Um modelo com **overfitting** tem mais coeficientes do que o necessário. É um modelo com **pouca capacidade de generalização**: ele terá alta acurácia para os dados de treinamento e acurácia extremamente baixa para os dados de teste.



Viés alto  
(subajuste)



Ajuste de boa  
qualidade



Variância alta  
(superajuste)

# Overfitting

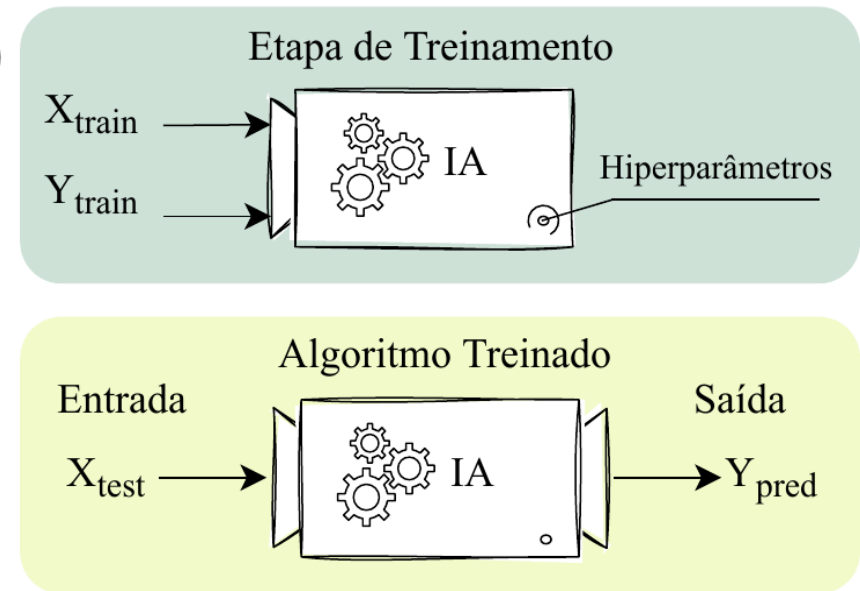
(a)

| Dados |       |       |     |       |       |     |       |     |
|-------|-------|-------|-----|-------|-------|-----|-------|-----|
|       | $x_1$ | $x_2$ | ... | $x_n$ | $y_1$ | ... | $y_m$ |     |
| Train | 1     | 1.2   | 21  | ...   | 0.27  | 1   | ...   | 4.5 |
|       | 2     | -4.2  | 44  | ...   | 0.74  | 0   | ...   | 5.0 |
|       | ...   | -3.7  | 30  | ...   | 0.11  | 2   | ...   | 3.7 |
|       | ...   | ...   | ... | ...   | ...   | ... | ...   | ... |
| Test  | k-1   | 7.1   | 14  | ...   | 0.34  | 2   | ...   | 2.8 |
|       | k     | -1.5  | 42  | ...   | 0.55  | 0   | ...   | 4.2 |

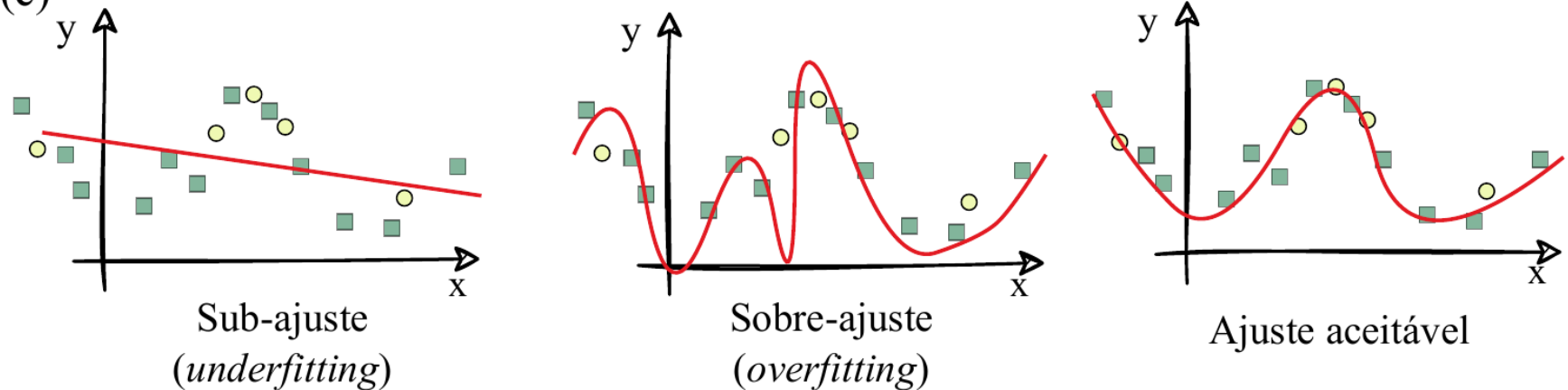
Features

Saída

(b)

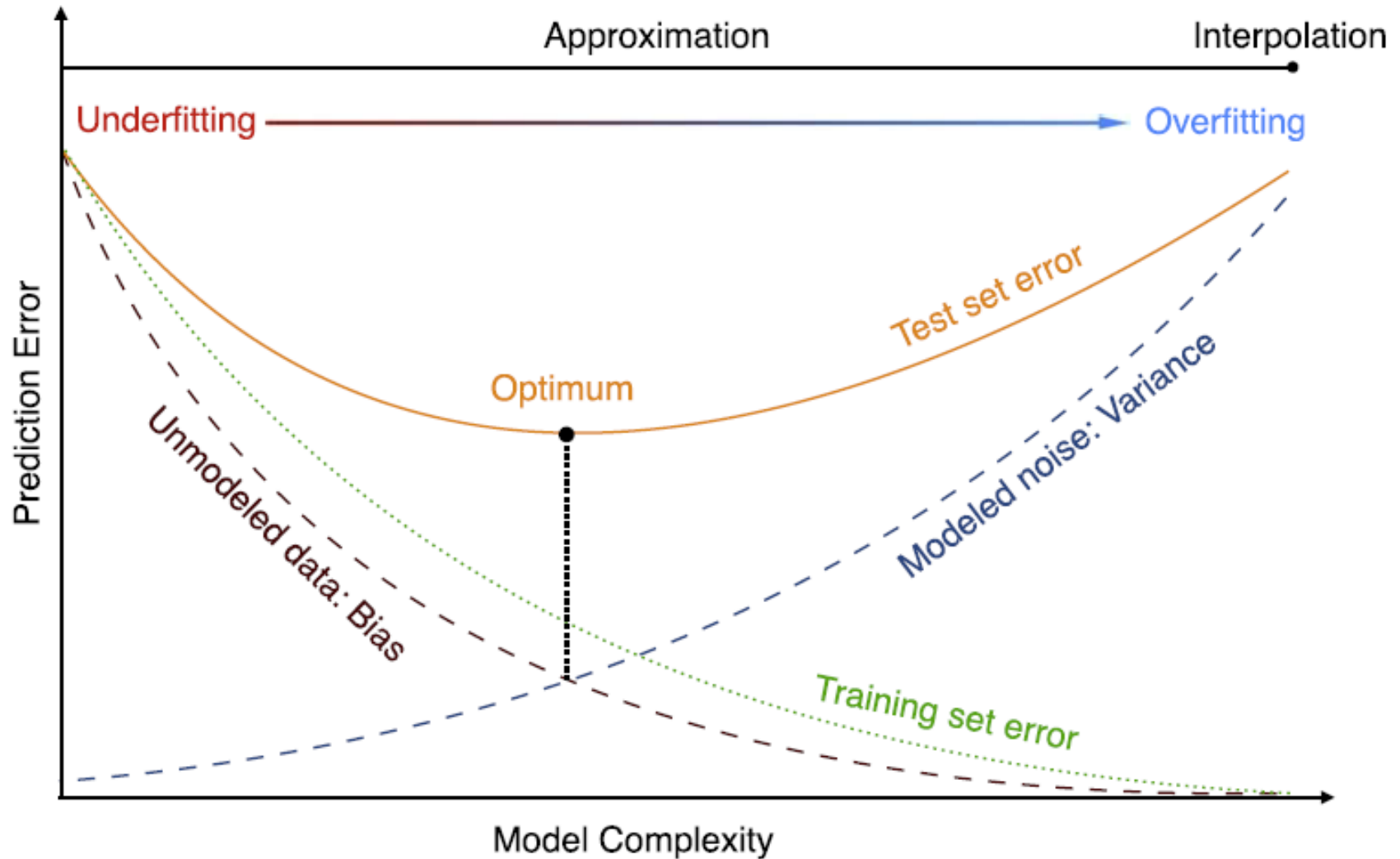


(c)



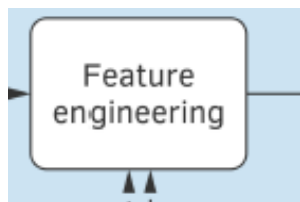
■ Dados de treinamento      ● Dados de teste

# Overfitting



# Seleção de Features

- Outra etapa importante durante o trabalho em Aprendizado de Máquina é a Seleção de Features (colunas) que serão usadas;
- É importante nunca passar para o algoritmo de ML uma coluna alvo dentro das features selecionadas (seria como se o enunciado da questão tivesse a alternativa correta). Observe se a coluna alvo não foi obtida de outra coluna do dataframe, caso tenha sido, é importante não passar nenhuma das duas como o conjunto X de features usadas;



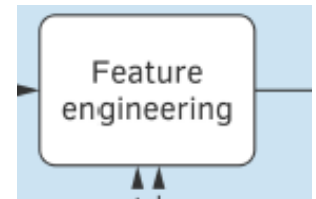
|              | Features  |          |           | Alvo      |
|--------------|-----------|----------|-----------|-----------|
|              | Age       | Altura   | Peso      | BMI       |
| Faixa Etária | $x_1$     | $x_2$    | $x_3$     | $y$       |
| 15-20        | 18.312200 | 1.798929 | 72.147214 | 22.279081 |
| 20-25        | 22.021567 | 1.810217 | 74.359530 | 22.672084 |
| 25-30        | 26.831556 | 1.816001 | 76.298422 | 23.109146 |
| 30-35        | 31.559215 | 1.816906 | 76.954611 | 23.283090 |
| 35-40        | 36.075269 | 1.825914 | 78.768323 | 23.601338 |
| 40-45        | 40.909091 | 1.835455 | 80.243182 | 23.793042 |

$$BMI = \frac{W}{H^2}$$



# Seleção de Features

- As vezes temos  **muitas colunas de dados** : nestes casos os algoritmos podem ficar lentos tentando avaliar todas as características, e a aplicação dos algoritmos torna-se complicada (precisa-se de muita informação para cada nova previsão do algoritmo). A solução é aplicar técnicas de redução de dimensionalidade e seleção de features;
- A  **seleção de features**  é feita observando-se a importância da feature para o desempenho do algoritmo de ML: features que impactam pouco o modelo são descartas enquanto aqueles que impactam mais, são mantidas;
- A  **redução de dimensionalidade**  pode ser feita usando-se outro algoritmo de ML que irá descobrir quais features são importantes ou como combinar as colunas para criar apenas uma (por exemplo, como fazer 8 colunas virarem 2 sem perder muita informação).





# TRANSFORMAÇÕES DE DADOS

# Label Encoding e One Hot Encoding

- Quando conseguimos ordenar os dados usamos o Label Encoding
- Quando não conseguimos ordenar os dados usamos o One Hot Encoding

## Label Encoding

| SAFETY-LEVEL<br>(TEXT) | SAFETY-LEVEL<br>(NUMERICAL) |
|------------------------|-----------------------------|
| None                   | 0                           |
| Low                    | 1                           |
| Medium                 | 2                           |
| High                   | 3                           |
| Very-High              | 4                           |

## One Hot Encoding

| Color  |   |   |   |
|--------|---|---|---|
| Red    | 1 | 0 | 0 |
| Red    | 1 | 0 | 0 |
| Yellow | 0 | 1 | 0 |
| Green  | 0 | 0 | 1 |
| Yellow | 0 | 1 | 0 |

# Normalização e Padronização

- Alguns modelos de Machine Learning exigem que os valores estejam em escalas similares para que eles não se tornem tendenciosos. Por exemplo:
  - Se temos o peso, altura e o tamanho da camisa que uma pessoa usa. Podemos tentar usar esses dados para estimar qual o tipo de camisa uma pessoa pode comprar
  - Para isso, o nosso modelo recebe os valores do peso e da altura e estima a saída de tamanho da camisa.

| id | Peso | Altura(m) | Camisa |
|----|------|-----------|--------|
| 1  | 75   | 1,77      | G      |
| 2  | 80   | 1,85      | G      |
| 3  | 92   | 1,8       | G      |
| 4  | 67   | 1,69      | M      |
| 5  | 88   | 1,9       | GG     |
| 6  | 105  | 1,95      | GG     |

A escala do peso é muito maior que a altura!

# Normalização e Padronização

- Caso as grandezas dos dados envolvidos forem muito diferentes, podemos padronizar ou normalizar os nossos dados:

Sklearn: `MinMaxScaler()`

- Normalização:  $X_{changed} = \frac{X - X_{min}}{X_{max} - X_{min}}$

Onde:

$X_{changed}$  é o valor normalizado

$X$  é o valor antes da normalização

$X_{min}$  é o menor valor do conjunto de dados

$X_{max}$  é o maior valor do de dados

Sklearn: `StandardScaler()`

- Padronização:

$$z_i = \frac{x_i - \mu}{\sigma}$$

Onde:

$z_i$  é o i-ésimo valor padronizado;

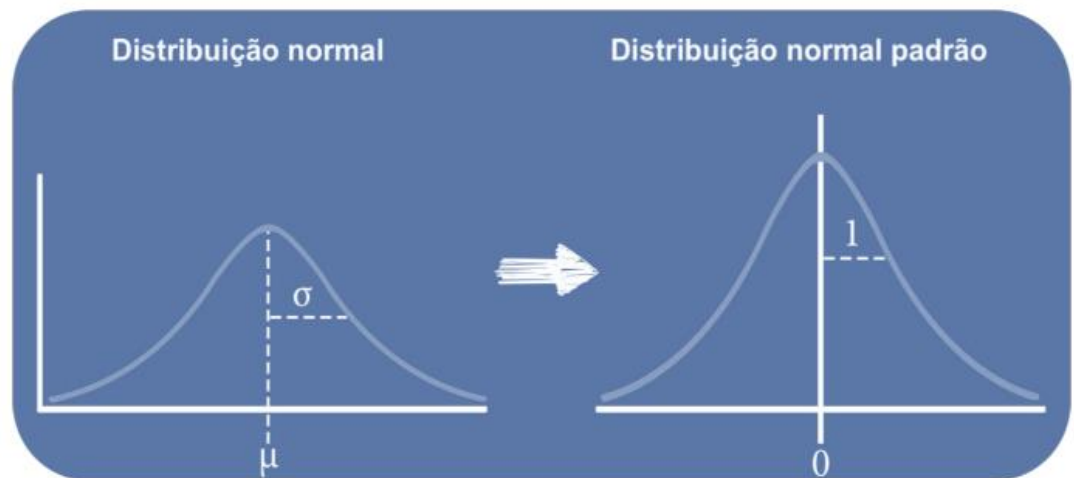
$x_i$  é o i-ésimo valor original

$\sigma$  é o desvio padrão dos dados.

$\mu$  é a média dos dados

# Normalização e Padronização

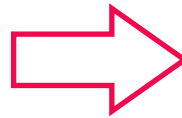
- A normalização pode ser aplicada quando a distribuição dos dados não é normal ou se o desvio padrão dos mesmos for muito pequeno;
- A normalização permite comparar a importância das features quando usamos modelos paramétricos como Regressão Linear e Regressão Logística;
- A padronização dos dados transforma os mesmos em uma distribuição normal padrão;
- A padronização é recomendada quando os dados estão em uma distribuição normal;



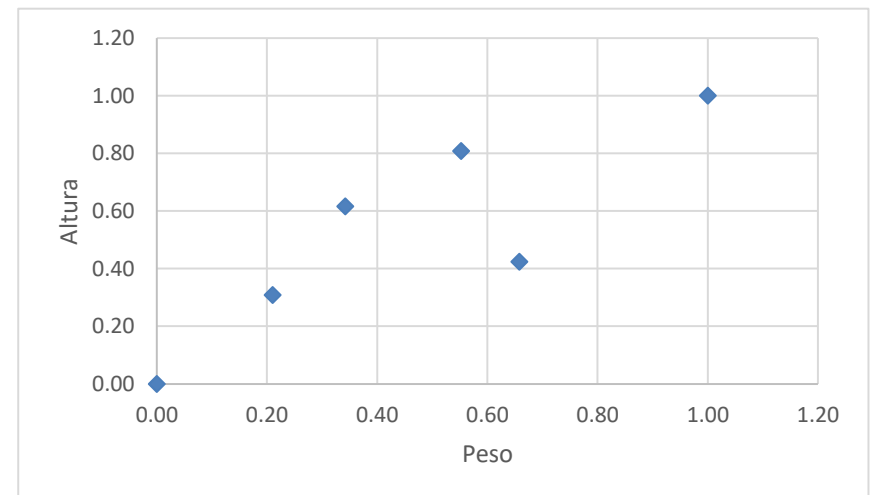
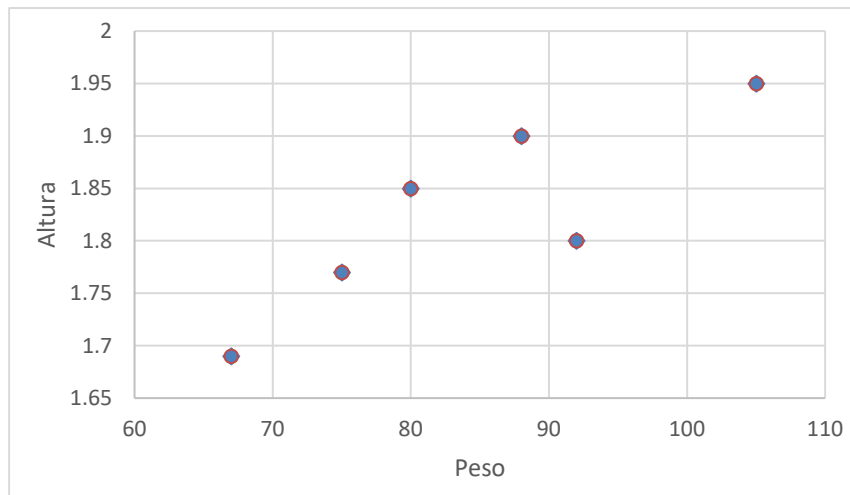
# Normalização e Padronização

- Após a **normalização** dos dados:

| id | Peso | Altura(m) | Camisa |
|----|------|-----------|--------|
| 1  | 75   | 1,77      | G      |
| 2  | 80   | 1,85      | G      |
| 3  | 92   | 1,8       | G      |
| 4  | 67   | 1,69      | M      |
| 5  | 88   | 1,9       | GG     |
| 6  | 105  | 1,95      | GG     |



| id | Peso | Altura(m) | Camisa |
|----|------|-----------|--------|
| 1  | 0,21 | 0,31      | G      |
| 2  | 0,34 | 0,62      | G      |
| 3  | 0,66 | 0,42      | G      |
| 4  | 0,00 | 0,00      | M      |
| 5  | 0,55 | 0,81      | GG     |
| 6  | 1,00 | 1,00      | GG     |

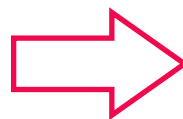




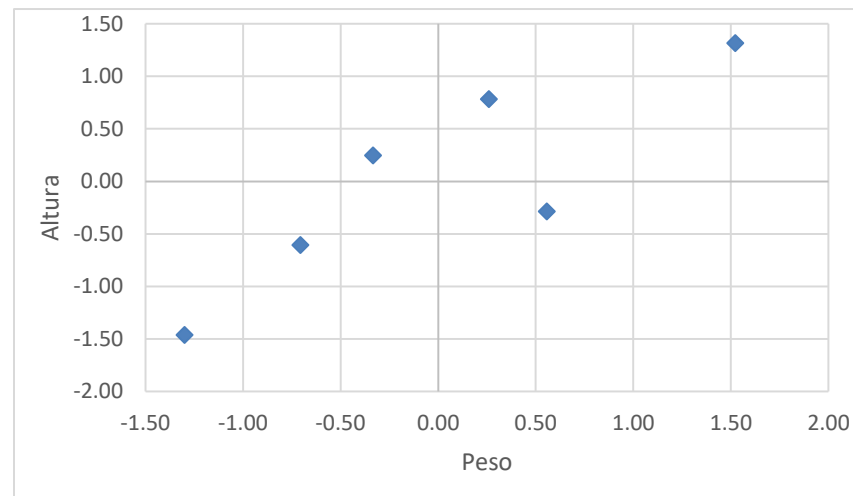
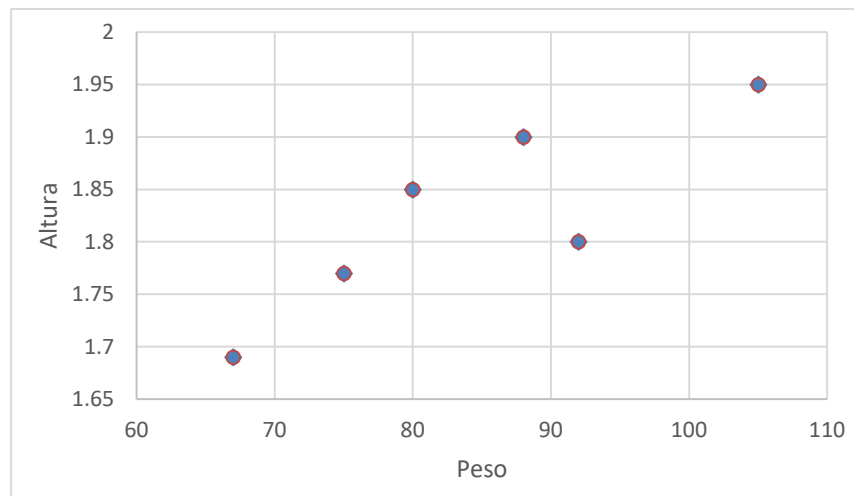
# Normalização e Padronização

- Após a **padronização** dos dados:

| id | Peso | Altura(m) | Camisa |
|----|------|-----------|--------|
| 1  | 75   | 1,77      | G      |
| 2  | 80   | 1,85      | G      |
| 3  | 92   | 1,8       | G      |
| 4  | 67   | 1,69      | M      |
| 5  | 88   | 1,9       | GG     |
| 6  | 105  | 1,95      | GG     |



| id | Peso  | Altura(m) | Camisa |
|----|-------|-----------|--------|
| 1  | -0,71 | -0,61     | G      |
| 2  | -0,33 | 0,25      | G      |
| 3  | 0,56  | -0,29     | G      |
| 4  | -1,30 | -1,46     | M      |
| 5  | 0,26  | 0,78      | GG     |
| 6  | 1,52  | 1,32      | GG     |



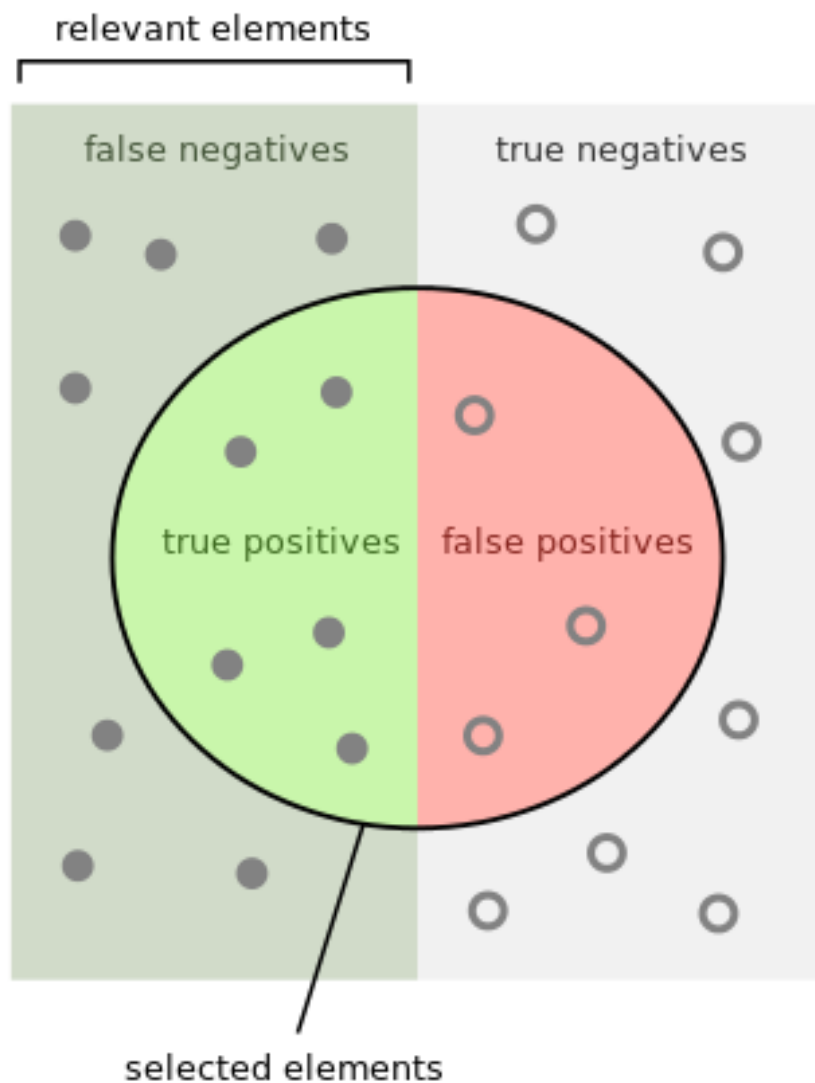


# **MÉTRICAS DE DESEMPENHO DE CLASSIFICAÇÃO**

- Na classificação temos dados **categóricos**. Nossa predição é uma string portanto. Como medir o desempenho do classificador?
- Vamos contabilizar dados  $k$  exemplos, quantos colocamos nas classes certas; Se dividirmos o número de acertos pelo total de exemplos testados, já temos uma métrica de desempenho;
- Entretanto essa métrica pode ser muito simplista. Podemos usar métricas mais sofisticadas;
- Algumas métricas possíveis:
  - Matriz de confusão
  - Precisão e Recall
  - Acurácia
  - F1 score
  - Curva ROC

- **Verdadeiro Positivo** (True Positive - TP): que é da classe e foi corretamente classificado;
- **Verdadeiro Negativo** (True Negative - TN): que não é da classe e não foi classificado dentro da classe;
- **Falso Positivo** (False Positive - FP): que não é da classe, mas foi classificado dentro dela;
- **Falso Negativo** (False Negative - FN): que é da classe, mas foi classificado fora dela;

# Precisão e Recall



$$precisão \stackrel{\text{def}}{=} \frac{TP}{TP + FP}$$

How many selected items are relevant?

$$\text{Precision} = \frac{\text{Green Semi-circle}}{\text{Green Semi-circle} + \text{Red Semi-circle}}$$

How many relevant items are selected?

$$\text{Recall} = \frac{\text{Green Semi-circle}}{\text{Green Semi-circle} + \text{Green Rectangle}}$$

$$revocação \stackrel{\text{def}}{=} \frac{TP}{TP + FN}$$

# Matriz de Confusão

|                             |                 | <i>gold standard labels</i>        |                       |                                               |
|-----------------------------|-----------------|------------------------------------|-----------------------|-----------------------------------------------|
|                             |                 | gold positive                      | gold negative         |                                               |
| <i>system output labels</i> | system positive | <b>true positive</b>               | <b>false positive</b> | <b>precision</b> = $\frac{tp}{tp+fp}$         |
|                             | system negative | <b>false negative</b>              | <b>true negative</b>  |                                               |
|                             |                 | <b>recall</b> = $\frac{tp}{tp+fn}$ |                       | <b>accuracy</b> = $\frac{tp+tn}{tp+fp+tn+fn}$ |

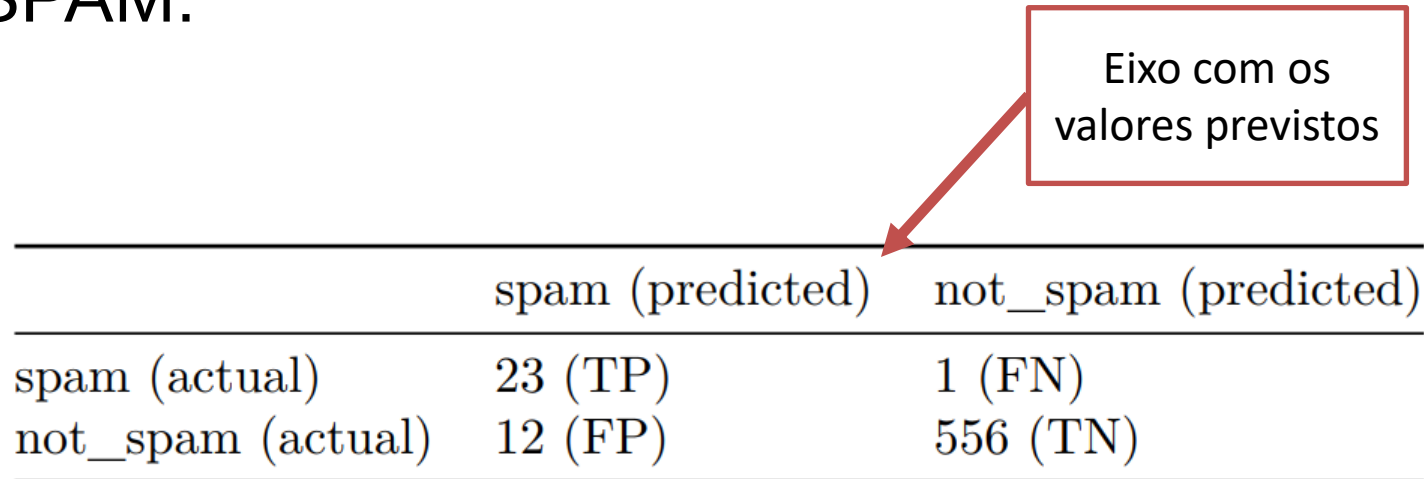
# Matriz de confusão

- Exemplo de matriz de confusão de e-mails de SPAM:

|                   |                          |          |                          |  |
|-------------------|--------------------------|----------|--------------------------|--|
|                   | Verdadeiro Positivo - TP |          | Falso Negativo - FN      |  |
|                   | spam (predicted)         |          | not_spam (predicted)     |  |
| spam (actual)     | 23 (TP)                  | 1 (FN)   |                          |  |
| not_spam (actual) | 12 (FP)                  | 556 (TN) |                          |  |
|                   | Falso Positivo - FP      |          | Verdadeiro Negativo - TN |  |

# Matriz de confusão

- Exemplo de matriz de confusão de e-mails de SPAM:



|                   | spam (predicted) | not_spam (predicted) |
|-------------------|------------------|----------------------|
| spam (actual)     | 23 (TP)          | 1 (FN)               |
| not_spam (actual) | 12 (FP)          | 556 (TN)             |

Eixo com os  
valores previstos

Eixo para os valores  
esperados de  
classificação



# Matriz de confusão

- Exemplo de matriz de confusão de e-mails de SPAM:

|                   | spam (predicted) | not_spam (predicted) |
|-------------------|------------------|----------------------|
| spam (actual)     | 23 (TP)          | 1 (FN)               |
| not_spam (actual) | 12 (FP)          | 556 (TN)             |

24 e-mails expostos ao modelo  
que realmente são SPAMS

# Matriz de confusão

- Exemplo de matriz de confusão de e-mails de SPAM:

|                   | spam (predicted) | not_spam (predicted) |
|-------------------|------------------|----------------------|
| spam (actual)     | 23 (TP)          | 1 (FN)               |
| not_spam (actual) | 12 (FP)          | 556 (TN)             |

568 e-mails expostos ao  
modelo que NÃO SÃO  
SPAMS

# Matriz de confusão

Exemplo de matriz de confusão de e-mails de SPAM:

$$\begin{aligned} \text{acurácia} &= \frac{TP + TN}{TP + TN + FP + FN} = \frac{23 + 556}{23 + 556 + 12 + 1} = \\ &= \frac{579}{592} = 97,80\% \end{aligned}$$

$$\text{precisão} = \frac{TP}{TP + FP} = \frac{23}{23 + 12} = \frac{23}{35} = 65,71\%$$

$$\text{revocação} = \frac{TP}{TP + FN} = \frac{23}{23 + 1} = \frac{23}{24} = 95,83\%$$

# Matriz de confusão

- No nosso exemplo de SPAM desejamos ter uma precisão maior ou um recall maior?
  - Optamos pela acurácia como métrica quando todas as classes são importantes;
  - Caso optarmos por um modelo de **alta precisão**, ele vai separar da nossa caixa de entrada e enviar para a pasta de SPAM poucos e-mails relevantes (teremos poucos falsos positivos para SPAM);
  - Caso optarmos por um modelo de **alta revocação**, ele vai separar da nossa caixa de entrada e enviar para a pasta de SPAM a maioria dos e-mails que é realmente um SPAM (teremos poucos falsos negativos para SPAM);
- É mais importante classificar corretamente e-mails legítimos do que e-mails de spam;
- Melhor ter e-mails relevantes na sua caixa de entrada!
- Logo podemos dizer que nesse caso é melhor que o modelo tenha alta precisão;

- Quando o problema não é tão simples de ser analisado, isto é, não é fácil decidir entre Precisão e Revocação, podemos usar o F1 Score;
- O F1 Score é a **média harmônica** da Precisão e da Revocação;

$$F_1 = \left( \frac{2}{\frac{1}{recall} + \frac{1}{precisão}} \right) = 2 \times \frac{precision \times recall}{precision + recall}$$

# Matriz de Confusão: mais de duas classes

- Quando há mais de duas classes, ainda podemos calcular a matriz de confusão normalmente. Perceba que, assim como no caso de duas classes, dependendo do escopo considerado, as métricas de precisão, recall e f1-score mudam:

$$precisão_{urgente} \stackrel{\text{def}}{=} \frac{TP}{TP+FP}$$

$$revocação_{urgente} \stackrel{\text{def}}{=} \frac{TP}{TP+FN}$$

|                 | Urgente<br>(previsão) | Normal<br>(previsão) | SPAM<br>(previsão) |
|-----------------|-----------------------|----------------------|--------------------|
| Urgente (label) | 8                     | 5                    | 3                  |
| Normal (label)  | 10                    | 50                   | 30                 |
| SPAM (label)    | 1                     | 60                   | 300                |

Diagram illustrating the confusion matrix for three classes (Urgente, Normal, SPAM) with corresponding labels and metrics:

- TP (True Positive):** 8 (Urgente predicted as Urgente)
- FN (False Negative):** 5 (Normal predicted as Urgente) + 3 (SPAM predicted as Urgente)
- FP (False Positive):** 10 (Normal predicted as Urgente) + 1 (SPAM predicted as Urgente)
- TN (True Negative):** 50 (Normal predicted as Normal) + 300 (SPAM predicted as SPAM)

# Matriz de Confusão: mais de duas classes

$$precisão_{normal} \stackrel{\text{def}}{=} \frac{TP}{TP+FP}$$

$$revocação_{normal} \stackrel{\text{def}}{=} \frac{TP}{TP+FN}$$

|                 | Urgente<br>(previsão) | Normal<br>(previsão) | SPAM<br>(previsão) |
|-----------------|-----------------------|----------------------|--------------------|
| Urgente (label) | 8                     | 5                    | 3                  |
| Normal (label)  | 10                    | 50                   | 30                 |
| SPAM (label)    | 1                     | 60                   | 300                |

Diagram illustrating the Confusion Matrix for three classes (Urgente, Normal, SPAM) with predicted and actual labels. The matrix is annotated with True Negatives (TN), False Positives (FP), and False Negatives (FN).

Annotations:

- FN (False Negatives):** Indicated by blue arrows pointing to the cells (Urgente label, Urgente prediction), (Normal label, Urgente prediction), and (Normal label, SPAM prediction).
- FP (False Positives):** Indicated by red arrows pointing to the cells (Urgente label, Normal prediction), (Normal label, Normal prediction), and (SPAM label, Normal prediction).
- TN (True Negatives):** Indicated by orange arrows pointing to the cells (Urgente label, SPAM prediction), (Normal label, Urgente prediction), (Normal label, SPAM prediction), (SPAM label, Urgente prediction), (SPAM label, SPAM prediction), and (SPAM label, Normal prediction).

# Matriz de Confusão: mais de duas classes

$$precisão_{SPAM} \stackrel{\text{def}}{=} \frac{TP}{TP+FP}$$

$$revocação_{SPAM} \stackrel{\text{def}}{=} \frac{TP}{TP+FN}$$

|                 | <div>TN</div> <div>Urgente (previsão)</div> <div>Normal (previsão)</div> <div>SPAM (previsão)</div> <div>FP</div> |    |     |
|-----------------|-------------------------------------------------------------------------------------------------------------------|----|-----|
| Urgente (label) | 8                                                                                                                 | 5  | 3   |
| Normal (label)  | 10                                                                                                                | 50 | 30  |
| SPAM (label)    | 1                                                                                                                 | 60 | 300 |

FN

TP

Detailed description: The confusion matrix is a 3x3 grid. The columns represent predicted classes: Urgente, Normal, and SPAM. The rows represent actual labels: Urgente, Normal, and SPAM. The values are: Urgente (label) row: 8, 5, 3; Normal (label) row: 10, 50, 30; SPAM (label) row: 1, 60, 300. Annotations: An orange box labeled 'TN' points to the top-left cell (8). A red box labeled 'FP' points to the top-right cell (3). A blue box labeled 'FN' points to the bottom-left cell (1). A green box labeled 'TP' points to the bottom-right cell (300).



- As curvas ROC são usadas em classificadores com pontuação;
- ROC: **Receiver operating characteristic** (Característica de Operação do Receptor)
- Utilizada para classificadores binários (classificam em duas classes);
- Gráfico que compara dois pontos do classificador:
  - ***true positive rate (TPR)***: outro nome para a revocação
  - ***false positive rate (FPR)***: Proporção de exemplos negativos classificados de forma incorreta

- O TPR e o FPR podem ser definidos como:

$$TRP \stackrel{\text{def}}{=} \frac{TP}{TP + FN}$$

$$FRP \stackrel{\text{def}}{=} \frac{FP}{FP + TN}$$

As curvas ROC só podem ser usadas para avaliar classificadores que retornam uma pontuação (ou probabilidade) de previsão entre as classes.

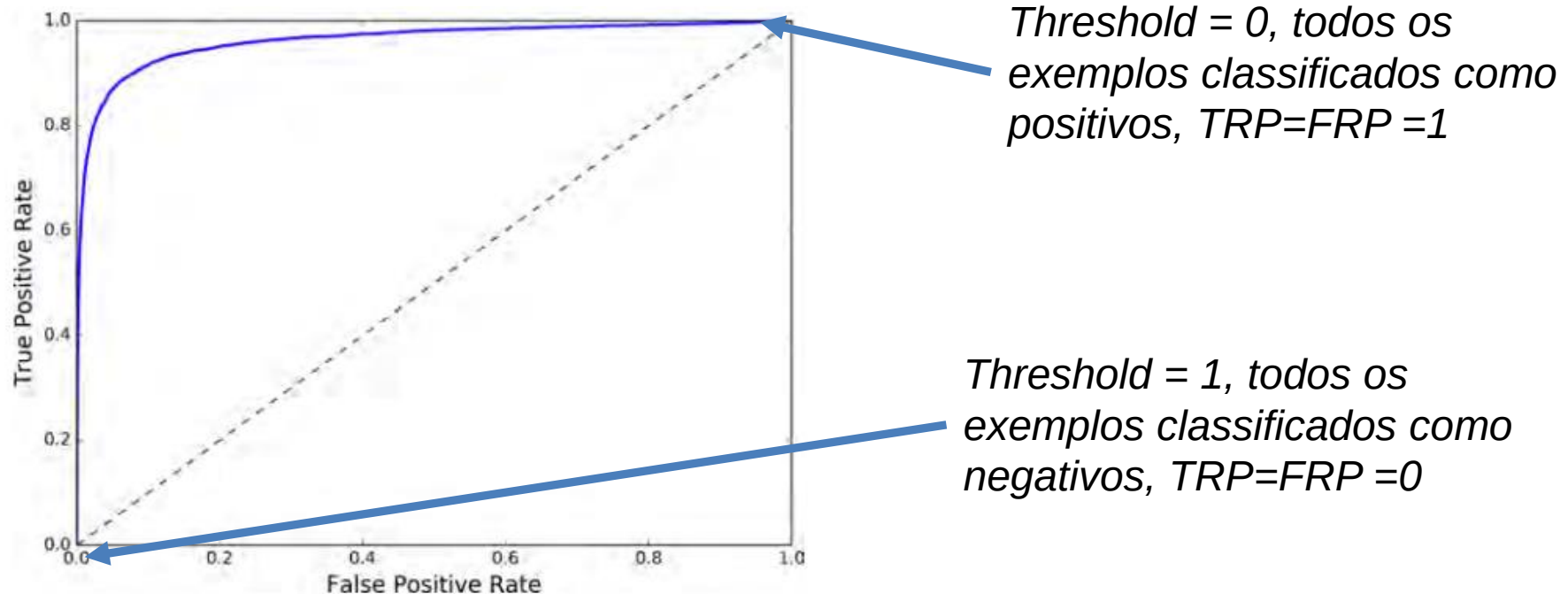
Por exemplo: seja  $\hat{p}$  um modelo qualquer descrito por  $\hat{p} = h_{\theta}(x)$ , com  $x$  sendo a entrada de dados. Como regra o modelo pode definir as classes como:

$$\hat{y} = \begin{cases} 0 & \text{if } \hat{p} < 0.5, \\ 1 & \text{if } \hat{p} \geq \underbrace{0.5}_{\text{threshold}}. \end{cases}$$

0: Classe negativa  
1: Classe positiva

# Curva ROC

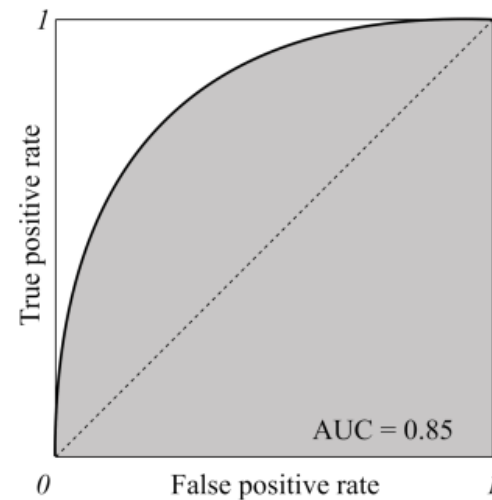
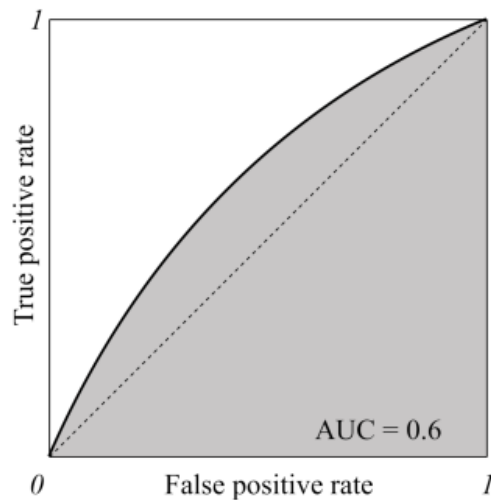
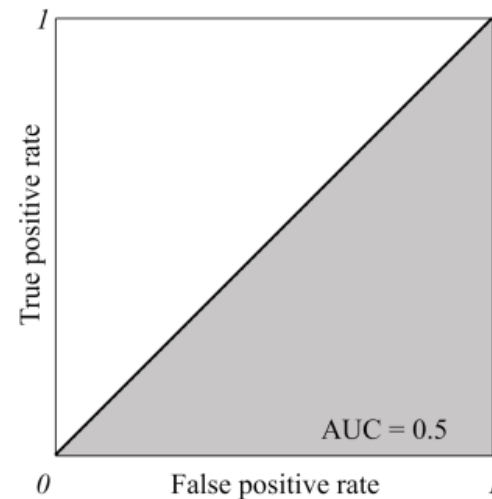
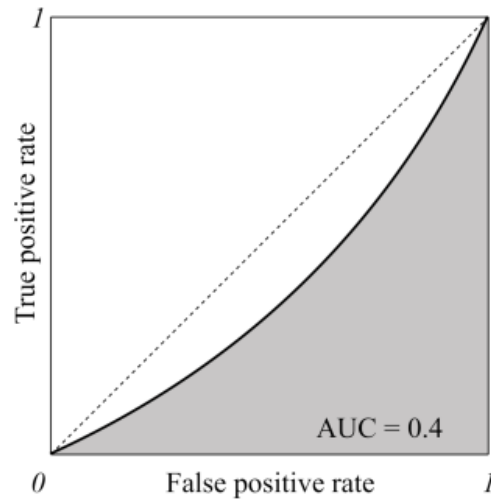
- Após calcular o TRP e o FRP para cada um dos valores podemos plotar o gráfico:



- Em azul temos o desempenho do modelo
- A linha tracejada representa um modelo que selecionaria aleatória cada uma das classes

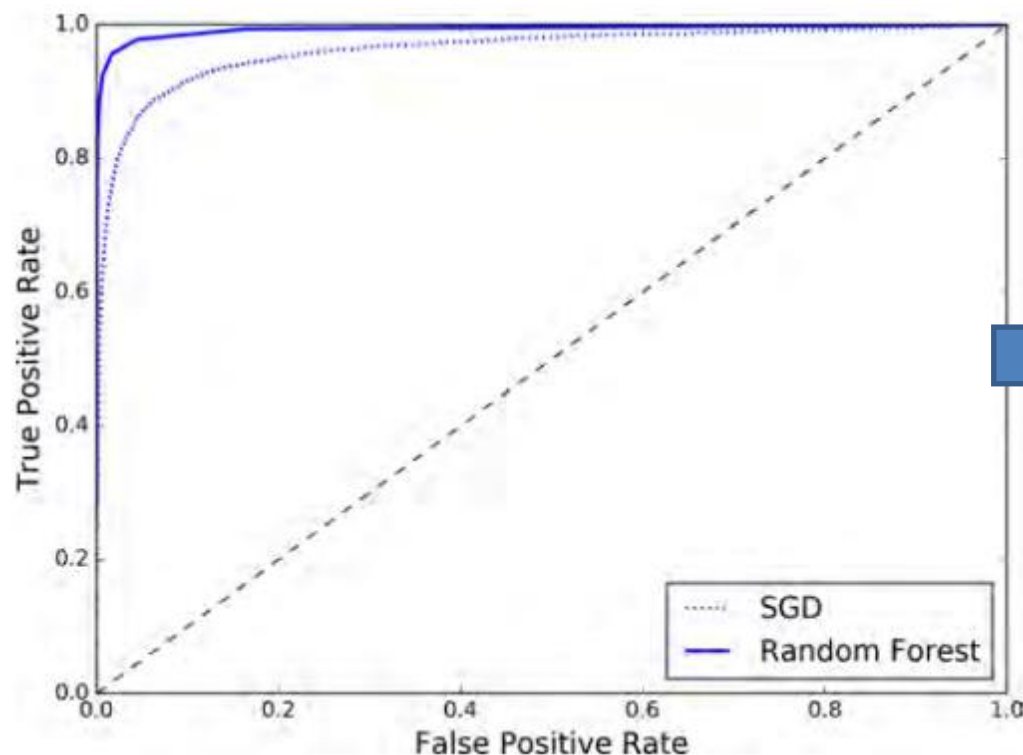
# Curva ROC

- Quanto maior a área abaixo da curva (AUC), melhor é o classificador



# Curva ROC

- Podemos usar ele até para comparar o desempenho de modelos diferentes:



O modelo de Random Forest possui AUC maior que o SGD. Logo, para esse estudo ele foi o melhor modelo

# Coeficiente Kappa de Cohen

- De maneira simplificada, o coeficiente Kappa de Cohen (Cohen's Kappa coefficient) mede o quanto o classificador está diferente de um classificador aleatório (o acaso);
- O valor K varia de -1 a 1, sendo que um classificador que recebe 1 é melhor do que um que recebe valores menores do que isso;
- K é o coeficiente de Cohen,  $p_0$  é a probabilidade empírica de concordância entre o rótulo assinalado e qualquer amostra,  $p_e$  é a concordância esperada quando dois rótulos aleatórios são emparelhados;

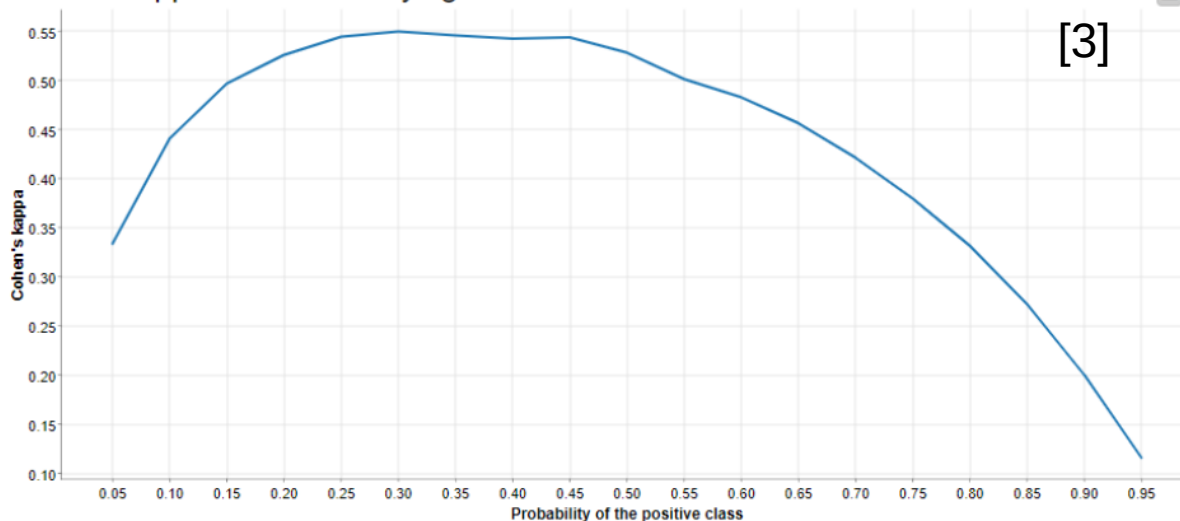
$$\kappa = \frac{p_0 - p_e}{1 - p_e}$$

# Coeficiente Kappa de Cohen

| VALUE OF K  | LEVEL OF AGREEMENT | % OF DATA THAT ARE RELIABLE | [2] |
|-------------|--------------------|-----------------------------|-----|
| 0 - 0.20    | None               | 0 - 4%                      |     |
| 0.21 - 0.39 | Minimal            | 4 - 15%                     |     |
| 0.40 - 0.59 | Weak               | 15 - 35%                    |     |
| 0.60 - 0.79 | Moderate           | 35 - 63%                    |     |
| 0.80 - 0.90 | Strong             | 64 - 81%                    |     |
| Above 0.90  | Almost Perfect     | 82 - 100%                   |     |

- O coeficiente Kappa tende a ser maior quando os dados possuem classes balanceadas;

Cohen's kappa values for varying class distributions



[3]

# Métricas de Classificação na prática

- Em Python já existem bibliotecas prontas para se criar a matriz de confusão e calcular as métricas de desempenho associadas;
- O scikit-learn traz módulos internos com funções prontas para isso:

[https://scikit-learn.org/stable/modules/model\\_evaluation.html#classification-metrics](https://scikit-learn.org/stable/modules/model_evaluation.html#classification-metrics)

```
1 matriz_confusao = confusion_matrix(y_true = y_test,  
2                                   y_pred = y_predicoes,  
3                                   labels=['Disk Hernia', 'Normal', 'Spondylolisthesis'])  
4  
5 # plotando uma figura com a matriz de confusao  
6 figure = plt.figure(figsize=(15, 5))  
7 disp = ConfusionMatrixDisplay(confusion_matrix = matriz_confusao,  
8                               display_labels=['Disk Hernia', 'Normal', 'Spondylolisthesis'])  
9 disp.plot(values_format='d')
```

```
1 # Metricas de precisão, revocação, f1-score e acurácia.  
2 print(classification_report(y_test, y_predicoes))
```







# **ALGORITMOS DE DE CLASSIFICAÇÃO**

# Algoritmos de Classificação

Existem vários algoritmos de ML que são usados como classificadores. Exemplos:

- Classificadores Bayesianos;
- Classificador kNN;
- Classificador por regressão logística;
- Máquina de vetor suporte (SVM);
- Árvores de Decisão e RandomForest;
- Entre outros...

**Qual é o melhor?** A resposta a essa pergunta depende do seu problema. Alguns algoritmos são melhores para um problema do que outros. Por isso devemos sempre usar métricas de desempenho para comparar (matriz de confusão, acurácia, f1score, curva ROC, etc.).

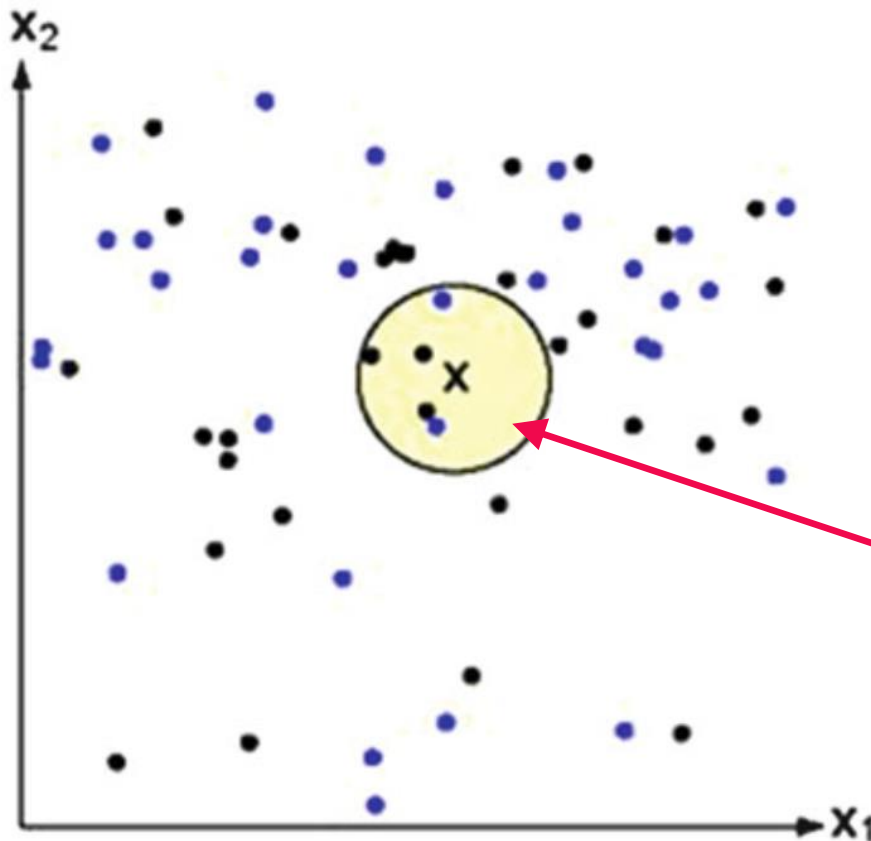
**Não existe almoço grátis!**

- Outro tipo de classificador com uma estratégia diferente é o **classificador k-vizinhos mais próximos** (do inglês, k-nearest neighbors, k-NN);
- A ideia do algoritmo k-NN é calcular a distância entre pontos no espaço. Ele é **não paramétrico**, o que significa que ele não tem uma função clara definida a princípio (a fronteira de decisão);
- Ele possui um parâmetro ajustável  $k$  que controla quantos vizinhos são considerados durante a tarefa de classificação;
- $k$  é um hiperparâmetro;

- Podemos resumir o algoritmo kNN da seguinte forma:
  1. O  $k$ -NN inicia a partir do ponto onde se encontra o exemplo  $X_1 = (x_1, x_2, x_3, \dots, x_n)_1$  que desejamos classificar.
  2. Na sequência encontramos os  $k$  vizinhos mais próximos a esse ponto  
 $(x_1, x_2, x_3, \dots, x_n)_2, (x_1, x_2, x_3, \dots, x_n)_3, \dots, (x_1, x_2, x_3, \dots, x_n)_i$ .
  3. Entre os  $k$  vizinhos mais próximos verificamos qual a classe mais frequente entre eles ( $c_i$ )
  4. Classificamos o exemplo  $x$  como pertencente a classe  $c_i$

# Classificador k-NN - Exemplo

- Supondo que cada entrada  $X_i$  tenha apenas dois atributos  $(x_1, x_2)$ . Podemos plotar um gráfico dos vários pontos, e colori-los com as classes  $c_i$  diferentes:



- Classe 1 – Preto
- Classe 2 – Azul
- Ponto novo para ser classificado representado por um x

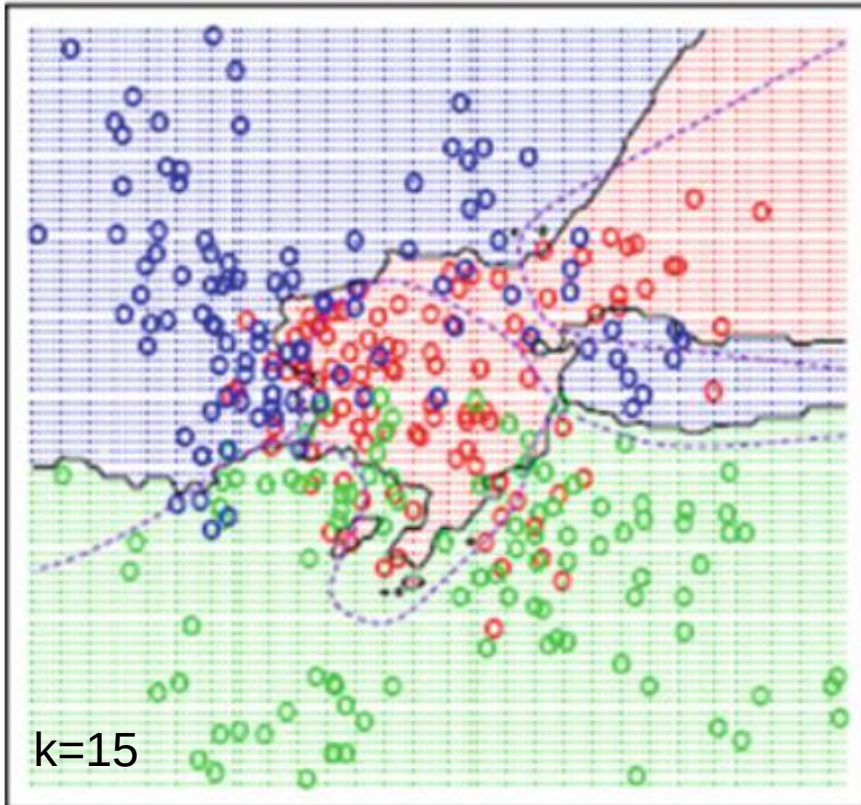
Se  $k = 5$  vizinhos mais próximos de x:

- 3 da classe 1
- 2 da classe 2
- **Logo x pertence a classe 1**

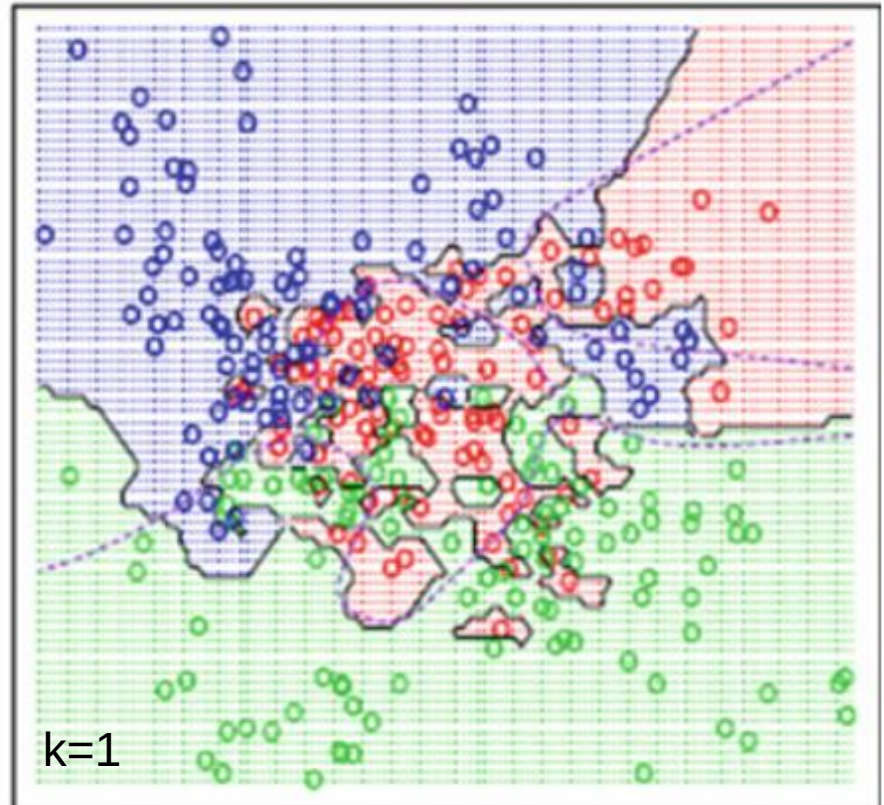
# Classificador kNN

- Podemos ter mais de duas classes (no exemplo a seguir, três classes representadas pelas cores azul, vermelho e verde) e diferentes números de  $k$ ;

a



b

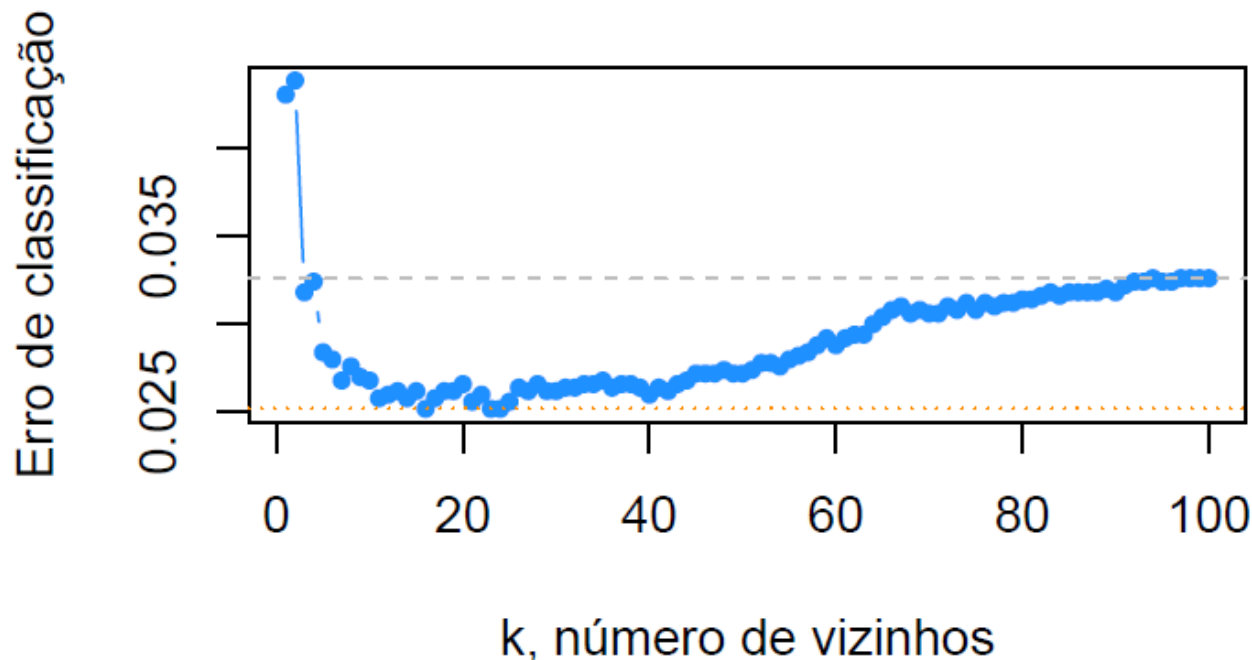


# Classificador kNN – Escolhendo k

- Para duas classes,  $k$  deve ser ímpar para evitar empates;
- Para mais de duas classes  $k$  ser ímpar não é o suficiente para evitar empates. Por exemplo 7-NN:
  - $C_1$  com três exemplos próximos
  - $C_2$  com três exemplos próximos
  - $C_3$  com um exemplos próximos
- Nesses casos temos que definir um critério para escolha da classe;
- Valores maiores de  $k$  ajudam a evitar empates;

# Classificador kNN – Escolhendo k

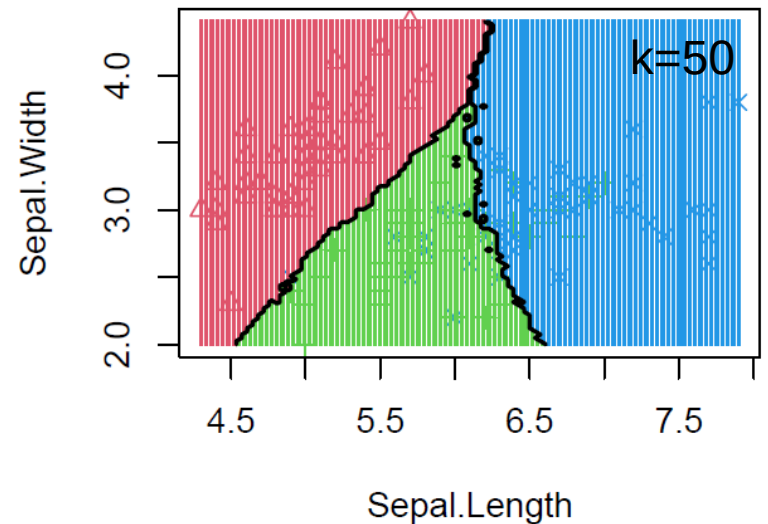
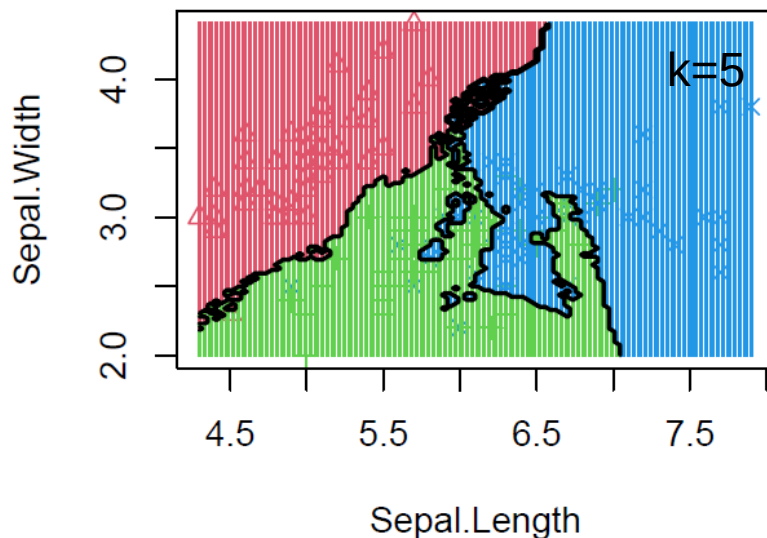
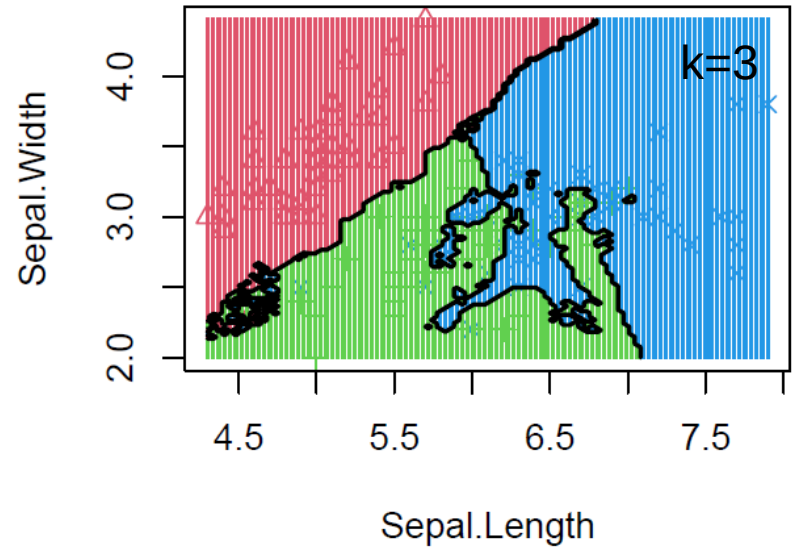
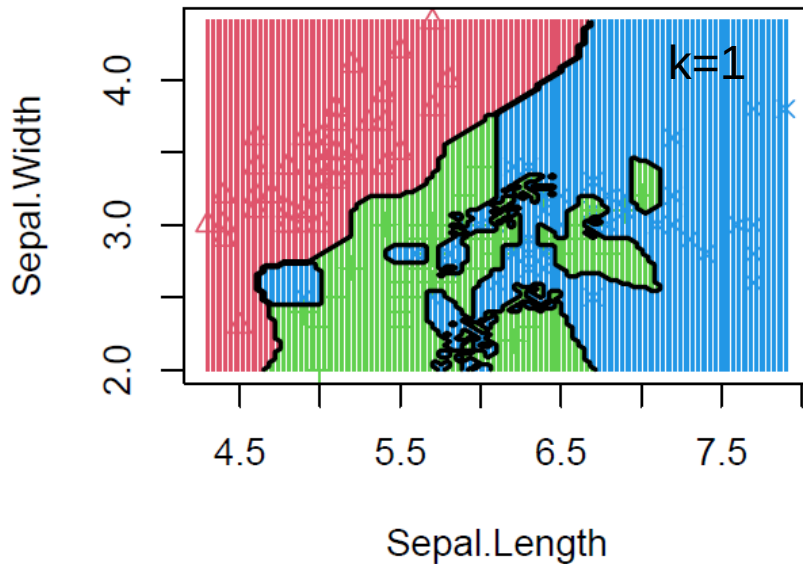
- A **melhor solução é testar diferentes valores de k** e verificar o erro de classificação (faça um gráfico para ajudar);
- Para evitar **overfitting** nos dados, teste o algoritmo com os **dados de validação**, encontrando o melhor k para esses exemplos;





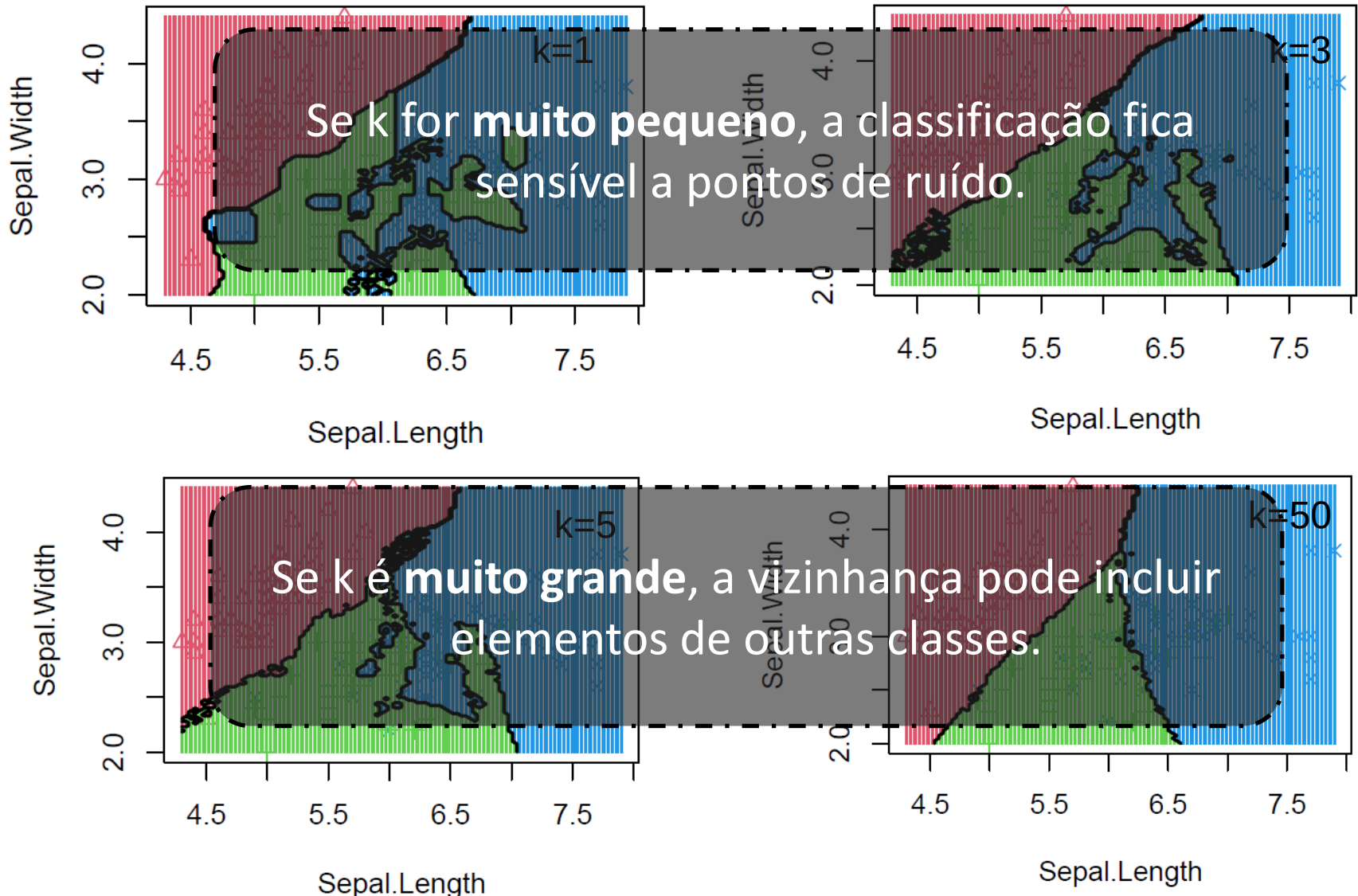
# Classificador kNN – Escolhendo k

- Fronteira de decisão de acordo com k:



# Classificador kNN – Escolhendo k

- Fronteira de decisão de acordo com k:

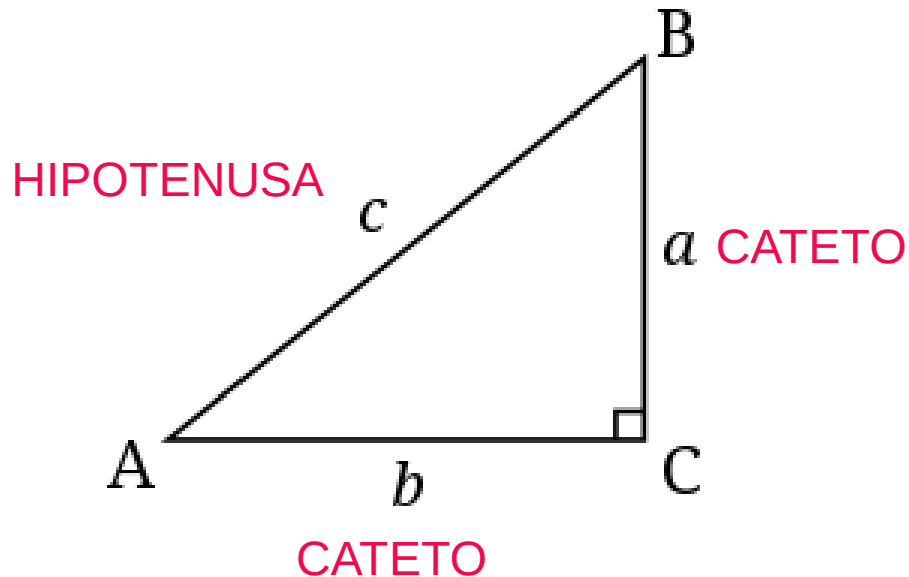


# Classificador kNN – Função Distância

- Como saber qual vizinho é mais próximo?
- Precisamos de métricas de distância. Existem várias!
- O desempenho do algoritmo k-NN é **altamente dependente da função distância escolhida**;
- Atributos com dimensionalidade muito diferente podem interferir no cálculo das distâncias, por exemplo uma medida em milhares para  $x_1$  e outra em décimos para  $x_2$ . Por isso precisamos colocá-los na mesma escala usando um escalonador;
- O tempo de predição pode ser alto se a base de dados (número de linhas na tabela) for grande. Neste caso existem técnicas avançadas de indexação (kd-tree, ball-tree) que podem ser usadas para melhorar o tempo do cálculo das distâncias;

# Classificador kNN – Função Distância

- Existem várias métricas de distância. Vamos ver algumas delas.
  - **Distância Euclidiana:** é a distância geométrica clássica que utiliza o Teorema de Pitágoras:



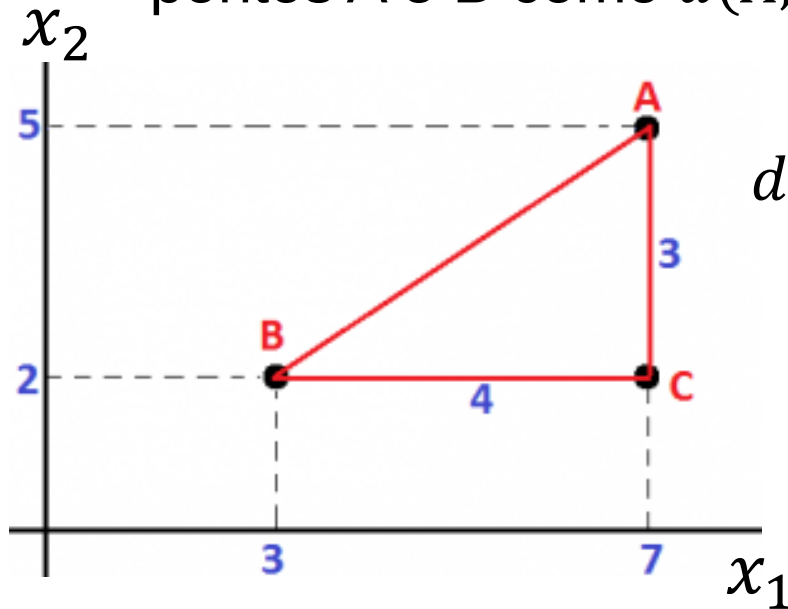
$$c = \sqrt{a^2 + b^2}$$

Qual é a distância em linha reta entre os pontos A e B?

Vamos colocar o triângulo retângulo sobre um eixo cartesiano e observar as coordenadas dos pontos A e B.

# Classificador kNN – Função Distância

- No eixo cartesiano cada ponto tem duas coordenadas (espaço bidimensional). Vamos denotar a distância entre os pontos A e B como  $d(A, B)$ . Assim, temos:



$$d(A, B) = \sqrt{(A_{x_1} - B_{x_1})^2 + (A_{x_2} - B_{x_2})^2}$$

$$d(A, B) = \sqrt{(7 - 3)^2 + (5 - 2)^2}$$

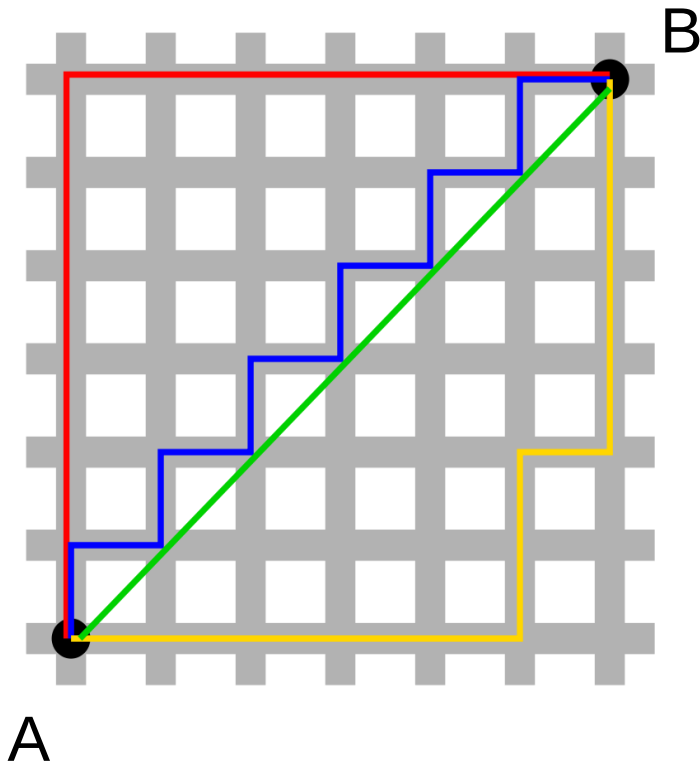
$$d(A, B) = \sqrt{(4)^2 + (3)^2} = \sqrt{25} = 5$$

- Se temos  $n$  dimensões ( $n$  atributos possíveis), teremos:

$$d(A, B) = \sqrt{\sum_{i=1}^n (A_{x_i} - B_{x_i})^2}$$

# Classificador kNN – Função Distância

- **Distância de Manhattan**: é baseada na geometria que um taxi percorre em Manhattan (taxicab geometry) ou em Lisboa (geometria pombalina).



$$d(A, B) = |A_{x1} - B_{x1}| + |A_{x2} - B_{x2}|$$



# Classificador kNN – Função Distância

- **Outras métricas:** <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.DistanceMetric.html>

**Metrics intended for real-valued vector spaces:**

| identifier    | class name          | args    | distance function                |
|---------------|---------------------|---------|----------------------------------|
| "euclidean"   | EuclideanDistance   | •       | $\sqrt{\sum (x - y)^2}$          |
| "manhattan"   | ManhattanDistance   | •       | $\sum ( x - y )$                 |
| "chebyshev"   | ChebyshevDistance   | •       | $\max ( x - y )$                 |
| "minkowski"   | MinkowskiDistance   | p       | $\sum ( x - y ^p)^{1/p}$         |
| "wminkowski"  | WMinkowskiDistance  | p, w    | $\sum ( w * (x - y) ^p)^{1/p}$   |
| "seuclidean"  | SEuclideanDistance  | V       | $\sqrt{\sum (x - y)^2 / V)}$     |
| "mahalanobis" | MahalanobisDistance | V or VI | $\sqrt{(x - y)' V^{-1} (x - y)}$ |



# Classificador kNN – Função Distância

- Algumas métricas com mais de um nome: [https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.distance\\_metrics.html](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.distance_metrics.html)

The valid distance metrics, and the function they map to, are:

| <b>metric</b>   | <b>Function</b>                          |
|-----------------|------------------------------------------|
| 'cityblock'     | metrics.pairwise.manhattan_distances     |
| 'cosine'        | metrics.pairwise.cosine_distances        |
| 'euclidean'     | metrics.pairwise.euclidean_distances     |
| 'haversine'     | metrics.pairwise.haversine_distances     |
| 'l1'            | metrics.pairwise.manhattan_distances     |
| 'l2'            | metrics.pairwise.euclidean_distances     |
| 'manhattan'     | metrics.pairwise.manhattan_distances     |
| 'nan_euclidean' | metrics.pairwise.nan_euclidean_distances |



- Outra observação importante é que o escalonamento pode ser extremamente importante para o  $k$ -NN;
- Imagine os dois exemplos:

$$\left. \begin{array}{l} \mathbf{x} = (t, 0.2, 254) \\ \mathbf{y} = (f, 0.1, 194) \end{array} \right\} d_M(\mathbf{x}, \mathbf{y}) = \sqrt{(1 - 0)^2 + (0.2 - 0.1)^2 + (254 - 194)^2}$$

- A terceira *feature* é a que tem maior contribuição na distância apenas pelo fato das escalas serem diferentes.

# kNN – Transformação dados discretos

- Existe a possibilidade de processarmos dados discretos como categorias no *k*NN.
  - As transformações **Label Encoding** e o **One Hot Encoding** pode nos dar valores numéricos para features categoricas.
  - Entretanto existem algumas considerações que podemos citar caso desejarmos criar um algoritmo de *k*NN para processar dados discretos;
  - As **features contínuas** podem ter distâncias maiores do que 1 e as **features discretas** tem apenas 1 de distância. Dessa forma, as **features contínuas** podem contribuir com um valor maior na distância do que as **features discretas**;

# kNN – Transformação dados discretos

- Exemplo: **Features:**  $x = (x_1, x_2, x_3)$

$$x^{(1)} = (2, 1.5, \text{summer})$$

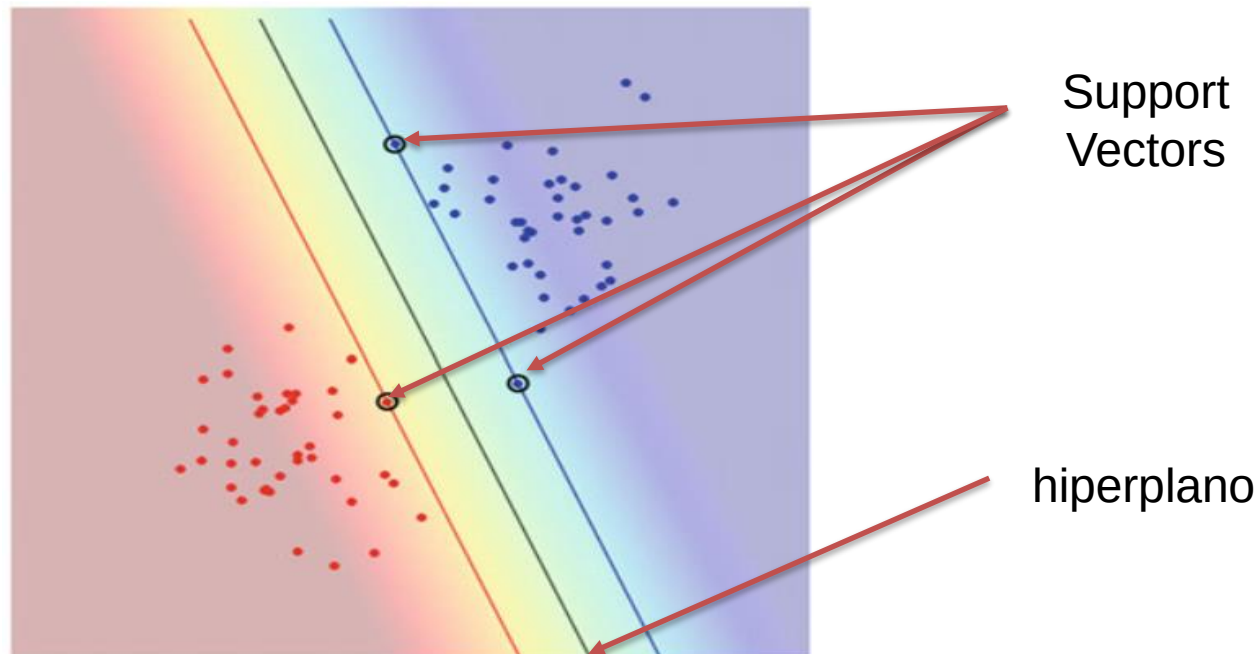
**Entradas:**  $x^{(2)} = (1, 0.5, \text{winter})$

$$x^{(3)} = (1, .25, 0.75, \text{autumn})$$

- Uma consideração importante nesse caso é que estações vizinhas podem ter características mais próximas do que estações mais distantes
  - Como o verão e o outono versus verão e inverno
  - Primavera e outono versus verão e inverno
- Nesses casos, dois valores (0 e 1) não serão suficientes para incorporar essa informação ao nosso modelo.
- Logo valores intermediários poderiam ser incorporados

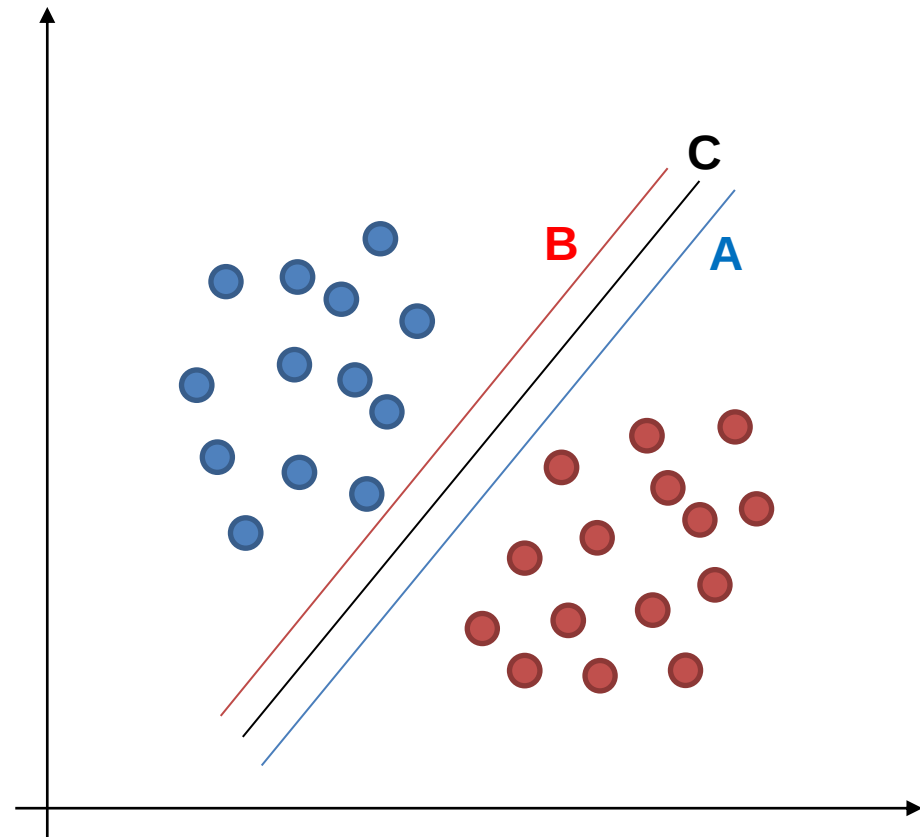
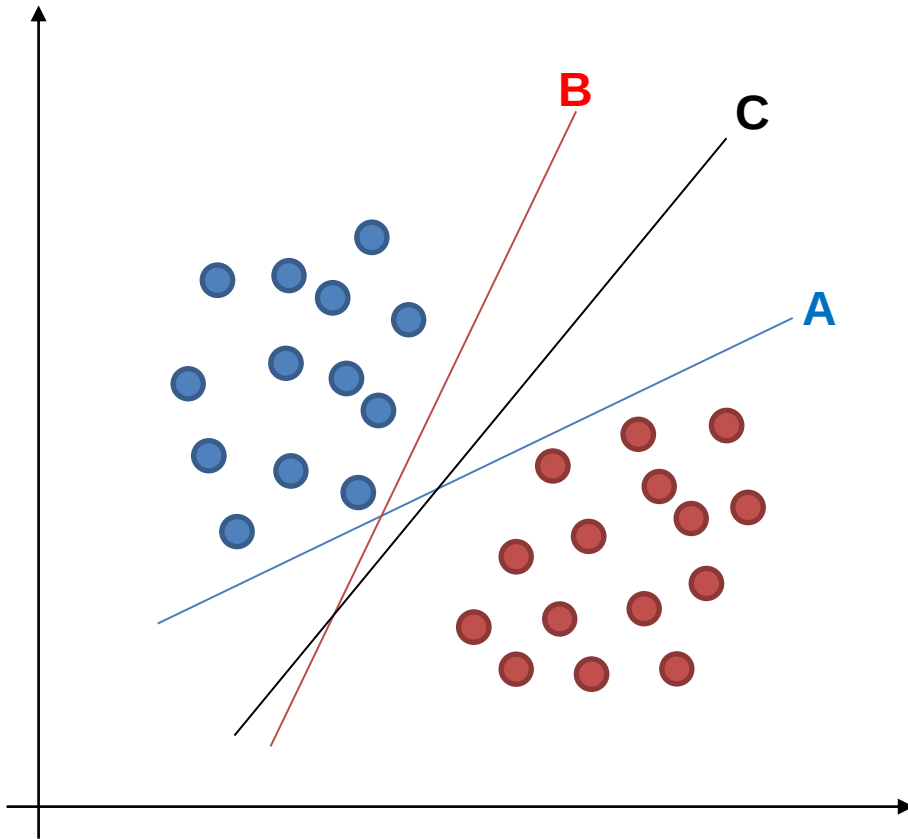
# Máquina de Vetor Suporte - SVM

- As máquinas de vetor suporte (Support-vector Machine – SVM) são um conjunto de métodos de **aprendizados de máquina supervisionado** (como os outros métodos de classificação que vimos até aqui no curso);
- Elas se baseiam na ideia de separar os dados através de hiperplanos;



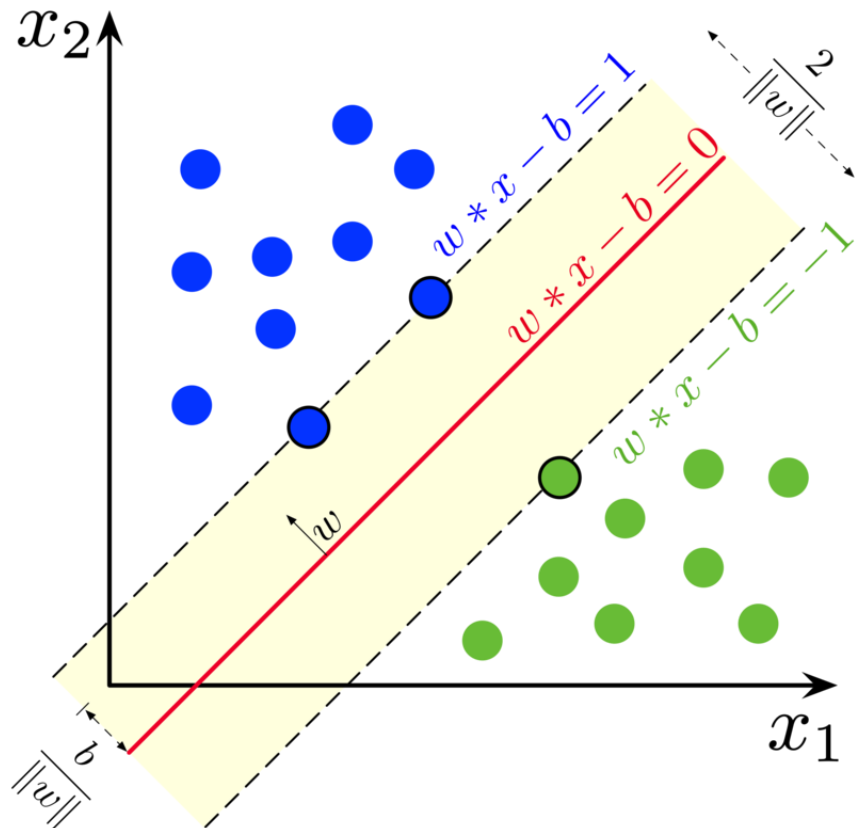
# Máquina de Vetor Suporte - SVM

- Queremos buscar o melhor hiperplano de separação.
- Qual é melhor? Ou, em outras palavras, qual divide melhor as duas classes (azul e vermelho)?



# Máquina de Vetor Suporte - SVM

- O SVM responde essa pergunta maximizando a largura da margem;
- Para isso podemos imaginar três retas: uma central que representa a solução do algoritmo e duas retas auxiliares chamadas de margem
- Retas em espaços com mais de duas dimensões são chamadas de hiperplanos.
- Quando temos mais de dois atributos, i.e.,  $(x_1, x_2, x_3, \dots, x_n)$  teremos um **hiperplano de separação das classes**.

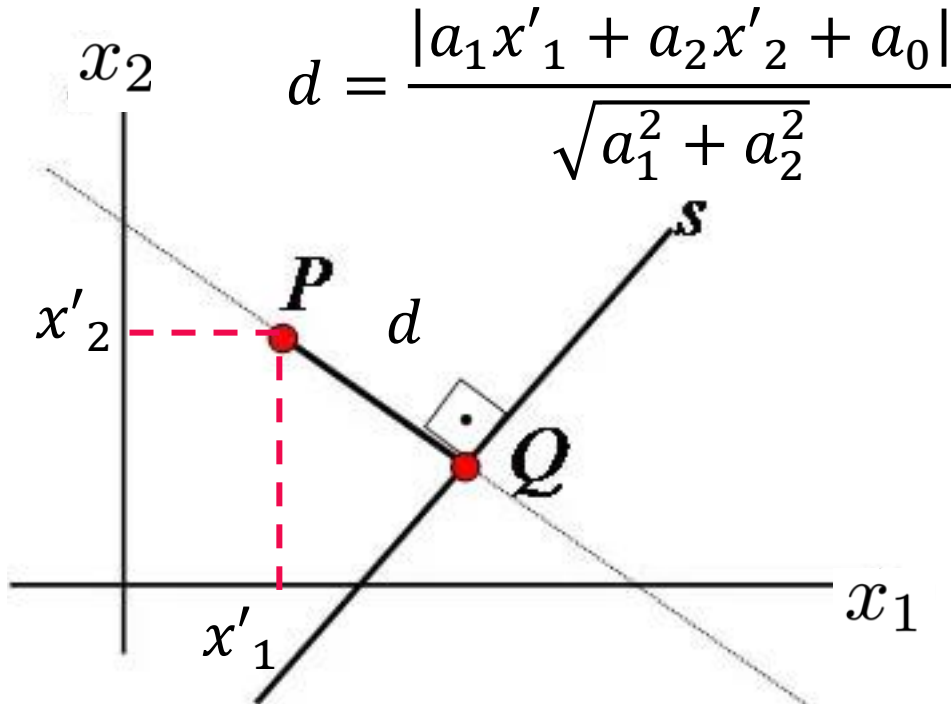


# Máquina de Vetor Suporte - SVM

- O hiperplano pode ser definido como:

$$z = a_0 + a_1x_1 + a_2x_2 + \cdots + a_Nx_N = a_0 + \boldsymbol{\theta}^T \mathbf{x}$$

- Qual é a distância  $d$  entre um ponto de coordenadas  $x_1$  e  $x_2$  tais que  $P(x'_1, x'_2)$  e a reta  $s$ ?



Vetor de coeficientes do hiperplano:

$$\boldsymbol{\theta} = (a_1, a_2, a_3, \dots, a_N)$$

$$d_i = \frac{|\boldsymbol{\theta}^T \mathbf{x}^i + a_0|}{||\boldsymbol{\theta}||}$$

# SVM – Função Custo

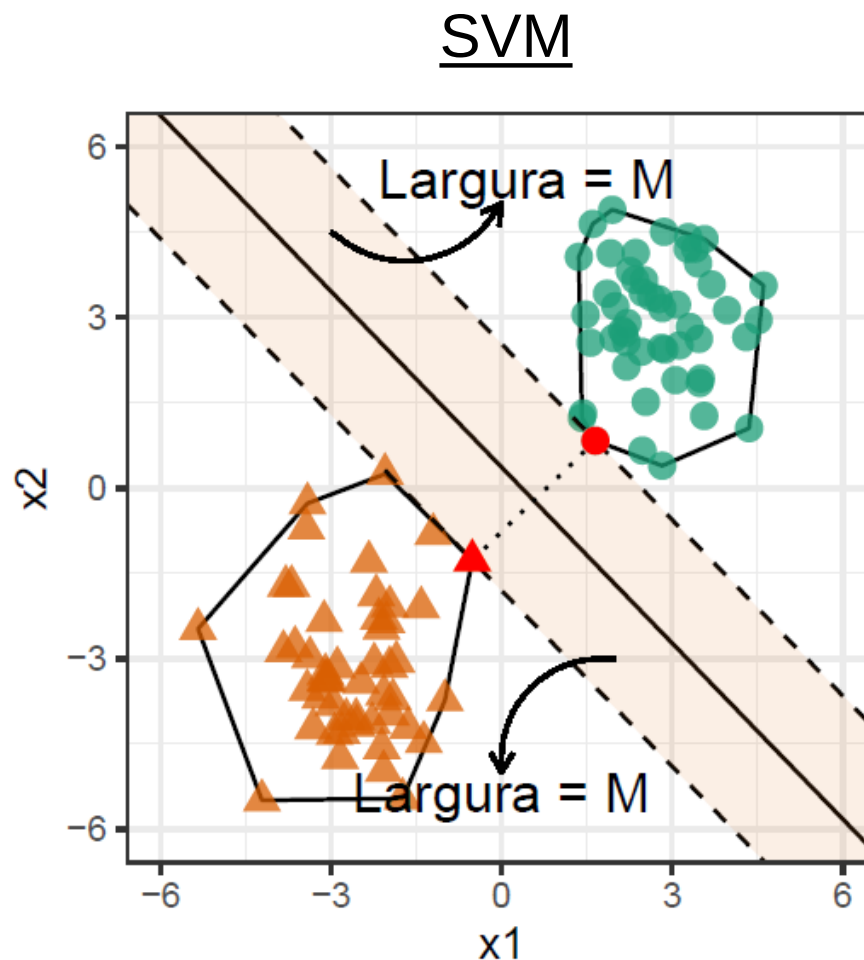
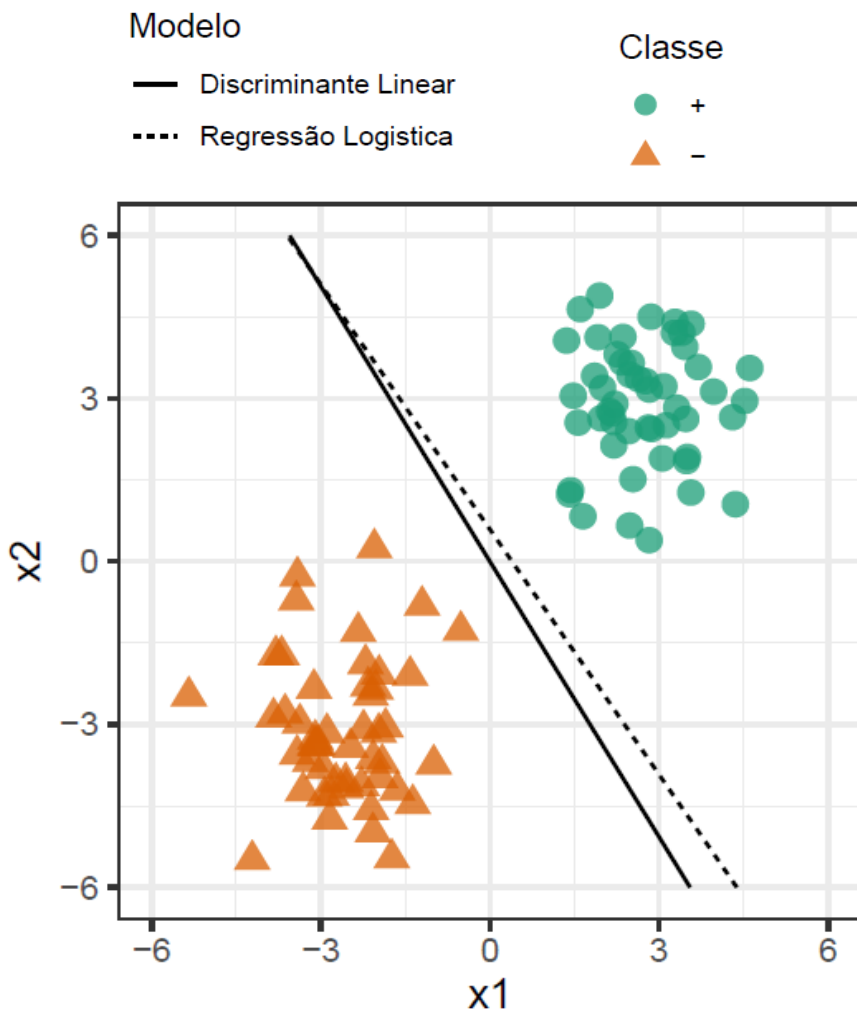
- O algoritmo de SVM quer encontrar os coeficientes  $a_1, a_2, a_3, \dots, a_N$  que definem o hiperplano de separação;
- Uma vez definido o hiperplano fica fácil classificar os dados como aqueles que estão de um lado ou de outro do hiperplano de separação;
- Para encontrar os coeficientes, mais uma vez usamos um método de otimização em que queremos minimizar uma função custo;
- A função custo da SVM é:

$$Custo(y, \hat{y}) = \begin{cases} \max(0, 1 - z), & \text{se } y = 1 \\ \max(0, 1 + z), & \text{se } y = 0 \end{cases}$$



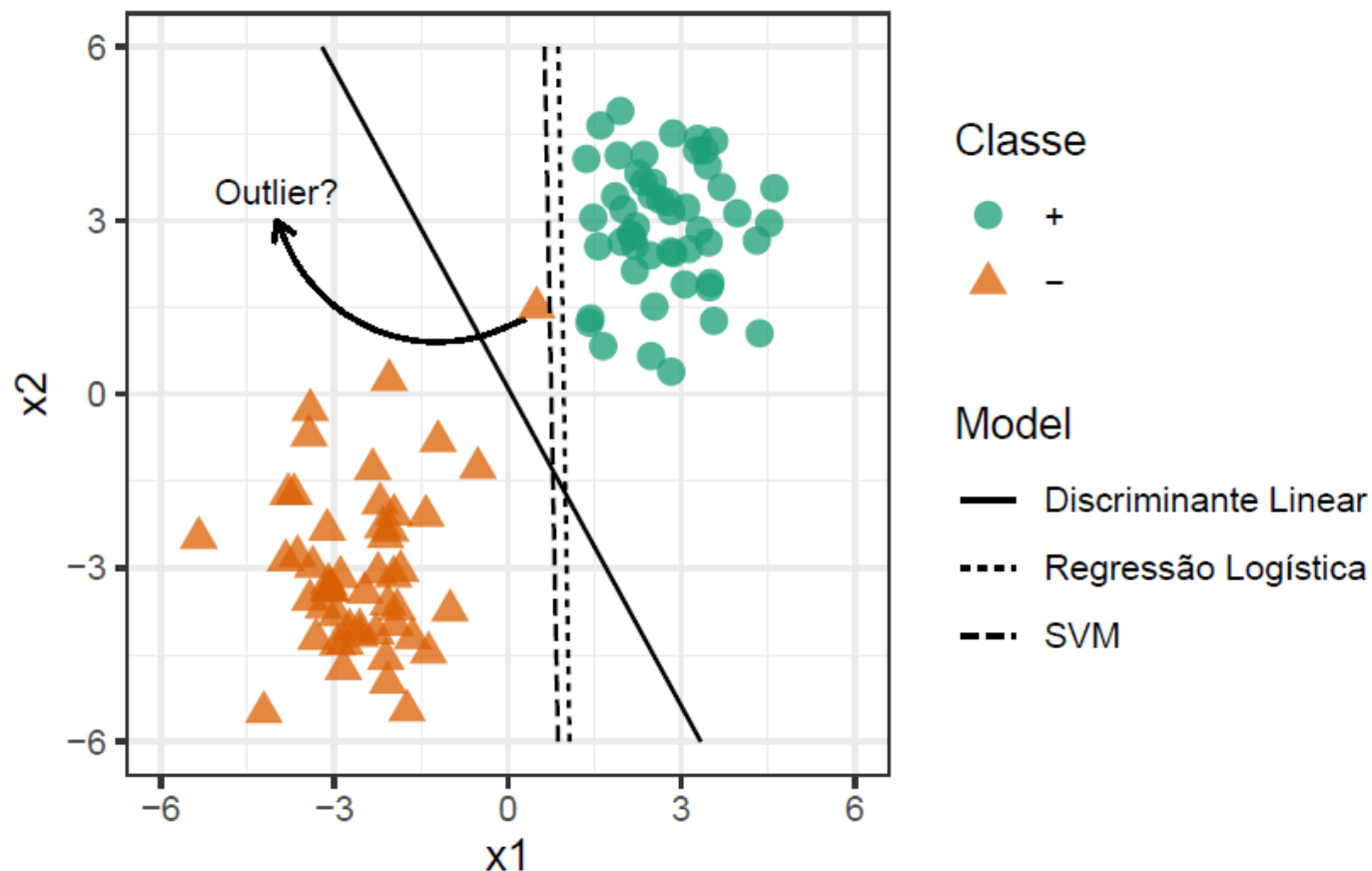
# SVM – Parâmetro C

- Em comparação com outros modelos de classificação o SVM é chamado de **classificador de margem larga**.



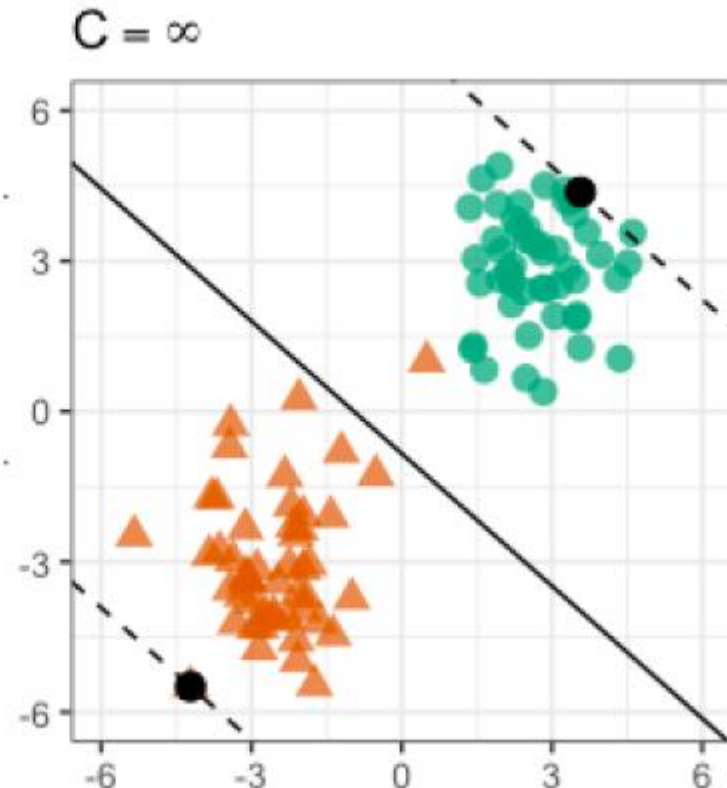
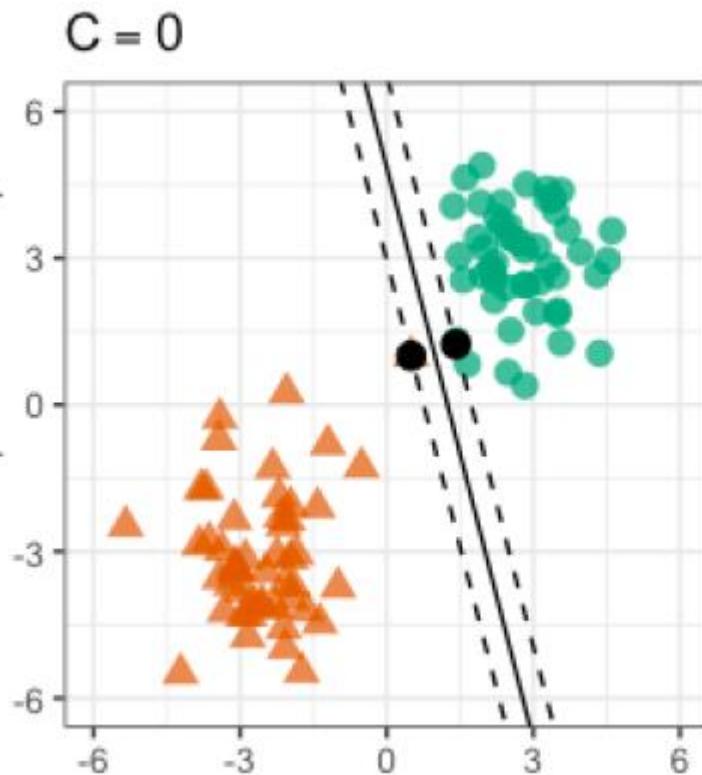
# SVM – Parâmetro C

- A margem larga pode gerar problema de classificação, principalmente quando os dados da classe tem alta variabilidade (variância alta). Para tratar isso, precisa-se fazer ajustes do algoritmo SVM usando o fator C.



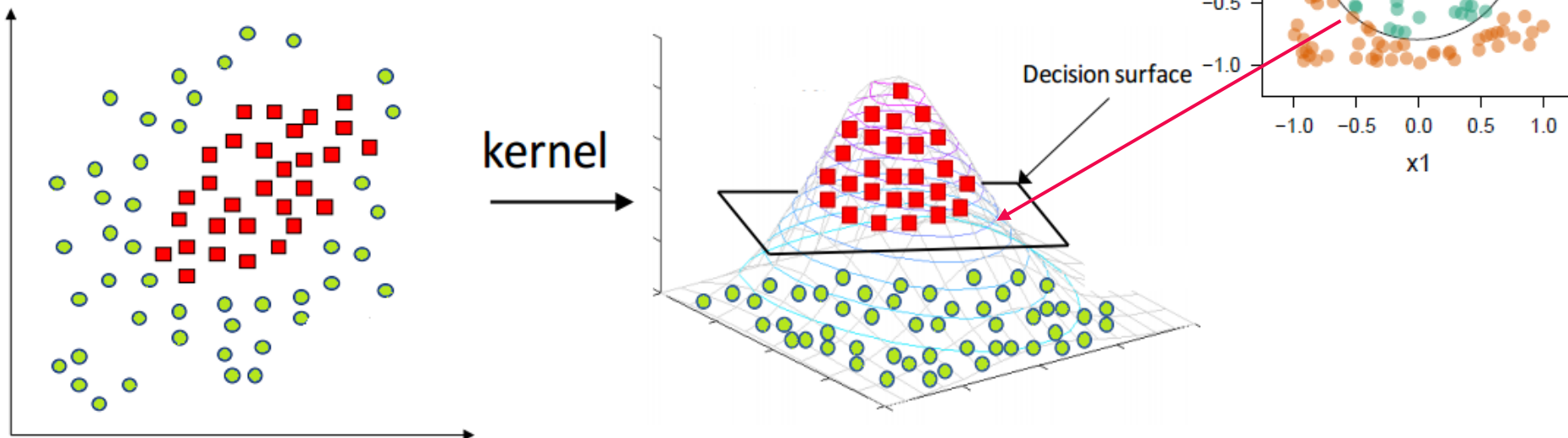
# SVM – Parâmetro C

- O parâmetro C cria o conceito de **margem suave** (soft) que é a ideia de permitir pequenas violações (alguns exemplos podem cair dentro da margem);

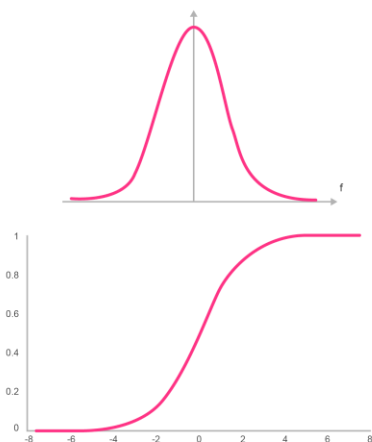


# SVM - Kernel

- A SVM também consegue lidar com dados que não são linearmente separáveis usando um truque de kernel;
- A ideia é observar os dados em uma dimensão diferentes. Para realizar isso, o algoritmo aplica uma função sobre os dados chamada função de kernel. Varias são possíveis:
  - Kernel Linear;
  - Kernel Polinomial;
  - Kernel Função Base Radial (RBF);
  - Kernel Sigmoidal;



# SVM - Kernel



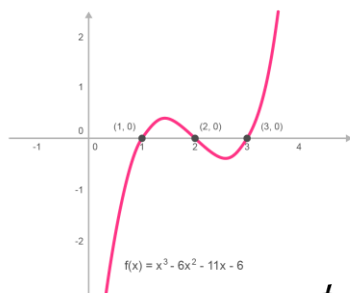
Gaussian RBF Kernel

Função  $\varphi(x_i, x_j)$

$$\exp(-\sigma \|x_i - x_j\|^2)$$

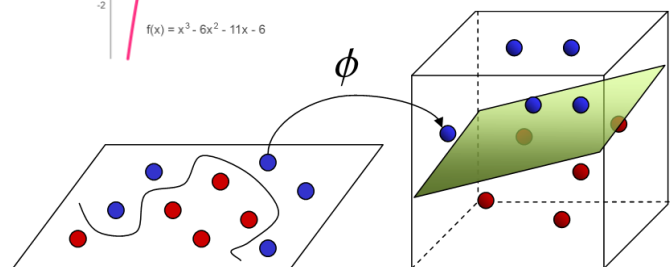
Sigmoid Kernel

$$\tanh(\delta(x_i \cdot x_j) + k)$$



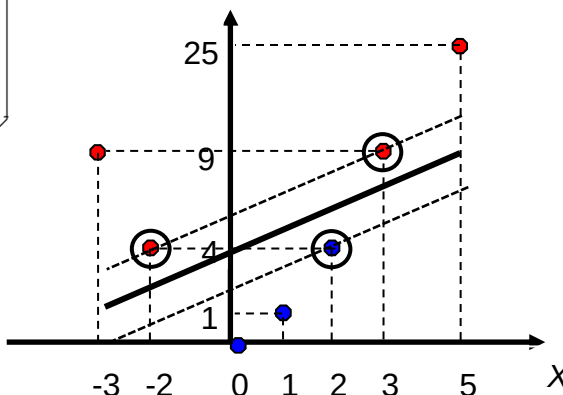
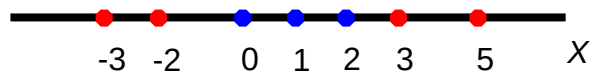
Polynomial Kernel

$$(\delta(x_i \cdot x_j) + k)^d$$



Input Space

Feature Space

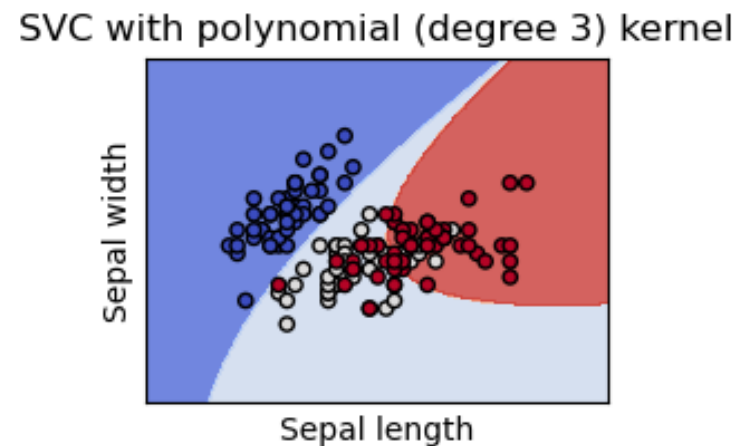
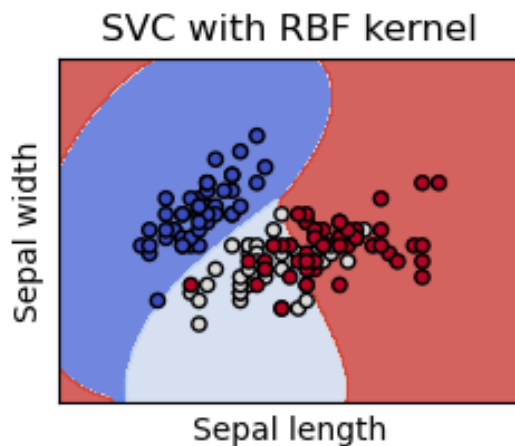
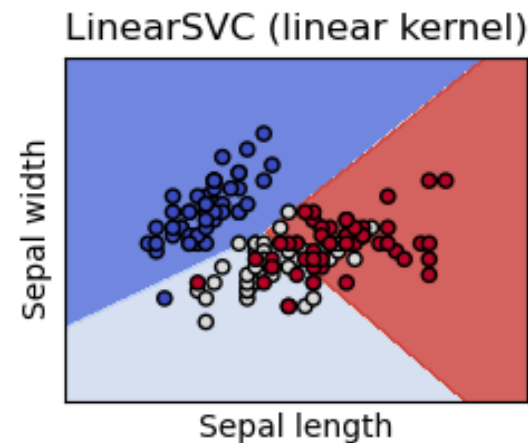
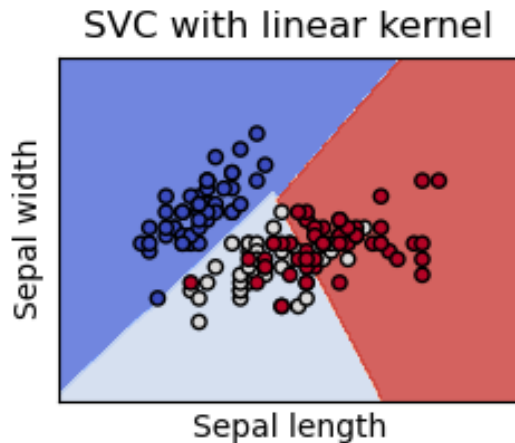


Elevando para uma  
dimensão linearmente  
separável ( $R1 \rightarrow R2$ ):

Kernel:  $\phi(x) = (x, x^2)$

# SVM - Kernel

Mais informações: <https://scikit-learn.org/stable/modules/svm.html>



# Classificador Bayesiano

Classificadores Bayesianos são modelos probabilísticos que usam informação sobre a distribuição dos dados e a distribuição de probabilidade de pertencimento a cada classe; Vamos relembrar um pouquinho de probabilidade:

**Espaço amostral  $S$ :** é o conjunto de todos os resultados possíveis de um experimento aleatório

Exemplo: Lançamento de uma moeda:

$$S = \{Cara, Coroa\}$$

Exemplo: Lançamento de uma dado de 6 faces:

$$S = \{1, 2, 3, 4, 5, 6\}$$

**Probabilidade  $P(A)$ :** é a probabilidade do evento A ocorrer;

Exemplo: Lançamento de uma moeda não viciada, se denotamos o evento A como aquele em que o lançamento rende uma coroa, teremos que:

$$P(A) = \frac{1}{2}$$

Exemplo: Lançamento de uma dado de 6 faces não viciado, se denotamos o evento A como aquele em que o lançamento rende um 5, teremos que:

$$P(A) = \frac{1}{6}$$



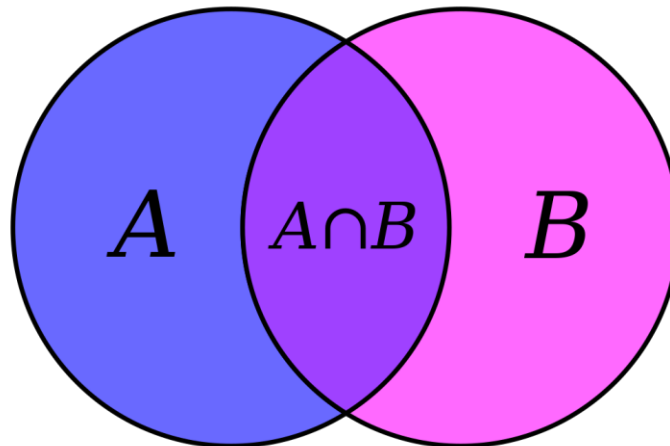
# Bayesiano - Revisão Probabilidade

**Probabilidade da união de eventos:**  $P(A \cup B)$  ou  $P(A \text{ ou } B)$

**Probabilidade da intersecção de eventos:**  $P(A \cap B)$  ou  $P(A \text{ e } B)$

Relação entre as duas:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$



**Eventos independentes:** são aqueles cuja probabilidade de ocorrer não depende do que aconteceu antes. Por exemplo, a chance de um dado dar 5 não depende da chance de um dado ter dado 6 em um lançamento passado.

**Atenção:** se a pergunta for, dado que tirei 5 no dado, qual a chance de tirar 5 de novo. Question! Você quer saber qual a chance de tirar 5 nesse novo lançamento ou a chance de dois lançamentos consecutivos darem 5?

A: chance de dar 5 a qualquer momento  $P(A) = \frac{1}{6}$

B: chance de dar 5 depois de ter dado 5  $P(B) = \frac{1}{6}$

C: chance de dar dois 5 seguidos  $P(C) = P(A) * P(B) = \frac{1}{6} * \frac{1}{6} = \frac{1}{36}$

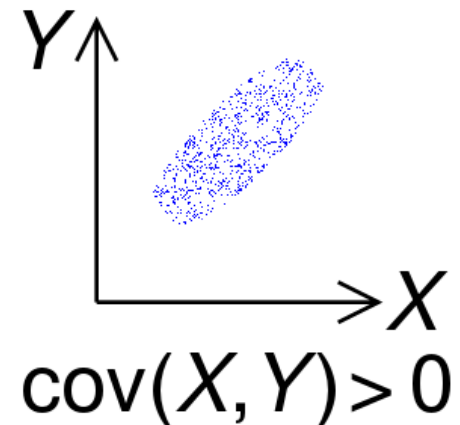
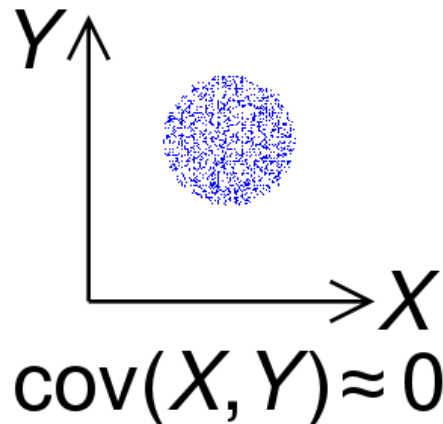
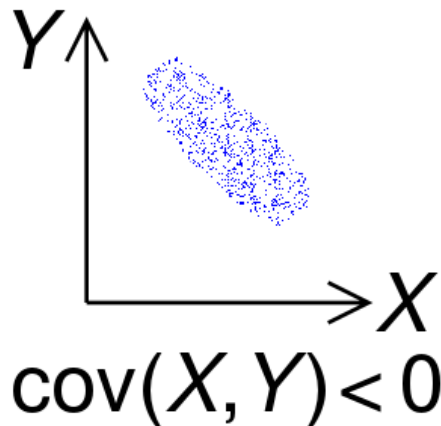
# Bayesiano - Revisão Probabilidade

**Eventos independentes:** estatisticamente a independência entre duas variáveis pode ser medida a partir da covariância entre elas.

$$\text{cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$$

$$E[X] = \sum_{i=1}^{\infty} x_i p(x_i)$$

$$E[X] = \int_{-\infty}^{\infty} x f(x) dx$$



(ex: o coeficiente de Pearson usa a covariância para determinar a correlação entre dados)

# Bayesiano - Revisão Probabilidade

**Probabilidade condicional:** quando um evento depende do outro.

$$P(A|B) = \frac{P(A \text{ e } B)}{P(B)}$$

Diagram illustrating the components of the conditional probability formula:

- $P(A|B)$ : Probabilidade de A, dado que o B ocorreu
- $P(A \text{ e } B)$ : Probabilidade de A e B ocorrerem
- $P(B)$ : Probabilidade de B ocorrer

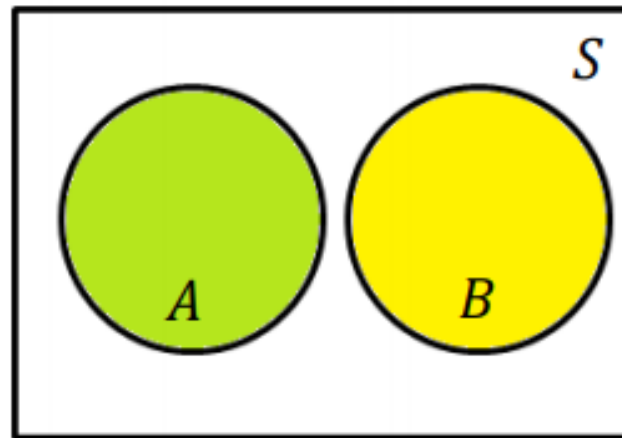
# Bayesiano - Revisão Probabilidade

## Probabilidade condicional:

Se os eventos forem independentes  $P(A \text{ e } B) = P(A) * P(B)$

$$P(A|B) = \frac{P(A \text{ e } B)}{P(B)} = \frac{P(A) * P(B)}{P(B)} = P(A)$$

Eventos  
independentes!



$A$  e  $B$  disjuntos

# Bayesiano - Revisão Probabilidade

## Exemplo: lançamento de dois dados de quatro lado

Neste caso vamos considerar que estamos interessados nos resultados conjuntos. O espaço amostral será:

$$S = \{(1,1); (1,2); (1,3); (1,4); \\ (2,1); (2,2); (2,3); (2,4); \\ (3,1); (3,2); (3,3); (3,4); \\ (4,1); (4,2); (4,3); (4,4)\}$$

A probabilidade de obter cada um dos pares é:

$$P = \frac{1}{16} = P(\text{dado}A) * P(\text{dado}B)$$



# Bayesiano - Revisão Probabilidade

**Exemplo:** lançamento de dois dados de quatro lado

Qual a probabilidade de pelo menos um dos lançamentos serem 1?

$$S = \{(1,1); (1,2); (1,3); (1,4); (2,1); (2,2); (2,3); (2,4); (3,1); (3,2); (3,3); (3,4); (4,1); (4,2); (4,3); (4,4)\}$$

Evento A  $P(A) = \frac{4}{16}$

Evento B  $P(B) = \frac{4}{16}$

$$P(\text{pelo menos 1 lançamento é igual a 1}) = \frac{7}{16}$$

# Bayesiano - Revisão Probabilidade

**Exemplo:** lançamento de dois dados de quatro lado

Qual a probabilidade do evento A ocorrer dado que ocorreu o evento B?

$S = \{(1,1); (1,2); (1,3); (1,4); (2,1); (2,2); (2,3); (2,4); (3,1); (3,2); (3,3); (3,4); (4,1); (4,2); (4,3); (4,4)\}$

Evento A  $P(A) = \frac{4}{16}$

Evento B  $P(B) = \frac{4}{16}$

$$P(A \text{ e } B) = \frac{1}{16}$$
$$P(A|B) = \frac{P(A \text{ e } B)}{P(B)} = \frac{\frac{1}{16}}{\frac{4}{16}} = \frac{4}{16} = \frac{1}{4}$$



# Bayesiano - Teorema de Bayes

Sabemos que:

$$P(A|B) = \frac{P(A \text{ e } B)}{P(B)} \text{ e } P(B|A) = \frac{P(A \text{ e } B)}{P(A)}$$

Podemos isolar  $P(A \text{ e } B)$ :

$$P(A \text{ e } B) = P(A|B)P(B)$$

$$P(A \text{ e } B) = P(B|A)P(A)$$

Assim chegamos no teorema de Bayes:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$



Thomas Bayes (1701-1761)

# Bayesiano - Teorema de Bayes

**Exemplo:** paciente com sintomas faz um teste para saber se está realmente doente. O teste é um **classificador** do tipo que atribui rótulos, **doente** e **não-doente**. Sabemos que ele pode errar e dar **falsos positivos** ou **falsos negativos**.

Vamos considerar:

$A = \{\text{indivíduo com a doença}\}$

$B = \{\text{Saída positiva do nosso modelo}\}$

Probabilidade de ter a  
doença e testar  
positivo

|           | $B$ | $\bar{B}$ |
|-----------|-----|-----------|
| $A$       | TP  | FN        |
| $\bar{A}$ | FP  | TN        |

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

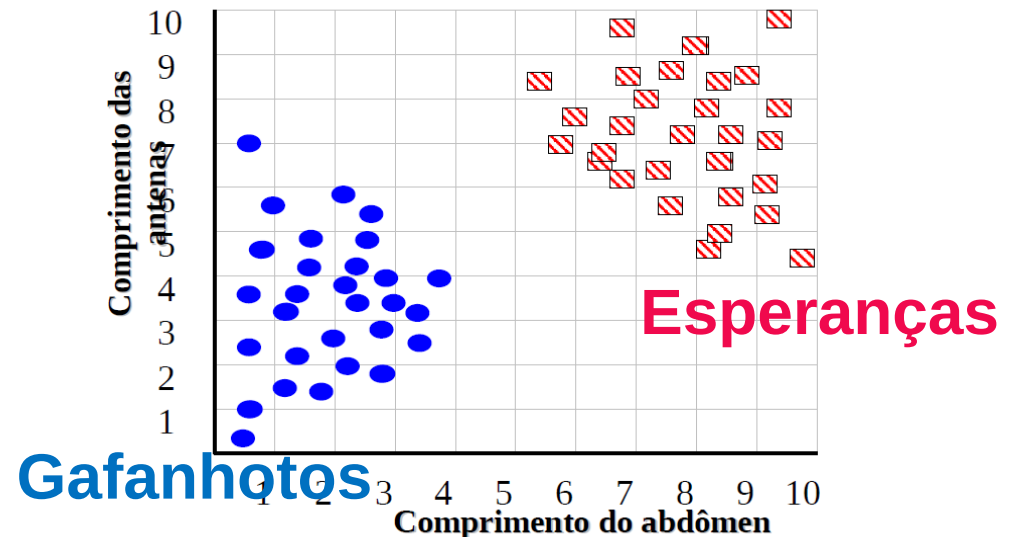
Probabilidade de testar  
positivo e efetivamente estar  
doente

# Bayesiano - Teorema de Bayes

Dado que temos uma classe  $C$ , qual a probabilidade de termos as features nos valores  $f_1, f_2, f_3, \dots, f_n$

$$P(C|f_1, f_2, f_3, \dots, f_n) = \frac{P(C)P(f_1, f_2, f_3, \dots, f_n|C)}{P(f_1, f_2, f_3, \dots, f_n)}$$

Dado as features nos valores  $f_1, f_2, f_3, \dots, f_n$  qual a probabilidade de serem classificadas como pertencente a classe  $C$



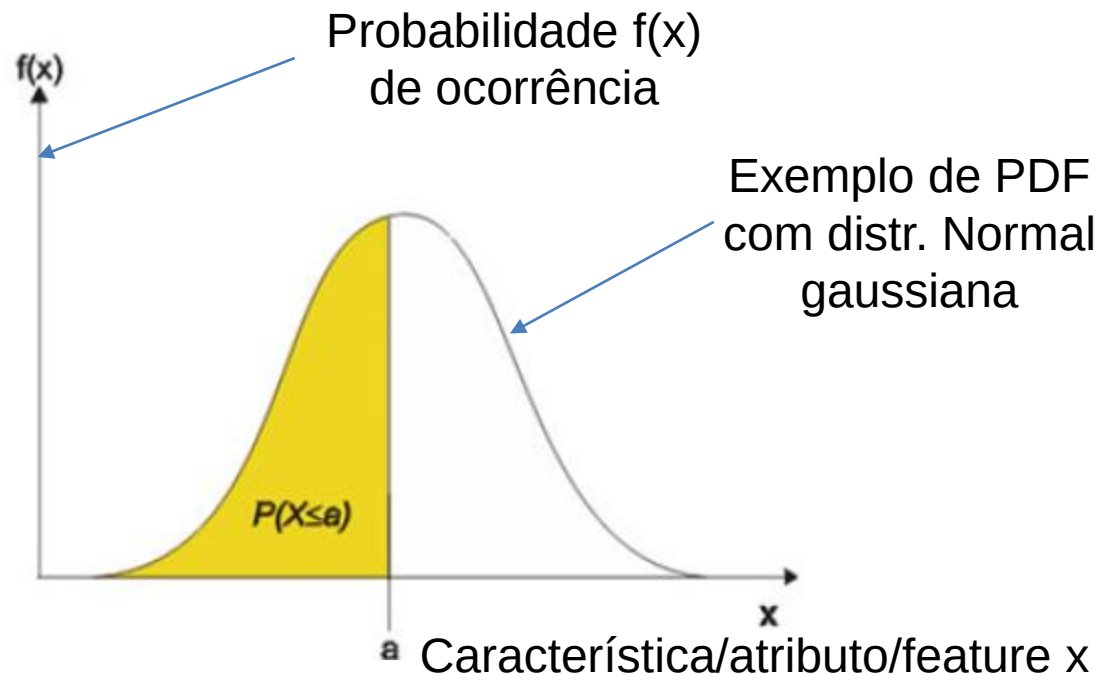
# Classificadores Bayesianos

Os classificadores Bayesianos utilizam a regra de Bayes para fazer as classificações. Para isso utilizam as funções densidade de probabilidade (do inglês, Probability density function ou PDF). Elas são obtidas através de histogramas de distribuição dos dados.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

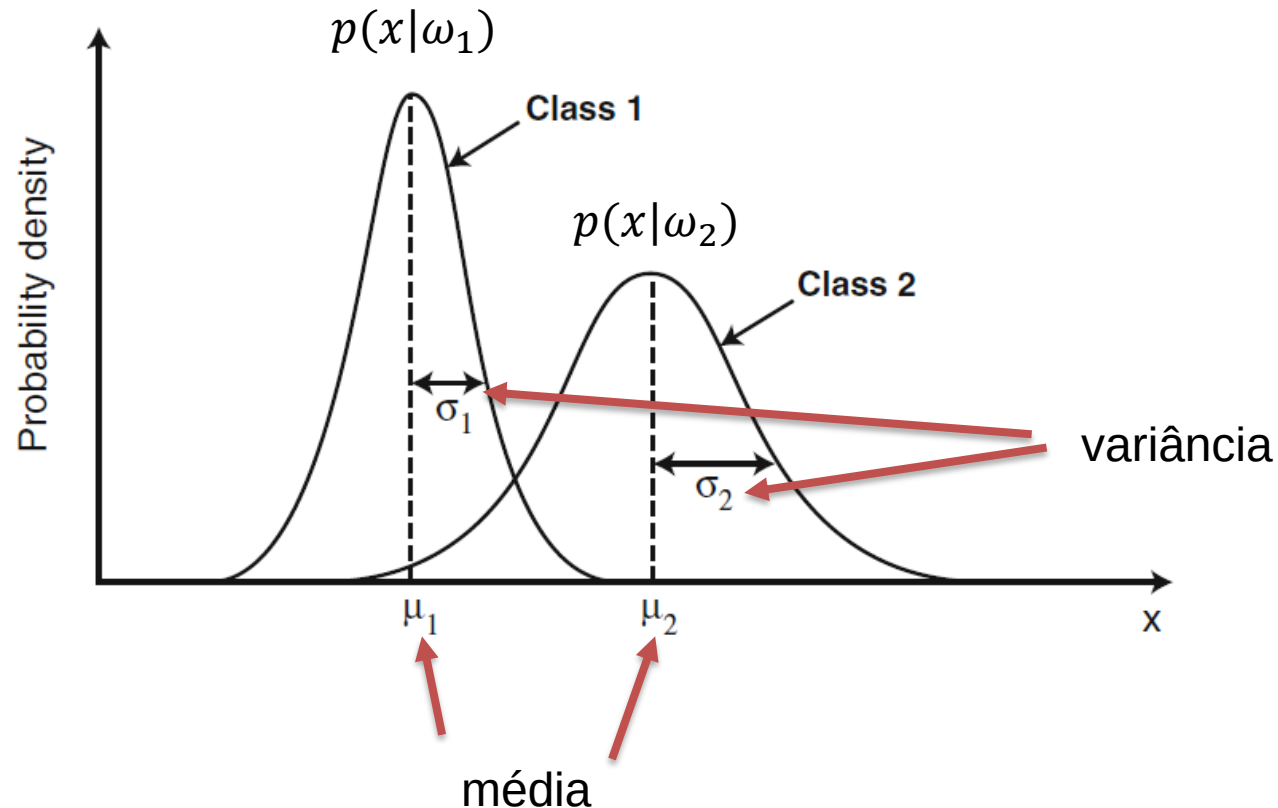
$\mu = \text{média}$

$\sigma^2 = \text{variância dos dados}$



# Classificadores Bayesianos

Exemplo: para o caso de um único atributo  $x_1$  e duas classes  $\omega_1$  e  $\omega_2$ , com os dados de treinamento podemos estimar  $p(x|\omega_1)$  e  $p(x|\omega_2)$ .



# Classificadores Bayesianos

Podemos usar a seguinte regra:

*Se  $p(\omega_1|x) > p(\omega_2|x)$  escolhemos a classe  $\omega_1$*   
Do contrário escolher a classe  $\omega_2$

Reescrevendo com a regra de Bayes:

*Se  $\frac{p(x|\omega_1)p(\omega_1)}{p(x)} > \frac{p(x|\omega_2)p(\omega_2)}{p(x)}$  escolhemos a classe  $\omega_1$*

Do contrário escolher a classe  $\omega_2$

# Classificadores Bayesianos

Rearranjando os termos:

$$\text{Se } \frac{p(x|\omega_1)}{p(x|\omega_2)} > \frac{p(\omega_2)}{p(\omega_1)} \text{ escolhemos a classe } \omega_1$$

Do contrário escolher a classe  $\omega_2$

Supondo que medimos o valor da variância de duas classes e suas variâncias eram de  $\sigma^2 = 1$  e com médias de  $\mu_1 = 4$  e  $\mu_2 = 10$ . As probabilidades a priori eram iguais de  $P(\omega_1) = P(\omega_2) = 0,5$ . Usando o classificador de Bayes:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \rightarrow \frac{\exp(-\frac{1}{2}(x-4)^2)}{\exp(-\frac{1}{2}(x-10)^2)} > \frac{0.5}{0.5} > 1 \quad \text{choose } \omega_1$$

else choose  $\omega_2$

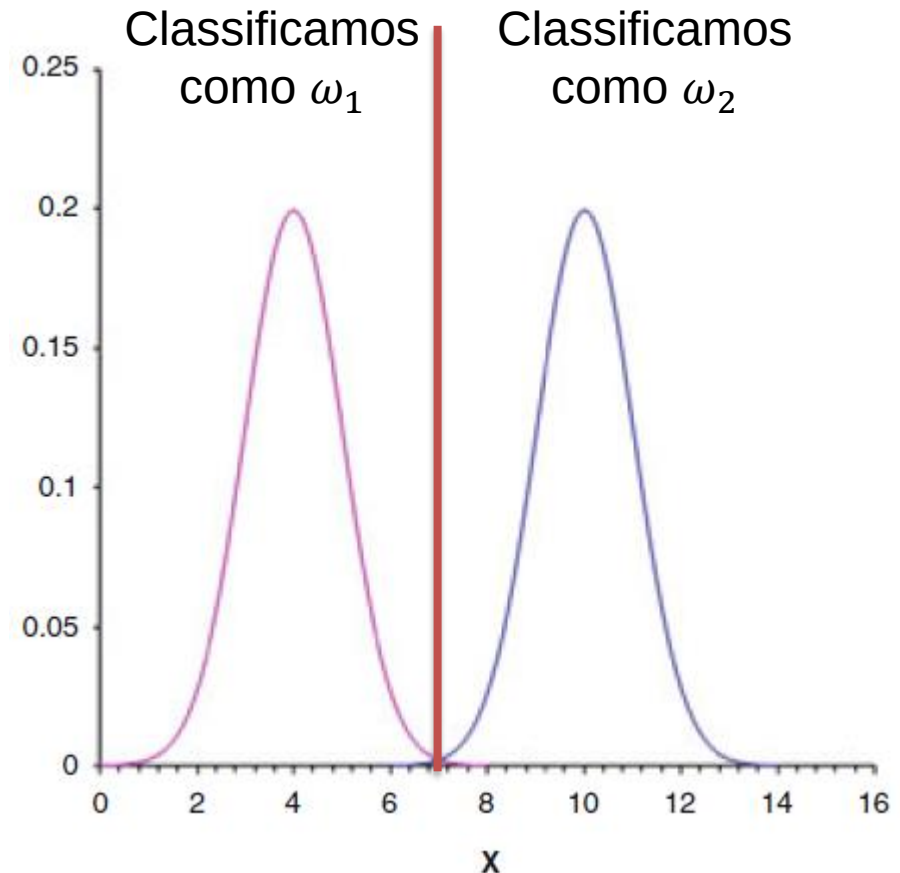
# Classificadores Bayesianos

*Reescrevendo a equação anterior:*

$$(x - 4)^2 - (x - 10)^2 < 0$$

*Resultando em:*

$x < 7$     choose  $\omega_1$   
else    choose  $\omega_2$





# Classificadores Bayesianos

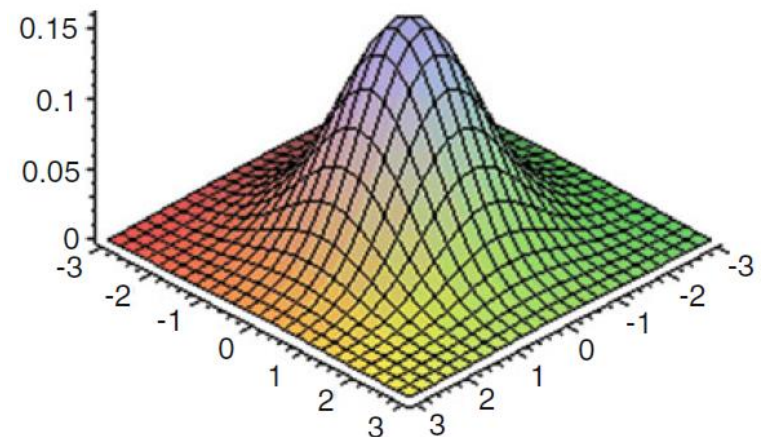
Como proceder quando temos mais features?

Podemos usar a equação para multivariáveis Gaussianas.

$$f(\mathbf{X}) = \frac{1}{((2\pi)^{n/2} |\Sigma|^{1/2})} \exp\left(-\frac{1}{2}(\mathbf{X} - \mu)^T \Sigma^{-1} (\mathbf{X} - \mu)\right)$$

Onde:

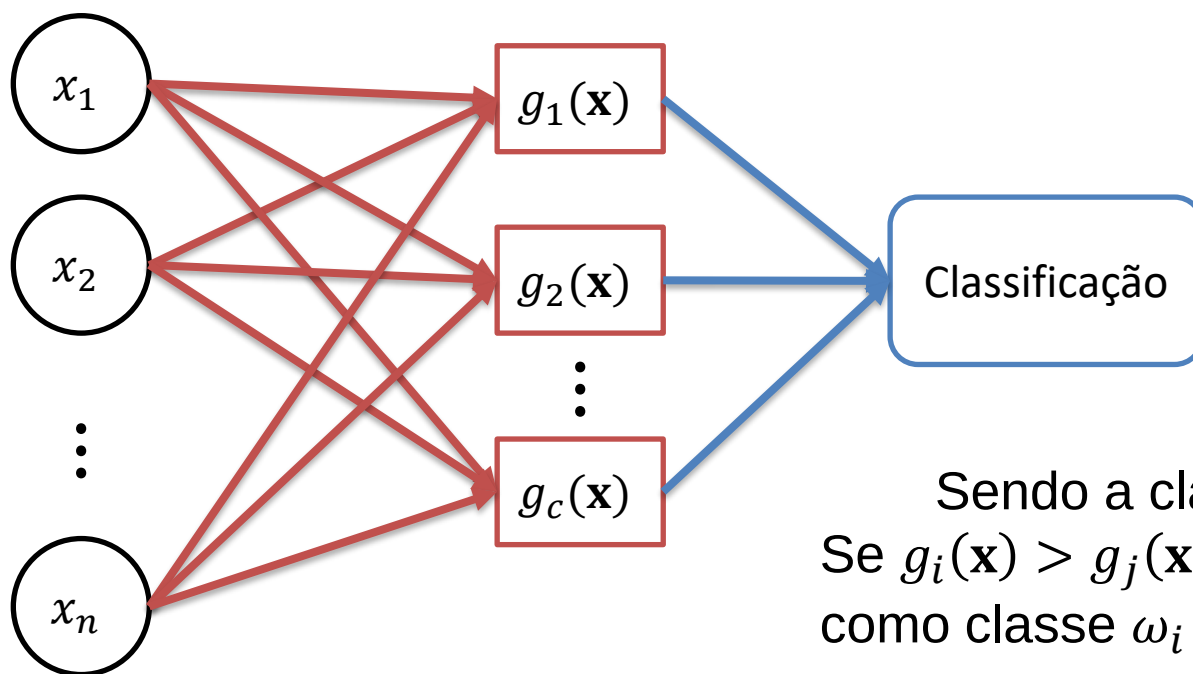
- $\mathbf{X}$  é o vetor com as features
- $\mu$  é o vetor com as médias
- $\Sigma$  é a matriz de covariância



# Bayesiano - Funções Discriminantes

Considerem o caso de múltiplas classes  $\omega_i, i = 1, 2, \dots, c$

Dado um vetor com as nossas features  $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$  podemos criar um sistema de classificação que utiliza as *funções discriminantes*  $g_i(\mathbf{x})$

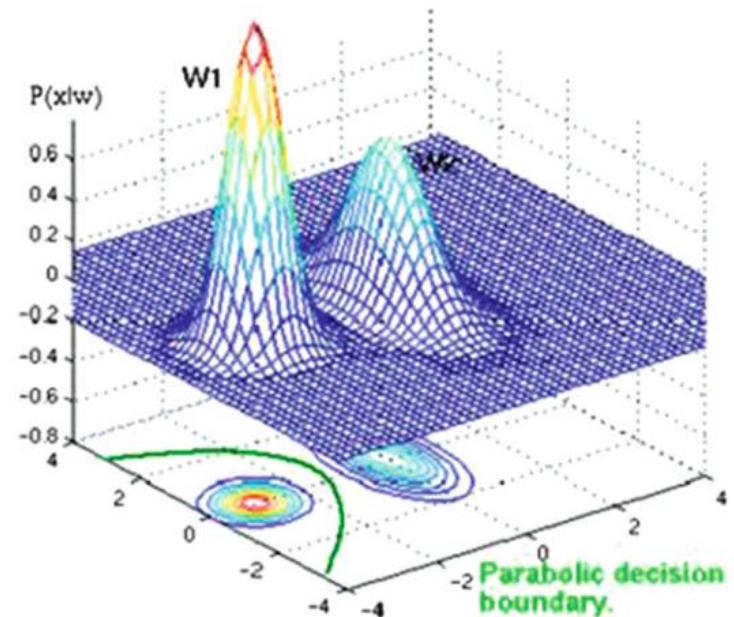
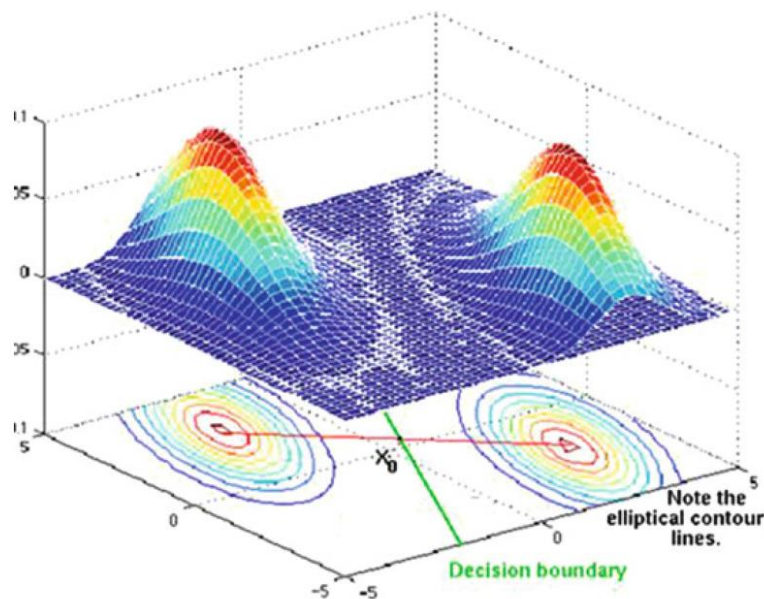


Sendo a classificação dada por:  
Se  $g_i(\mathbf{x}) > g_j(\mathbf{x})$  com  $i \neq j$ , classificar  
como classe  $\omega_i$

# Bayesiano - Funções Discriminantes

Podemos usar a regra de Bayes como função discriminante gaussiana (**gaussian naive bayes - GNB**);

Também podemos definir outras funções discriminantes para o classificador bayesiano, a partir de regras que irão providenciar um discriminante linear (**Linear Discriminant Analysis – LDA**) ou um discriminante quadrático (**Quadratic Discriminant Analysis - QDA**).



## Naive Bayes

### Vantagens:

- É robusto a ruídos isolados (outliers);
- É robusto a atributos irrelevantes;
- Capaz de classificar amostras com valores ausentes;
- Complexidade computacional linear em todas as variáveis do problema;

### Desvantagens:

- Desempenho pode ser afetado pela presença de atributos correlacionados.

- Na tarefa de **Classificação** temos **rótulos descritivos** (variável alvo) e **atributos numéricos observados** para cada classe (variáveis independentes).
- Na **Classificação Bayesiana** vimos como podemos utilizar informações sobre a distribuição dos valores dos atributos numéricos para definir uma função de atribuição de classe (um if/else que atende certo limiar de separação entre as distribuições – **fronteira de decisão**). O método como um todo é baseado nas regras de **probabilidade condicional**. Perceba que neste caso, o rótulo é atribuído pela função if/else que seleciona a classe ao final do algoritmo;
- Se mapeássemos as classes (rótulos) para valores numéricos poderíamos utilizar algum **método de regressão?**

- Vamos pensar no caso da classificação binária como exemplo. Pode ser gatos/cachorros, ou doente/não-doente. Se chamarmos uma classe de 1 e outra de 0, podemos definir uma fronteira de decisão real para o valor predito. Em termos matemáticos:
  - **Atributos numéricos:**  $(x_1, x_2, x_3, \dots, x_n)$ 
    - **Classe alvo:**  $y = [0 \text{ ou } 1], y \in \mathbb{N}$
    - **Classe predita:**  $\hat{y} = [0,1], y \in \mathbb{R}$
- Qual função podemos utilizar para regredir os números  $(x_1, x_2, x_3, \dots, x_n)$  em um intervalo real de 0 até 1?

# Regressão Logística

- Podemos utilizar a função logística:

$$f(\omega) = \frac{1}{1 + e^{-\omega}}$$

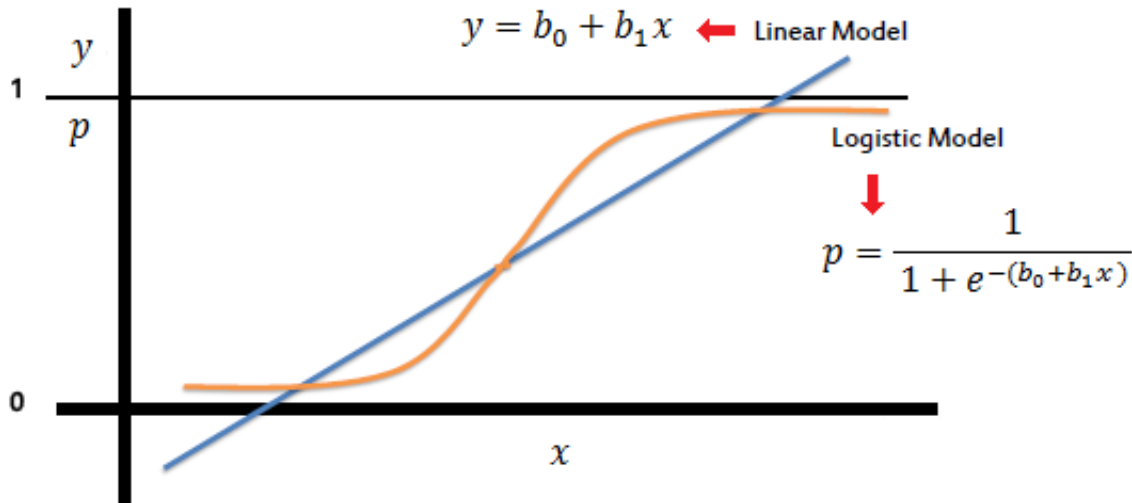
Para o nosso caso é conveniente que  $\omega$  seja uma função linear do nosso atributo  $x$  (para termos os pesos de cada atributo como na regressão linear e polinomial). Então podemos reescrever a função logística como:

$$f(x) = \frac{1}{1 + e^{-(b_0 + b_1 x)}} = \hat{y}$$

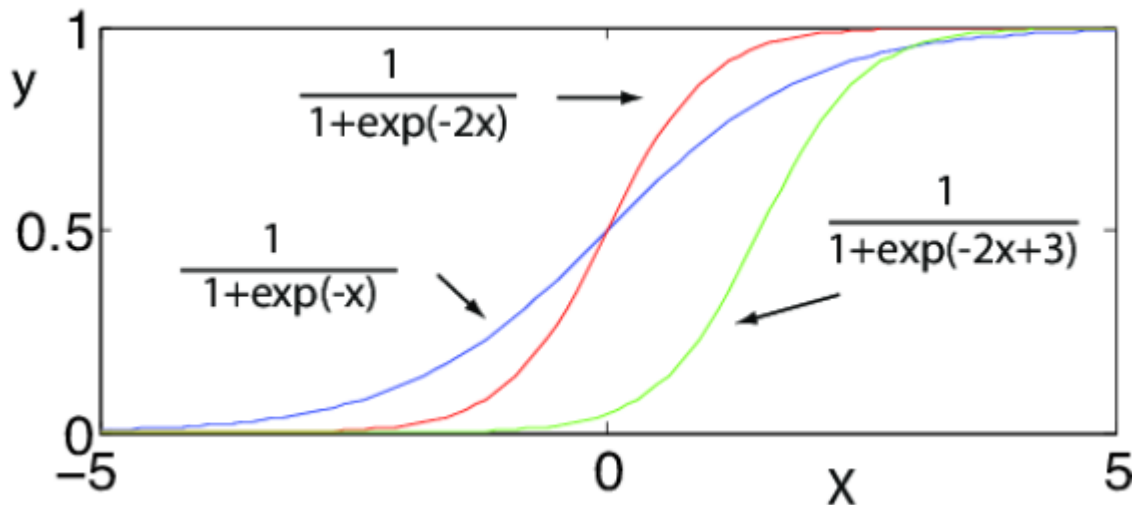
Onde  $b_0$  e  $b_1$  são coeficientes constantes de ajuste do modelo.

# Regressão Logística

- Gráfico da função logística:



Classe será 1 se  $\hat{y} \geq 0.5$  e a classe predita será a 0 se  $\hat{y} < 0.5$



Como encontrar os valores dos coeficientes  $b_0$  e  $b_1$  ?



# Regressão Logística – Função Custo

- Vamos regredir os valores dos coeficientes calculando o erro entre o valor de  $y$  real e o valor de  $\hat{y}$  predito;
- Para fazer isso usamos uma **função custo** (também chamada de função erro);
- ***Custo*** $(y, \hat{y}) = \textit{Erro}(y, \hat{y})$  é uma função para estimar a distância entre o valor real e o valor predito;
- Vamos definir como:

$$\textit{Custo}(y, \hat{y}) = \begin{cases} -\log(\hat{y}), & \text{se } y = 1 \\ -\log(1 - \hat{y}), & \text{se } y = 0 \end{cases}$$

# Regressão Logística – Função Custo

- Lembre-se, isso só é possível pois a regressão é uma tarefa de classificação supervisionada (para parte dos dados, sabemos de antemão qual é a classe, ou seja, qual o valor de  $y$ );
- Vamos calcular o custo médio para uma série de pares  $(y, \hat{y})$ . Para isso vale lembrar que  $y = \{0,1\}$ , de forma que podemos reescrever a função custo como:

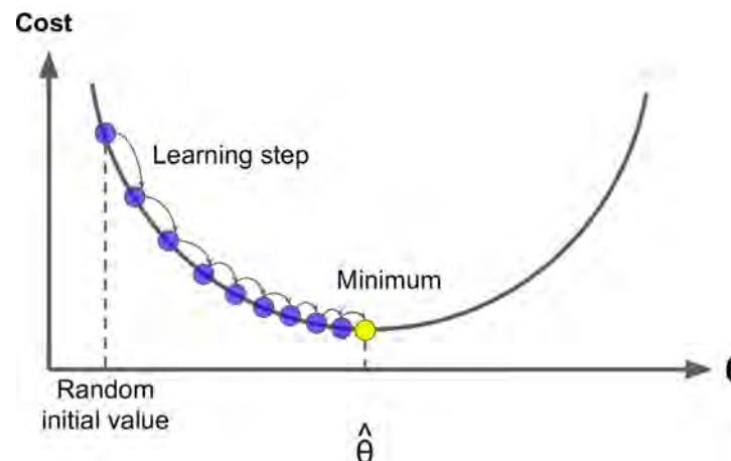
$$Custo(y, \hat{y}) = -(y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y}))$$

- Assim, o custo médio será:

$$E(Custo(y, \hat{y})) = -\frac{1}{n} \sum_i^n (y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i))$$

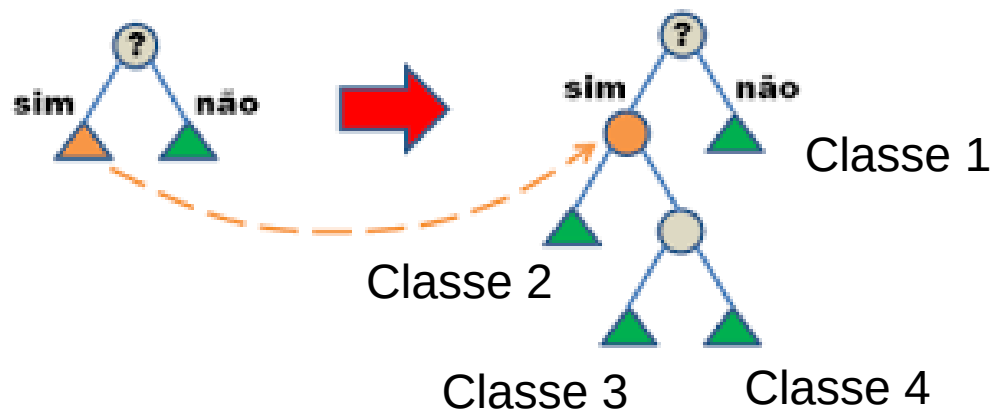
# Regressão Logística – Função Custo

- Por fim, assim como nos métodos de regressão, queremos **minimizar a função custo**, ou seja, encontrar os coeficientes  $b_0$  e  $b_1$  que produzem o menor erro médio;
- Existem vários esquemas de minimização (área da computação/matemática chamada de Otimização). Já falamos de alguns deles, mas para relembrar temos:
  - Máxima verossimilhança (pode ser caro computacionalmente);
  - Método da descida de gradiente (um dos mais usados);



# Regressão Logística - detalhes

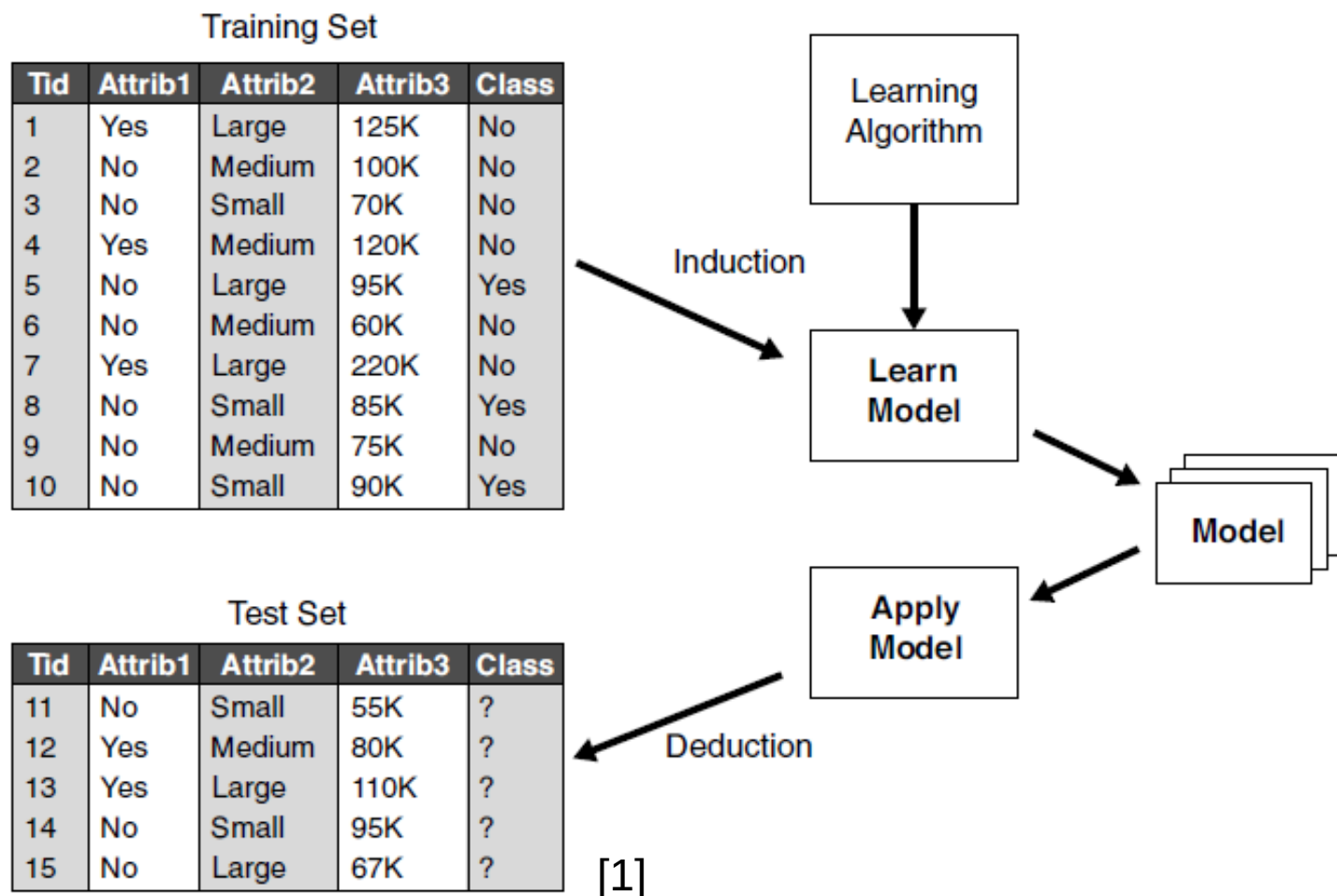
- Para o caso **multivariável** (mais de um atributo), teremos um vetor de atributos numéricos, e para cada um teremos um coeficiente  $b$  adicional.
  - Atributos numéricos:  $(x_1, x_2, x_3, \dots, x_n)$
  - Coeficientes do modelo:  $(b_0, b_1, b_2, b_3, \dots, b_n)$
- Para problemas **multiclasse** (mais do que duas classes) podemos usar vários classificadores binários interligados que selecionam cada uma das classes possíveis.



Detalhe: é preciso fazer todas as combinações possíveis!

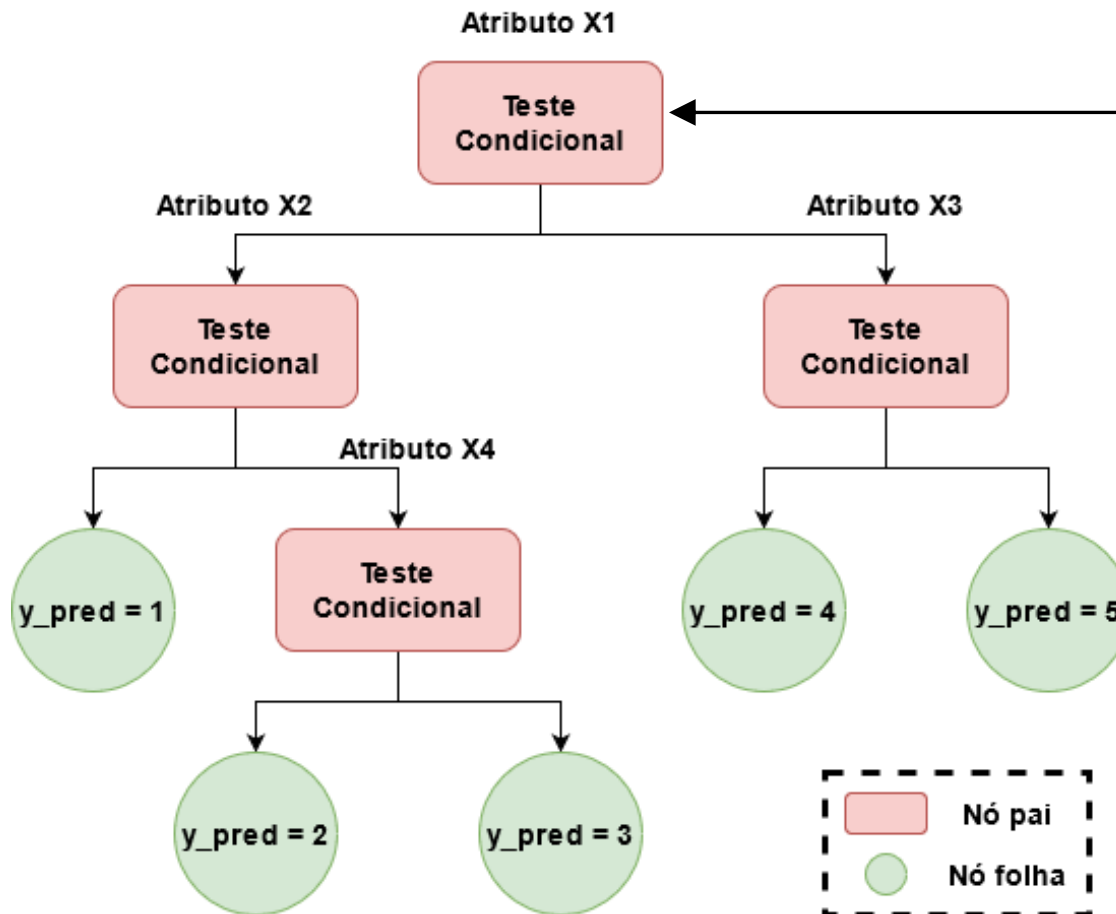
# Árvores de Decisão

O termo “Árvore de Decisão” se refere a um conjunto de técnicas de **Aprendizado de Máquina supervisionado não paramétrico**, que pode ser usado para classificação e regressão:



# Árvores de Decisão

Uma árvore de decisão é uma estrutura recursiva que representa uma lista de regras de decisão (if/else) em forma de uma árvore: **nó folha são as classes previstas** e **nós pais testam um atributo** sobre uma determinada condição.



O primeiro nó de uma árvore também é chamado de **nó raiz**.

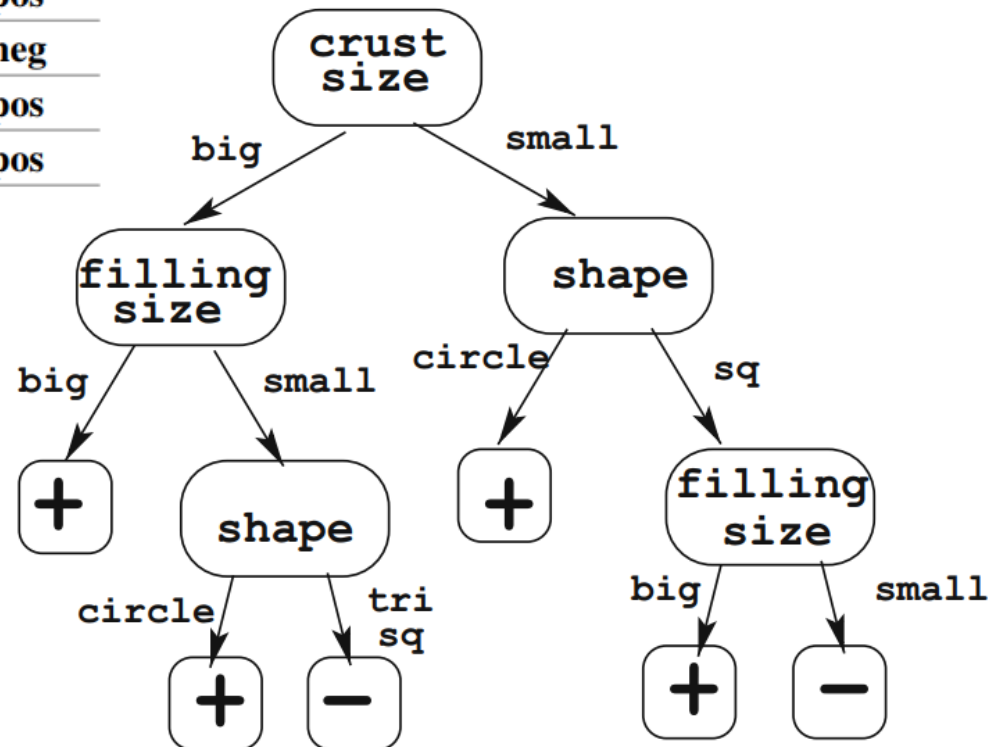
# Árvores de Decisão – Exemplo

| Example   | crust size | shape    | filling size | Class      |
|-----------|------------|----------|--------------|------------|
| <i>e1</i> | big        | circle   | small        | <b>pos</b> |
| <i>e2</i> | small      | circle   | small        | <b>pos</b> |
| <i>e3</i> | big        | square   | small        | <b>neg</b> |
| <i>e4</i> | big        | triangle | small        | <b>neg</b> |
| <i>e5</i> | big        | square   | big          | <b>pos</b> |
| <i>e6</i> | small      | square   | small        | <b>neg</b> |
| <i>e7</i> | small      | square   | big          | <b>pos</b> |
| <i>e8</i> | big        | circle   | big          | <b>pos</b> |

Dados estruturados em uma tabela.

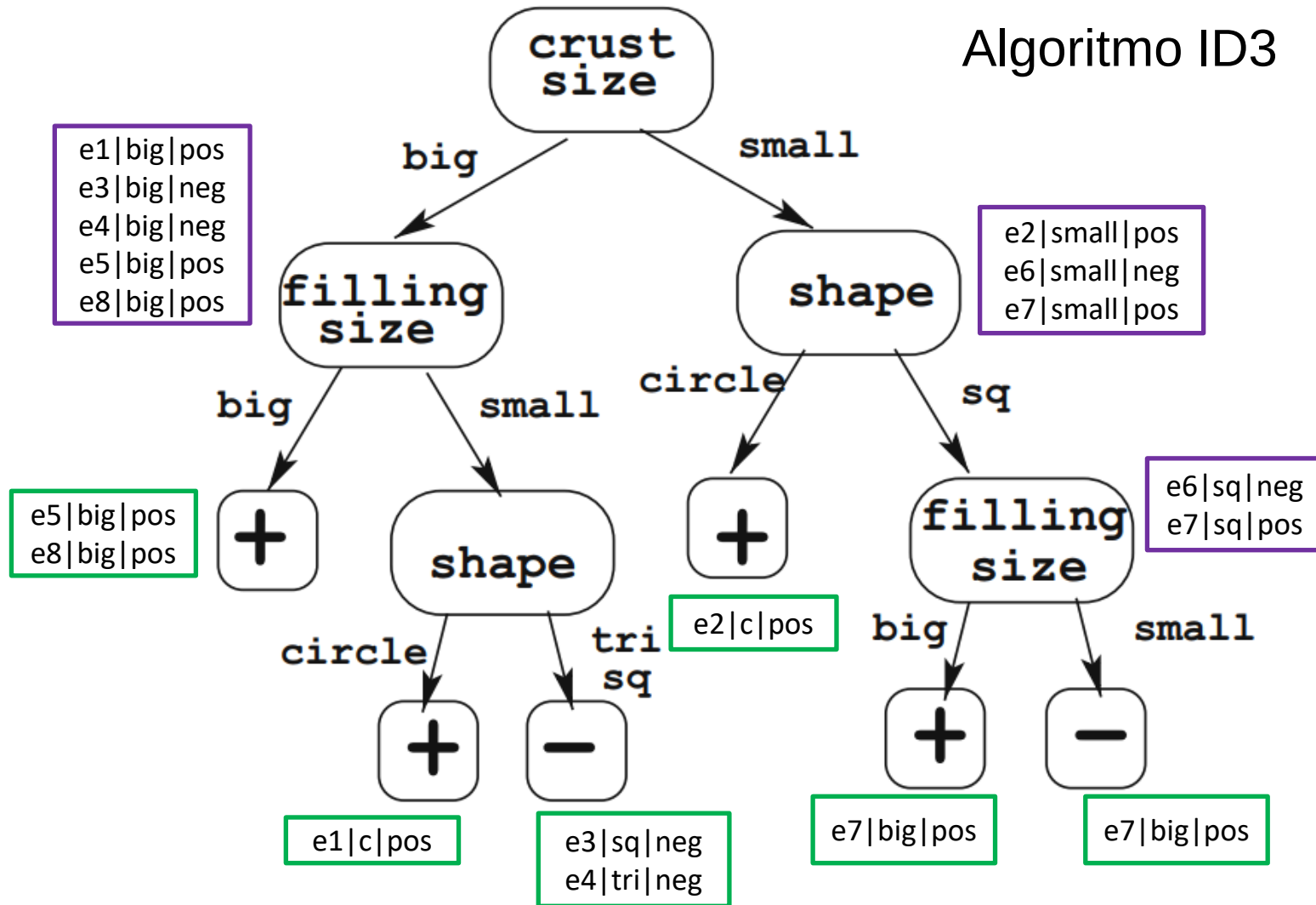
Dados separados nas classes através de uma árvore de decisão.

Algoritmo ID3



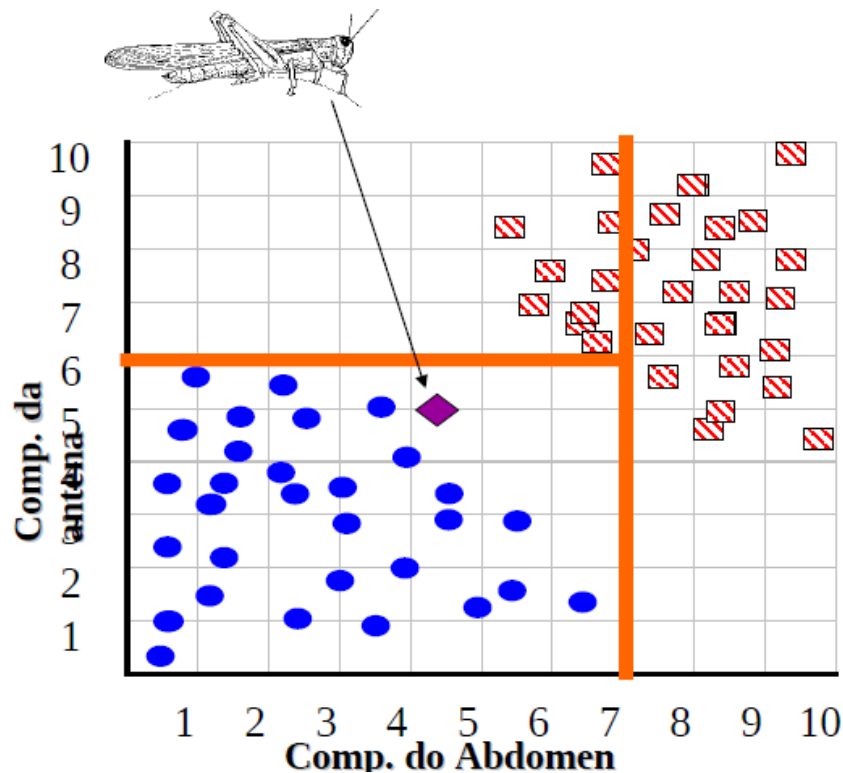
# Árvores de Decisão – Exemplo

Algoritmo ID3

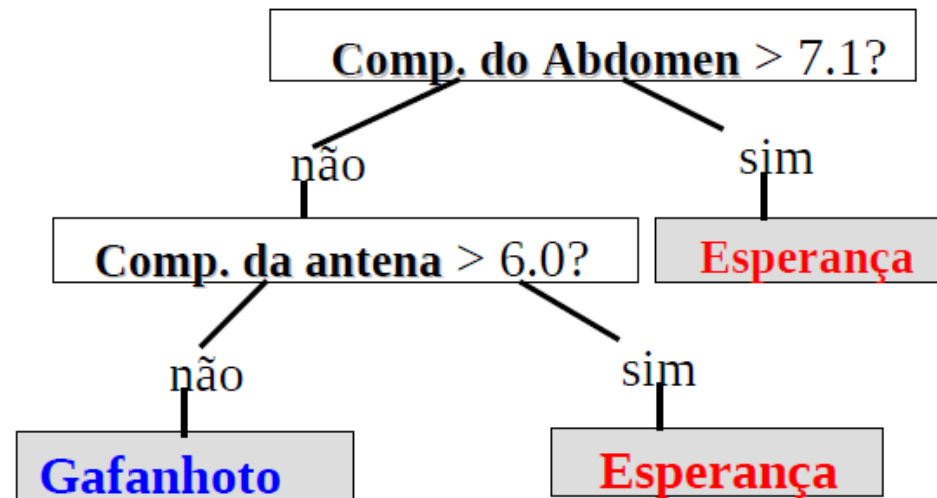




# Árvores de Decisão – Exemplo



## Algoritmo C4.5



Agora que entendemos o que é uma árvore de decisão precisamos entender como criar um Algoritmos de Aprendizado para o modelo, que é o mesmo que responder a pergunta:

**Como construir uma Árvore de Decisão?**

# Árvores de Decisão – Algoritmos

- A árvore é construída de cima para baixo, recursivamente (dividir e conquistar), particionando os dados de treinamento através de testes condicionais;
- No início, todos os exemplos de treinamento do modelo estão na raiz da árvore;
- Exemplos são particionados baseados nos atributos de teste;
- Existem vários algoritmos para implementar uma árvore de decisão:
  - Algoritmo de Hunt-Szymanski (1976): diff (file comparison)
  - 1R: regras usam só um atributo (Holte, 1993);
  - ID3: binária categórica (Ross Quinlan, 1986);
  - C4.5 similar a ID3 porém permite atributos numéricos;
  - CART: similar a C4.5 porém permite regressão;
  - C5.0: última versão que usa menos memória;
- **ScikitLearn: implementa uma versão otimizada do CART**  
(<https://scikit-learn.org/stable/modules/tree.html>)

- Como selecionar os valores limites dos atributos para realizar a melhor partição do dados? Existem varias métricas (chamadas de “medidas de impureza”):
  - **Entropia;**
  - **Índice de Gini;**
  - **Erro de classificação;**
- Se denotarmos  $i$  como a classe alvo entre  $c$  classes possíveis e  $t$  como o nó da árvore,  $p(i|t)$  será a frequência que a classe  $i$  aparece dentro do nó:

$$\text{Entropy}(t) = - \sum_{i=0}^{c-1} p(i|t) \log_2 p(i|t),$$

$$\text{Gini}(t) = 1 - \sum_{i=0}^{c-1} [p(i|t)]^2,$$

$$\text{Classification error}(t) = 1 - \max_i [p(i|t)],$$

# Árvores de Decisão – Particionamento

- Por padrão:
  - CART usa Gini;
  - ID3 e C4.5 usa Entropia;
- No ScikitLearn existe um hiperparâmetro chamada *criterion* onde se pode definir o uso de Gini ou Entropia (default=gini);
- Independentemente da métrica de impureza, em todas as árvores estamos interessados no Ganho de Informação;

*Exemplos com cada atributo após a divisão*

*Entropia ou Gini para cada atributo após a divisão*

$$\text{Gain}(S, A) = \text{Impurity}(S) - \sum_{i=1}^k \frac{|S_{vi}|}{|S|} \text{Impurity}(S_{vi})$$

*Entropia ou Gini antes da divisão dos dados*

*Total de exemplos*

# Árvores de Decisão – Particionamento

- Exemplo:** considere a seguinte tabela de dados. Vamos usar a Entropia como Métrica:

| Examples | Weather | Parents visiting? | Money | Decision (category) |              |
|----------|---------|-------------------|-------|---------------------|--------------|
| 1        | Sunny   | Yes               | Rich  | Cinema              |              |
| 2        | Sunny   | No                | Rich  | Tennis              |              |
| 3        | Windy   | Yes               | Rich  | Cinema              |              |
| 4        | Rainy   | Yes               | Poor  | Cinema              | Cinema: 6x   |
| 5        | Rainy   | No                | Rich  | Stay in             | Tennis: 2x   |
| 6        | Rainy   | Yes               | Poor  | Cinema              | Stay in: 1x  |
| 7        | Windy   | No                | Poor  | Cinema              | Shopping: 1x |
| 8        | Windy   | No                | Rich  | Shopping            |              |
| 9        | Windy   | Yes               | Rich  | Cinema              |              |
| 10       | Sunny   | No                | Rich  | Tennis              |              |

Entropia Inicial:

$$H(S) = -0,6 \times \log_2 0,6 - 0,2 \times \log_2 0,2 - 2 \times 0,1 \times \log_2 0,1 = 1,571$$

# Árvores de Decisão – Particionamento

Determinando o melhor ganho -  $G(S, \text{weather})$ ;

| Examples | Weather | Parents visiting? | Money | Decision (category) |
|----------|---------|-------------------|-------|---------------------|
| 1        | Sunny   | Yes               | Rich  | Cinema              |
| 2        | Sunny   | No                | Rich  | Tennis              |
| 3        | Windy   | Yes               | Rich  | Cinema              |
| 4        | Rainy   | Yes               | Poor  | Cinema              |
| 5        | Rainy   | No                | Rich  | Stay in             |
| 6        | Rainy   | Yes               | Poor  | Cinema              |
| 7        | Windy   | No                | Poor  | Cinema              |
| 8        | Windy   | No                | Rich  | Shopping            |
| 9        | Windy   | Yes               | Rich  | Cinema              |
| 10       | Sunny   | No                | Rich  | Tennis              |

*Sunny:*  
Cinema: 1x  
Tennis: 2x  
Stay in: 0  
Shopping: 0

*Windy:*  
Cinema: 3x  
Tennis: 0  
Stay in: 0  
Shopping: 1x

*Rainy:*  
Cinema: 2x  
Tennis: 0  
Stay in: 1x  
Shopping: 0

$$\begin{aligned}\text{Gain}(S, \text{weather}) &= 1.571 - (|S_{\text{sun}}|/10) \times \text{Entropy}(S_{\text{sun}}) - (|S_{\text{wind}}|/10) \\ &\quad \times \text{Entropy}(S_{\text{wind}}) - (|S_{\text{rain}}|/10) \times \text{Entropy}(S_{\text{rain}}) \\ &= 1.571 - (0.3) \times (0.918) - (0.4) \times (0.8113) - (0.3) \\ &\quad \times (0.918) = 0.70\end{aligned}$$

# Árvores de Decisão – Particionamento

Determinando o melhor ganho -  $G(S, \text{parents})$ ;

| Examples | Weather | Parents visiting? | Money | Decision (category) |
|----------|---------|-------------------|-------|---------------------|
| 1        | Sunny   | Yes               | Rich  | Cinema              |
| 2        | Sunny   | No                | Rich  | Tennis              |
| 3        | Windy   | Yes               | Rich  | Cinema              |
| 4        | Rainy   | Yes               | Poor  | Cinema              |
| 5        | Rainy   | No                | Rich  | Stay in             |
| 6        | Rainy   | Yes               | Poor  | Cinema              |
| 7        | Windy   | No                | Poor  | Cinema              |
| 8        | Windy   | No                | Rich  | Shopping            |
| 9        | Windy   | Yes               | Rich  | Cinema              |
| 10       | Sunny   | No                | Rich  | Tennis              |

Yes:  
*Cinema: 5x*  
*Tennis: 0*  
*Stay in: 0*  
*Shopping: 0*

No:  
*Cinema: 1x*  
*Tennis: 2x*  
*Stay in: 1x*  
*Shopping: 1x*

$$\begin{aligned} \text{Gain}(S, \text{parents}) &= 1.571 - (|S_{\text{yes}}|/10) \times \text{Entropy}(S_{\text{yes}}) - (|S_{\text{no}}|/10) \\ &\quad \times \text{Entropy}(S_{\text{no}}) = 1.571 - (0.5) \times 0 - (0.5) \times (1.922) = 0.61 \end{aligned}$$

# Árvores de Decisão – Particionamento

Determinando o melhor ganho -  $G(S, \text{money})$ ;

| Examples | Weather | Parents visiting? | Money | Decision (category) |
|----------|---------|-------------------|-------|---------------------|
| 1        | Sunny   | Yes               | Rich  | Cinema              |
| 2        | Sunny   | No                | Rich  | Tennis              |
| 3        | Windy   | Yes               | Rich  | Cinema              |
| 4        | Rainy   | Yes               | Poor  | Cinema              |
| 5        | Rainy   | No                | Rich  | Stay in             |
| 6        | Rainy   | Yes               | Poor  | Cinema              |
| 7        | Windy   | No                | Poor  | Cinema              |
| 8        | Windy   | No                | Rich  | Shopping            |
| 9        | Windy   | Yes               | Rich  | Cinema              |
| 10       | Sunny   | No                | Rich  | Tennis              |

*Rich:*  
*Cinema: 3x*  
*Tennis: 2x*  
*Stay in: 1x*  
*Shopping: 1x*

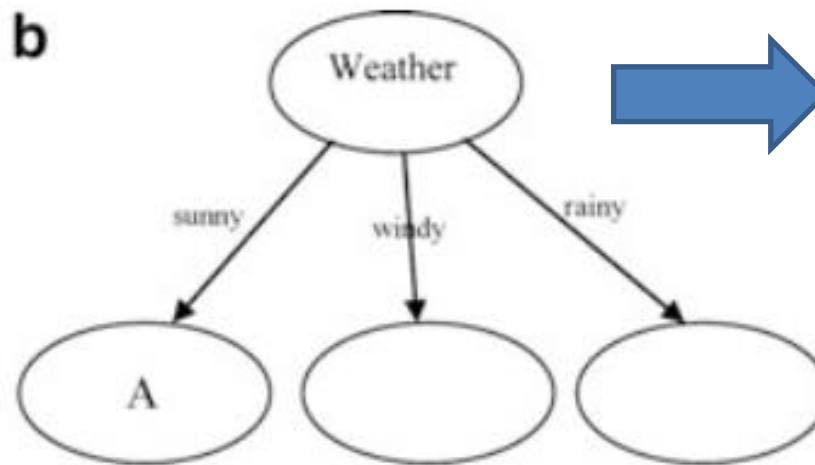
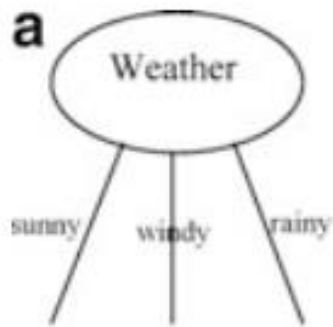
*Poor:*  
*Cinema: 3x*  
*Tennis: 0*  
*Stay in: 0*  
*Shopping: 1x*

$$\begin{aligned}\text{Gain}(S, \text{money}) &= 1.571 - (|S_{\text{rich}}|/10) \times \text{Entropy}(S_{\text{rich}}) \\ &\quad - (|S_{\text{poor}}|/10) \times \text{Entropy}(S_{\text{poor}}) \\ &= 1.571 - (0.7) \times (1.842) - (0.3) \times 0 = 0.2816\end{aligned}$$



# Árvores de Decisão – Particionamento

Entre os três primeiros, Weather tem o maior ganho. Logo ele deve ser o primeiro ramo.



Procedimento se repete agora para cada ramo até classificar todos os itens

- Vantagens:
  - Fácil de entender a ideia geral;
  - Fácil de gerar regras;
- Desvantagens:
  - Sensível a ruídos;
  - Pode apresentar overfitting (principalmente quando há muitos atributos);
  - Particionamentos retangulares (não trata características correlatas muito bem);
  - Podem ser muito grandes (consumo de memória e tempo) e técnicas de poda devem ser aplicadas (aumento da sofisticação do algoritmo);

Dicas práticas (avanzadas) de utilização podem ser vistas aqui: <https://scikit-learn.org/stable/modules/tree.html>

- A melhor forma de fazer seria testar todas as árvores possíveis (todas as sequências de particionamento, por exemplo). Entretanto isso é proibitivo do ponto de vista computacional (consumo de memória e de tempo);
- Por isso, todos os algoritmos e métodos discutidos possuem um componente aleatório (por exemplo para iniciar os cálculos);
- Para evitar **overfitting** as árvores usam técnicas de poda:
  - **Poda prévia**: parar a construção da árvore cedo dado um limiar (difícil escolher o valor de limiar apropriado);
  - **Poda posterior**: remover ramos depois da árvore estar pronta usando outro conjunto de dados;
- Uma forma interessante de melhorar a performance de um Algoritmo de Árvore de Decisão é criar um **conjunto de árvores possíveis e usar a moda das predições** de todas as árvores. Isso é chamado de **RandomForest**.

# Referências

- [1] Scikit Learn, <https://scikit-learn.org/stable/>
- [2] Cohen's Kappa Interpretation, <https://www.datanovia.com/en/blog/kappa-coefficient-interpretation/>
- [3] Cohen's Kappa: what it is, when to use it, how to avoid pitfalls  
<https://www.knime.com/blog/cohens-kappa-an-overview>
- [4] Pang-Ning Tan – Introduction to Data Mining, Springer 2013.
- [5] Mirsolav Kubat, An Introduction to Machine Learning, Springer 2017.
- [6] Ronaldo Prati, Mineração de Dados, UFABC 2021.
- [7] Miguel Bozer, Inteligência Artificial e Computacional, FIAP 2021.
- [8] Algoritmos de Árvores, <https://medium.datadriveninvestor.com/tree-algorithms-id3-c4-5-c5-0-and-cart-413387342164>
- [9] Gini vs Entropia, <https://towardsdatascience.com/gini-index-vs-information-entropy-7a7e4fed3fcb>
- [10] Árvores de Decisão <https://scikit-learn.org/stable/modules/tree.html>
- [11] Classificador por Árvore de Decisão <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
- [12] Classificador por RandomForest <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

- [13] DOUGHERTY, Geoff. **Pattern Recognition and Classification**: an introduction. New York: Springer International Publishing, 2013.
- [14] GÉRON, Aurélien. Hands-On Machine Learning with Scikit-Learn and TensorFlow. Sebastopol: O'reilly Media, 2017
- [15] KUBAT, Miroslav. **An Introduction to Machine Learning**. 2. ed. Ebook: Springer International Publishing, 2017.
- [16] Burkov, A. Machine Learning Engineering. True Positive Inc., 2020.
- [17] H. Ferreira, et al. Introduzindo aprendizado de máquina em cursos de física: o caso do rolamento no plano inclinado, Revista Brasileira de Ensino de Física, 2022.

Copyright © 2023 Prof. Henrique Ferreira dos Santos

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).